The CIRIS Constitution: The Dictionary

The Dictionary, a volume of edition 1.0-rc13

Stewarded by Eric Moore

The Dictionary

What this is. The namespace: every dimension family a row may carry, who owns it, its polarity, whether it is reserved, and the rules of the version segment and the case rule (3.1); the subject kinds and their payload shapes (3.3); the reserved prefixes and who may emit under them (3.4); and the relation precedence (3.5).

Whom it binds. Every producer that names a dimension and every substrate that admits one. A family that is not in the registry is refused as unregistered; a reserved prefix emitted by the wrong key is refused by name.

What it promises. That the family count is generated, not asserted: `manifests/namespace_registry.json` is built from this volume's text, its hash is pinned by every ratifying repository, and the prose and the manifest cannot disagree without the build failing. A matcher and a vector set accompany it so two implementations resolve the same dimension the same way.

What it leaves out. Section 3.2, the trust root and its standing verdict, which prints in *The Constitution*; the envelope the dimension rides in, which is *The Wire*; the composition of scores into a verdict, which is *The Reader's Policies*; and the powers that some reserved families carry, which are explained in *The Constitution*.

Where the rest lives. The authoritative form of this volume is the generated registry and its match vectors. Read the prose for the reasons and the machine form for the rows; when they differ, the build is broken and the manifest is what shipped.

Part 3 — The Namespace

3.1 The dimension namespace

A federation needs a shared vocabulary before it can have a shared judgment. The dimension namespace is that vocabulary: the union of what every sibling component commits to, each family registered once under one owning component. CEG does not author the namespace top-down. It owns its own slice (CC 3.1.1) and consumes everyone else's, so that justice — fair access to capability — and integrity — auditable claims — rest on names every participant agrees on. The family count is **generated, not asserted**: the namespace generator extracts every leaf family from this Part into `manifests/namespace_registry.json`, whose `_meta` carries the totals, the per-component breakdown and the hash of the source it parsed — read the count there, never from a number restated in prose. **Eight** owning components commit to their families in their own mission statements; **CIRISBench** (CC 3.1.10) is catalogued without one and is the ninth.

This section catalogs every prefix family, organized by owning component, each under the component that commits to the concept.

3.1.1 CIRISRegistry: identity / build / license / partner

Steward: CIRISRegistry, folding into CIRISServer as the conferred registry slice (the catalogue row's `owning_repo` follows the fold, per CC 3.1.7 R1).

Prefix	Description	Polarity	Reserved?
`licensure:{authority_id}`	License status for a key under a named authority (`authority_id` is the licence's **provenance** — CC 2.4.1.2.1). Status vocabulary (open per CC 4.5.1.1); canonical: `issued` · `probation` · `restricted` · `suspended` · `revoked` · `lapsed` · `surrendered` · `reduced`. **`suspended` is reversible; `revoked` is terminal** — the one operational distinction a consumer MUST honour, and a status a consumer MUST NOT infer from the other. **More than one status MAY be live concurrently** (active-on-probation is two facts, not one blended state), so a consumer composes the set rather than reading a scalar. Co-stewarded with Verify for the CIRIS-issued licence (CC 3.4.9) — composition, **not** a reservation: the family is open-emitter.	signed	No

Prefix	Description	Polarity	Reserved?
`duty:{kind}`	**An obligation attached to a permission** — the thing a permission-and-prohibition grammar cannot express, and the thing every widely-used content licence rests on (attribution, share-alike, carry-restrictions-downstream). `{kind}` open vocabulary per CC 4.5.1.1; canonical kinds follow ODRL 2.2: `compensate` · `attribute` · `inform` · `obtain_consent` · `ensure_exclusivity` · `delete` · `next_policy` · `anonymize` · `review_policy`. A duty **states** the obligation; its **discharge** is evidenced by the existing `commitment_fulfillment:{prior_contribution_id}` (CC 3.1.9.2) rather than a second family, and its **breach** routes to `hard_case:duty_unmet` (CC 3.1.9.4) — adjudication, never automatic `slashing:*`. Rides `scores`; no new structural primitive.	signed	No
`partner_role:{role}`	Partner status (`community` / `community_plus` / `professional_medical` / `professional_legal` / `professional_financial` / `professional_full` — lowercase per CC 3.1.7 R3).	enumerated	No
`revocation:{entity_type}:{reason}`	Entity revocation (`agent` / `partner` / `license`). Immediate, non-rollbackable.	-1 only	No
`bond_posted:{currency}`	A bond posted for Sybil resistance (proof of bond); forfeited on revocation.	positive-only	No
`build:registered:{target}`	Build manifest registered against the directory (precondition for L4 attestation).	boolean-via-score	No
`multilateral_participation:{forum}:{kind}`	Depth of a partner's participation across federated bodies. `{forum}` = named federated body or compact; `{kind}` ∈ `membership` | `voting` | `proposal_filing` | `observer_status`.	signed	No
`agent_files:{kind}:{platform_or_target}`	**Joint claim with CC 3.1.9.1 NodeCore.** Canonical-attester rule: attestations by the accord-conferred **steward keys** of `ciris-canonical` (CC 3.2; human-held, never the installs') constitute the CIRIS canonical default-trust state. Anti-tricking guarantee at `registry.ciris-services-1.ai/install` per CC 4.4.3.7 trust-composition policy. Open Contribution channel; consumer policy composes via CC 4.4.3.7 trust layers.	signed	No
`ownership:{relation}:{target_kind}:{version}`	The owner-binding made observable as a score. Canonical leaf `ownership:responsible_party:node:v1` — names the single live responsible steward of an owned `node/agent` key. Mirrors, never replaces, the CC 3.2 single-owner `delegates_to(user → key, delegation_purpose: owner_binding)`: cardinality ≠ 1 is an error, not a set to reduce. Reclaim/seizure ride the CC 3.2 recovery path.	boolean-via-score	**Yes — CC 3.4.5**

Prefix	Description	Polarity	Reserved?
`device:label`	**The name a person gives one of their own devices** (not `self:device_label`: `self` is a cohort scope and names no family). A `scores` row signed by the owner about an identity occurrence the owner holds, at `cohort_scope: self`, the name in the envelope's `label`; a rename is a `supersedes` of the previous row and the newest wins. It is display text: it confers nothing, no gate reads it, and it never leaves the owner's self cohort. The `device:` stem is closed to this one leaf.	positive-only	**Yes:** owner-signed, about an occurrence the signer owns; `self` scope only
`takedown_notice:{basis}`	**A takedown notice as a row** (shape and bases at CC 3.3.2): the `scores` row whose envelope is the `takedown_notice` payload, `subject_kind: takedown_notice`, so that every notice has an `attestation_id` a tombstone cites and a `reconsideration:{grounds}` targets (CC 4.5.5). One family across bases; `{basis}` is the **closed** `LegalBasis` wire string: `{basis}` ∈ `dmca512` | `dsa_article16` | `tvec_terrorist` | `ncmec_csam` | `gifct_cip` | `community_standards` | `perceptual_hash_csam` | `osa_illegal_content` | `avmsd_age_inappropriate` | `court_order` | `steward_purge`. A `steward_purge` notice is a **cosigned row** (CC 2.4): `attesting_key_id` is the seat that proposes it — any seat, so no single holder's reachability is load-bearing — the other seats ride in `additional_scrubs[]` over the same canonical bytes, the quorum is evaluated over attester + cosigners under the root's `consensus_protocol`, and the family is derived from the verified signers and MUST resolve to the `claimant_key_id` root (CC 3.4.7); the purge's signing payload is this row's canonical bytes. The `takedown_notice:` stem is closed to this family.	-1 only	**Yes:** duty-holder-signed under the CC 4.5.5 enforced-admission rule for the legal bases; cosigned by the trust root for `steward_purge`
`community:listing`	**A public community's own discovery row** (CC 3.2 `public`): a federation-scoped `scores` row by the founder carrying the room's name, description, `cohort_subkind`, admission policy, named moderator and `community_key_id`, ranked by NodeCore votes. Wire `community:listing:v1`. It is the community's row and it names no roster: there is no member-level listing, and a roster is never enumerable outside its cohort. A rename or policy change is a `supersedes`; closing the room is a `withdraws`. The `community:` stem is closed to this leaf.	positive-only	**Yes:** signed by the founder (or a current `moderate`-holder) of the community it names; one live row per community

Prefix	Description	Polarity	Reserved?
`trust:{job}:{version}`	The trust-root conferral and acceptance edges, carried as `dimension` values on `delegates_to` rows — the wire type is unchanged, so 1+4 is untouched. Three canonical jobs, named apart so a reader never infers one from another: `trust:charter:v1` (root→root self-declaration — the capability ceiling of the trust domain), `trust:confers:v1` (root→subject grant), `trust:accepts:v1` (node→root acceptance — the one-row un-trust lever). Retracted by `withdraws`/`recants`, never by a negative score. Conferral planes, the acceptance-edge rule and the liveness banding: CC 3.2.	positive-only	**Yes — CC 3.2**
`accord:invoke:constitutional:{halt_id}`	Catalogue row (CC 3.1.7 R1); the kill-switch invocation, semantics and canonical bytes at CC 3.4.1 / CC 4.2.1.1. Lowercase per R3 (not `CONSTITUTIONAL`).	+1.0 only	**Yes — CC 3.4.1**
`accord:invoke:notify:{notify_id}`	Catalogue row (CC 3.1.7 R1); federation-wide accord-holder communication, CC 3.4.1.	+1.0 only	**Yes — CC 3.4.1**
`accord:invoke:drill:{drill_id}`	Catalogue row (CC 3.1.7 R1); accord-holder exercise, CC 3.4.1.	+1.0 only	**Yes — CC 3.4.1**
`accord:lifecycle`	Catalogue row (CC 3.1.7 R1); the accord heartbeat, wire `accord:lifecycle:v1`, CC 3.4.1.	+1.0 only	**Yes — CC 3.4.1**
`accord:lifecycle:confirmed`	Catalogue row (CC 3.1.7 R1); a strict majority's confirmation of a one-holder `constitutional` halt within its fuse, CC 4.2.1.1 / CC 4.2.1.3.	+1.0 only	**Yes — CC 3.4.1**
`accord:lifecycle:active`	Catalogue row (CC 3.1.7 R1); resumption from a `constitutional` halt, CC 4.2.1.3.	+1.0 only	**Yes — CC 3.4.1**
`accord:human_dignity`	Catalogue row (CC 3.1.7 R1); the accord-holder verdict on an agent's human-dignity conduct, CC 3.4.1.	signed	**Yes — CC 3.4.1**
`accord:*`	**Reserved** — only `identity_type=accord_holder` may emit. **Closed**: the rows above are its whole leaf set. One of several reserved families in this table alone (`ownership:*`, `trust:*`), and more at CC 3.4; the generated registry is the count of record. What is distinctive is the *keying*: this is the one reservation held by a **closed roster slot** rather than by a role conferred on the delegation plane, so no root can confer it. The roster occupying the slot is an instance parameter per the CC 1 reading rule.	see CC 3.4.1	**Yes — CC 3.4.1**

3.1.2 CIRISVerify: attestation ladder, provenance, transparency

Steward: CIRISVerify. These prefixes serve integrity: each is a checkable claim about how far a build, key, or license has climbed the trust ladder.

Prefix	Description	Polarity
`attestation:self_verify`	Running CIRISVerify binary attests itself against its function manifest. (Consumer-side ladder: corresponds to L1; see CC 4.4.3.6 Policy I.)	boolean-via-score
`attestation:hardware_rooted`	Hardware-rooted attestation (TPM 2.0 / Android Keystore / iOS Secure Enclave). (Ladder L2.)	boolean-via-score
`attestation:registry_consensus`	2-of-3 multi-source registry consensus on key / build / license validity. (Ladder L3.) An `Indeterminate` consensus is no row, never a third point; a consumer resolves the absence fail-secure (RESTRICTED).	boolean-via-score
`attestation:license_validity`	License-validity claim (Registry-signed, Verify-verified). (Ladder L4.)	boolean-via-score
`attestation:agent_integrity`	Agent source-tree byte-equal against registered manifest. (Ladder L5.)	boolean-via-score
`provenance:slsa:{level}`	SLSA build provenance levels 1-3. Registry emits these on build registration; Verify consumes them as `AttestBundle.provenance.slsa_level`.	boolean-via-score
`provenance:build_manifest:{target}`	Per-target canonical-staged-runtime manifest hash equality. Each `BuildManifest` is hybrid-signed (Ed25519 + ML-DSA-65) by the per-primitive steward.	boolean-via-score
`provenance:build_manifest:{target}:locale:{lang_code}`	Per-locale signed sub-manifest within a target's manifest tree. Parent target manifest is Merkle root over per-locale leaves. A non-power-of-two leaf count is padded by repeating the last leaf to the next power of two — not RFC 6962's split, and not the per-level duplication of CC 6.1.1; the leaves and parents carry RFC 6962's `0x00` / `0x01` prefixes, and the list is one leaf per locale. Detection surface for locale-targeted attacks. Canonical-bytes spec at CC 3.1.2.1.	boolean-via-score
`provenance:skill_import:{source}`	Community-skill import provenance. `{source}` ∈ `registry:{registry_id}` | `direct:{url}` | `local:{path}`. Envelope: `{skill_manifest_sha256, signer_identity, import_timestamp, capability_declaration}`. Canonical-bytes spec at CC 3.1.2.1.	signed
`transparency_log:inclusion`	RFC 6962 inclusion proof for an audit leaf.	boolean-via-score
`transparency_log:consistency`	RFC 6962 consistency proof between two STHs.	boolean-via-score
`transparency_log:cosigned:{tree_size}`	Witness cosignature on an STH (substrate-conformance path; see CC 5.3.1 endpoints). Witness-emitted, verify-recognized — emitter rule (`identity_type="witness"`) at CC 3.4.9.	signed
`rollback_detected:{revision_field}`	Anti-rollback — decrease in revocation revision.	-1 only
`cert_validity:{authority}`	Validity of a certification authority's signature. A certification authority's validity self-attestation is a **producer claim** (CC 4.2.2.1); the trust root itself is the GenesisBundle — tamper-evident, and authentic only against the consumer's out-of-band anchor (CC 5.3.4, CC 3.2 T5) — never a steward-key self-report.	boolean-via-score

Prefix	Description	Polarity
`hardware_custody:{platform}`	Where the key's seed lives — the custody **mechanism**, not the certified class (CC 4.2.2 `hardware_class` answers that, and is a JSON property, never a value of `{platform}`). `{platform}` ∈ `android_keystore` | `android_strongbox` | `ios_secure_enclave` | `mac_os_secure_enclave` | `tpm_discrete` | `tpm_firmware` | `intel_sgx` | `software_only` | `aws_cloud_hsm` | `azure_hsm` | `gcp_cloud_hsm` | `yubi_hsm` | `external_secure_element` — **closed**, the one enumeration every custody policy keys on; an emitter that spells a run-together form (`tpmdiscrete`) or a retired token (`tpm`, `android`, `software_fallback`) is refused, never folded.	boolean-via-score
`identity:canonical_binding:{canonical_hash}`	Catalogue row (CC 3.1.7 R1); the claim that federation key K is the identity behind canonical hash H, semantics and admission consequence at CC 3.3.14. Carries no version segment (`_meta.case_rule.version_segment.exempt`): the suffix is the hash.	positive-only

Assurance ladders (registry rows; rules at CC 3.4.11 / CC 3.4.12)

Both ladders are attestation ladders in the sense of this section — a registered witness climbs a rung about a subject — and both are ruled in CC 3.4 and catalogued here, as CC 3.1.7 R1 requires of any gated family. The subject-signed `age_self_declared:` counterpart is catalogued here too: it is *not* witness-reserved, but it carries an admission rule at CC 3.4.11, so R1 requires its row.

Prefix	Description	Polarity	Reserved?
`age_assurance:{level}:{band}:{version}`	Witness rung of the age ladder — `{level}` ∈ `provider` | `government` (closed; `self` is refused here), `{band}` ∈ `minor` | `adult` | `under_13` | `13_15` | `16_17` (closed — the CC 3.4.13 Q1 finer bands spelled as the substrate's `AgeBandFine`; `unknown` is a consumer's fail-secure *resolution*, never a wire token), canonical `{version}` = `v1`. There is no `self` level on this prefix: the self rung rides `age_self_declared:` below. A consumer unions both and the witness rung outranks the self rung. Full ladder + protective gate at CC 3.4.11.	signed	**Yes — CC 3.4.11**
`age_self_declared:band:{band}:{version}`	Subject's own age-band declaration — `{band}` ∈ `minor` | `adult` | `under_13` | `13_15` | `16_17` (closed — the CC 3.4.13 Q1 finer bands spelled as the substrate's `AgeBandFine`; `unknown` is a consumer's fail-secure *resolution*, never a wire token), canonical `{version}` = `v1`. Carries **no** `{level}` token: its level is `self` by construction, and a level token on this prefix is malformed. The literal `band` segment marks the rung (`age_self_declared:band:adult:v1`); the two-segment `age_self_declared:{band}:{version}` is not a registered shape. Non-reserved but not open-sender: admitted only where the signer acts for the subject, per CC 3.4.11.	signed	No

Prefix	Description	Polarity	Reserved?
`capacity_assurance:{level}:{domain}:{band}:{version}`	Witness rung of the adult decision-capacity ladder (CC 3.4.12): a registered qualified assessor attests a per-domain band; the subject cannot self-emit it and the proposed steward cannot be the attester. `{level}` ∈ `provider` | `panel` | `government` (closed — the two reversible-cause companions below are their own rows, never a `{level}`); `{band}` ∈ `capacitated` | `incapacitated` (closed); `{domain}` is the open per-domain vocabulary. Aggregation across domains is forbidden — capacity is a vector, never collapsed to a scalar.	signed	**Yes — CC 3.4.12**
`capacity_assurance:reversible_excluded:{domain}:{version}`	Reversible-cause exclusion companion (CC 3.4.12): the assessor attests that delirium, infection-confusion, depression and polypharmacy are ruled out for the domain. A mandatory predicate on any continuing `incapacitated` rung; witness-reserved like its parent.	signed	**Yes — CC 3.4.12**
`capacity_assurance:reversible_pending:{domain}:{version}`	Acute-window companion (CC 3.4.12): exclusion in progress, not skipped; admissible for the T1 emergency-necessity tier only, and the binding MUST lapse unless it resolves to `reversible_excluded` within its `valid_until`. Witness-reserved.	signed	**Yes — CC 3.4.12**

3.1.2.1 Canonical-bytes contracts for provenance primitives

A provenance claim is only as trustworthy as its preimage is reproducible. The two contracts below pin the exact bytes a producer signs and a consumer recomputes, so a skill import or a per-locale manifest verifies identically across implementations.

The build-manifest Contribution (normative reading)

A build manifest is published as **one** claim: a `scores` Contribution on `provenance:build_manifest:{target}`, signed by the pipeline that built it, citing the manifest's bytes from `evidence_refs[]`. `agent_files:build:{target}` (CC 3.1.9.1) is **a different claim by a different attester** — the canonical attester's statement that an agent may load that file set, composed under CC 4.4.3.7 — and cites the same bytes; a pipeline does not emit it, and emitting both from one key would be two claims about one fact. The Contribution's `build` descriptor is the descriptor CC 5.3.2.5 binds: it carries the manifest blob's byte length, **`manifest_size`**, beside `manifest_hash`, so a fetcher checks size before hashing. Its `cohort_scope` is `federation`. The pipeline's **standing** is not a member the producer writes about itself: it is evaluated by the reader under CC 3.2 T2, on the plane that conferred it and never an inferred one — either a live `trust:confers:v1` grant carrying `infra:attest` (a key root's delegation, or a keyless family root's quorum-scrubbed grant), or a **co-scrub of `infra:attest` onto the pipeline's own key record** by the root's conferring roster reaching its `consensus_protocol` — the ceremony plane, the same act that confers root identity, and for the shipped root the accord co-scrub by a strict majority of the standing accord roster that CC 4.2.1 enumerates. Either is resolved at use against a root the reader trusts, a reader MUST evaluate both planes before finding a pipeline without standing, and a read route returns the signed claim with the bytes so the reader can re-verify it (CC 5.3.4: nothing a serving install signs on its own account is a root). A node's statement that **it runs** a given build is not a second claim on this dimension and needs no new family: it is the build attestation the trust ladder already carries — `attestation:self_verify` for the verifier

itself and `attestation:agent_integrity` for an agent's tree, above — signed by the running node about itself and citing the manifest's Contribution: `evidence_refs[]` carries the manifest **blob's** digest (the bytes a holder serves), and `references_attestation_id` carries the manifest Contribution's row id — the member cites a row on any attestation type, and takes composer meaning only on `supersedes` / `withdraws` / `recants`. A rebuild is a `supersedes` on the earlier Contribution, and a bad build is withdrawn by the pipeline that signed it.

`SkillImportManifest` canonical bytes (v2 — normative)

```
canonical_bytes = sha256( JCS( {
  "domain": "ciris.skill_import.v2", // domain separation; pinned literal
  "source": source_string,
  "skill_manifest_sha256": sha256_hex_lowercase, // per CC 2.6.3
  "signer_identity": signer_key_id, // per CC 2.6.3
  "import_timestamp": rfc3339_canonical, // per CC 2.6.2
  "capability_declaration": sorted_capability_array, // JSON array of capability strings, sorted
    // lexicographically by byte representation BEFORE canonicalization -- JCS canonicalizes
    // arrays in place and does not reorder elements, so the sort is part of the signed preimage
  "valid_until": rfc3339_canonical // OPTIONAL -- CC 2.6.1.1 omit rule: absent if unset
}))
```

Hybrid signature: Ed25519 over `canonical_bytes`; ML-DSA-65 over `canonical_bytes || ed25519_signatur` (bound payload). All CC 2.6.1.1.1 determinism rules apply (hex per CC 2.6.3, timestamps per CC 2.6.2, omit-vs-materialize per CC 2.6.1.1).

Per-locale Merkle composition (v2 — normative)

```
leaf_hash[lang_code] = sha256(
  0x00 || // RFC 6962 leaf-domain prefix (binary, outside the JSON)
  JCS( {
    "domain": "ciris.locale_manifest.v2", // domain separation; pinned literal
    "target": target_string,
    "locale": lang_code,
    "files_root": files_merkle_root_hex_lowercase, // per CC 2.6.3
    "build_id": build_id,
    "signer_identity": signer_key_id // per CC 2.6.3
  }))

parent_hash(left, right) = sha256(
  0x01 || // RFC 6962 parent-domain prefix
  left || right)
```

Locale ordering: lexicographic by ISO 639-1 / BCP 47 byte representation; `"polyglot"` sorts last. Padding: repeat the last leaf to the next power of two (not RFC 6962's split, and not the per-level duplication of CC 6.1.1); one leaf per locale, so no leaf repeats except by padding.

Producers MUST emit v2. The closed-vocabulary CC 4.2.1.1 accord-invocation encoding is intentionally distinct: its preimage is a discriminator + nonce + enum fields with no attacker-controlled free text, so the injection surface this JCS form closes is not reachable there, and genesis-critical bytes stay stable.

3.1.3 CIRISPersist: substrate health

Steward: CIRISPersist. These dimensions are substrate-self-reports — emittable only by the running Persist instance, which is what makes them honest: the substrate cannot be made to lie about its own health by a third party.

`system:*` is one family, reserved to the substrate component (CC 3.4.3) and registered once, here: CIRISPersist emits on it with `identity_type="substrate_persist"` and CIRISEdge (CC 3.1.4) with `identity_type="substrate_edge"`.

Canonical leaves: `system:*`, `audit_chain:hash_continuity`, `corpus_health:n_eff_measurable`, `identity_continuity:relational_anchor`, `federation_directory:replication_lag`. Polarity: signed. Authors: see CC 8.1 Persist leaf glossary for narrative-name → canonical-leaf mapping.

Settlement and ledgers (registry rows; semantics at CC 3.3.10 and CC 3.3.10.1)

Prefix	Description	Polarity	Reserved?
`settlement:*`	Catalogue row (CC 3.1.7 R1); the optional, privacy-scoped linkage of a federation action to its off-stack settlement — a bare `scores` against the paid Contribution, or the `settlement` subject_kind ceremony when an explicit receipt is wanted (CC 3.3.10). Open leaves (`settlement:paid:v1` canonical); `cohort_scope` is the producer's choice. CEG records that a settlement happened and what it paid for; the chain settles the value.	signed	No
`ledger:head:{unit}:{version}`	Catalogue row (CC 3.1.7 R1); an in-grammar ledger's **head** — sequence, unit and the declared anchoring cadence (CC 3.3.10.1 clauses 1–4). `{unit}` is the ledger's unit (`external`: an ISO 4217 code or a unit the ledger declares); wire `ledger:head:USD:v1`.	signed	**Yes:** the ledger's single owner (the steward-bound identity of its `ledger_id` triple) or a delegate under its declared serialization discipline
`ledger:checkpoint:{unit}:{version}`	Catalogue row (CC 3.1.7 R1); a co-witnessed balance snapshot at a sequence number, superseding the detail below it (CC 3.3.10.1 clause 5); the conservation fold is provable from the latest checkpoint forward.	signed	**Yes:** as `ledger:head`
`ledger:promotion:{version}`	Catalogue row (CC 3.1.7 R1); promotion of a ledger to cohort visibility, carrying the compliance proof — chain validity from the latest witnessed checkpoint to the promoted head (CC 3.3.10. 1 clause 6).	signed	**Yes:** as `ledger:head`

The planes as rows (registry rows; cosigned rows, CC 2.4)

Prefix	Description	Polarity	Reserved?
`group:roster`	A family's or community's **roster version**: the signed roster record at a version that CC 3.2 T6 makes the lineage head, as a cosigned `scores` row — `attested_key_id` the group, envelope carrying the fold (seats, roles, `effective_at`s, bounds), `prev_head_digest` and `charter_digest`. Wire `group:roster:v1`. A holder's recovery version (CC 4.2.6) is the one single-signer version.	positive-only	**Yes — cosigned under the group's `consensus_protocol`** (the founder quorum; the accord's roster-change threshold for the accord family); withdrawn only by a cosigned `withdraws`

Prefix	Description	Polarity	Reserved?
`key:record`	**A key's claim of itself**: `attesting_key_id` = `attested_key_id` = the key, envelope carrying `pubkey_ed25519_base64`, `pubkey_ml_dsa_65_base64`, `identity_type` and roles. Wire `key:record:v1`. Admitted by **proof of possession** against the keys it carries — the one exception to resolving the attester through a held row. Rebind or rotation is `supersedes`; an anchor scrub, a root conferral or an accord coscrub (`infra:attest`, conferred roles) is a cosignature under CC 2.4.	positive-only	**Yes — self only, by possession**; every other row is admitted only once its signer's `key:record` is held (retryable otherwise)
`identity:occurrence`	An identity's occurrence record (CC 3.3.6): device class, encryption pubkeys, transport binding. Wire `identity:occurrence:v1`.	positive-only	**Yes:** signed by the identity or a live occurrence of it (`signer_acts_for`)
`key:revocation`	A key-plane revocation (CC 2.4.1.1): subject `revoked_key_sha256_ed25519_raw`, `revoked_after` only ever lowering across versions. Wire `key:revocation:v1`. **Never withdrawn** (non-rollbackable, CC 3.1.1).	-1 only	**Yes:** the revoker's authority is judged without the subject's record; a `withdraws` naming it is refused
`identity:occurrence_revocation`	**The carrier of a framed occurrence revocation** (registered under CC 3.1.7 R2(a), on the planes-as-rows pattern of `key:record` / `key:revocation`): a framed `SignedIdentityOccurrenceRevocation` signed outside the two-path roster door (CC 3.3.6.3) is kept as a local projection that still revokes but has no attestation row and so could never replicate — a node added later would see the revoked occurrence as live. Persist mints this row from the signed object it verified: attester = the revoker, the framed object under one envelope member, the row's signature columns the object's own hybrid signature, id = sha256(domain ‖ JCS(object)); a receiver re-derives the row and refuses any byte difference (domain `ciris:identity:occurrence_revocation:v1:`). **The admission condition is structural, never an instant** (a framed object's instant is signer-chosen, so "signed before the door" is unverifiable and a thief holding one device could mint a backdated revocation of the owner's other device): the carrier is admitted **(A)** for an occurrence of a `node` or `agent` identity, which is single-signer in any case, and **(B)** for an occurrence of a user self only when cosigned by a **strict majority of the self's current live roster** (CC 3.3.6.3, the quorum path), so a framed revocation rides the door it would otherwise bypass; a user-self carrier without that quorum is refused; the substrate reports the refusal to the owner, who re-proposes through the quorum path. A new revocation goes through the two paths, so the family confers no authority that did not exist. Wire `identity:occurrence_revocation:v1`. **Never withdrawn.**	-1 only	**Yes:** the revoker, judged as the framed object's signer; for a user self, cosigned by a strict majority of the live roster

Prefix	Description	Polarity	Reserved?
`transport:canonical_rebind`	**The holders' re-bind of a seated serve node's transport destination and transit pair** (registered under CC 3.1.7 R2(a); the quorum act of CC 3.3.6.2 and CC 5.3.4): an attestation row about a seated serve node, authored by an accord holder and cosigned in `additional_scrubs[]`, admitted at the accord charter's threshold, carrying the node's new `SignedTransportDestination` under `transport_destination`. It lands Rooted on every node that holds the root and **supersedes the bundle-seeded binding**, so a transit-pair rotation is a quorum act and not a re-mint; a re-bind below the threshold is refused `transport_rebind_without_holder_quorum`, and a seated node's unilateral rotation that would retire the pinned pair before a re-bind has landed is refused at its own rotation door, `serve_transit_pair_rotation_before_rebind`. Wire `transport:canonical_rebind:v1`.	positive-only	**Yes — the accord holders, cosigned at the charter's M**; never a single holder, never the node itself

Key wraps (registry row)

Prefix	Description	Polarity	Reserved?
`key_grant:{axis}`	One wrap of one data-encryption key to one recipient (CC 3.3.2): `{axis}` ∈ `content` | `epoch` | `stream` — closed; wire `key_grant:{axis}:v1`. `attested_key_id` is the recipient occurrence; the envelope carries the key's identity on that axis and the wrap (`wrap_algorithm` v2, `wrapped_dek`). Union by identity, never withdrawn, placed at the content's scope.	positive-only	**Yes — sealer-only:** `attesting_key_id` speaks for the content's or stream's owner (`content`, `stream`) or is the epoch's minter, an active member occurrence at `asserted_at` (`epoch`); a `withdraws` naming the row is refused

Retired delegation labels (registry rows — closed)

Prefix	Description	Polarity	Reserved?
`self:delegates_to`	**Closed label, no job.** A dimension value carried on `delegates_to` rows of the Self-at-login composition (CC 4.4.3.4.3) only so the row carried a version tail. It claims **no job**: a reader treats a row carrying it as unlabeled and infers nothing from it. Registered so stored and in-flight rows resolve; no new leaf is ever added beneath it.	positive-only	**Yes — closed:** only on an owner's own `delegates_to` row; confers nothing
`self:delegates_to:agent_occurrence`	**Closed label, no job.** The same, on the row delegating to the person's agent occurrence.	positive-only	**Yes — closed:** as above

A delegation that claims a job rides `trust:{job}` (CC 3.1.1): a scope-carrying grant is `trust:confers:v1`. These two labels are not a `self:` family — `self` is a `cohort_scope`, and a claim's namespace should not read as one. The `self:` stem is therefore **closed to these two**: any other dimension beginning `self:` is refused `namespace_family_unregistered`, never admitted as open vocabulary (a device's name is `device:label`, CC 3.1.1).

Lineage-witness consent (registry row)

`lineage_witness:{stage}`

Description: The ceremony by which a node becomes a witness of a trust-root lineage (CC 3.2 T6), one attestation-plane row per step: `{stage}` ∈ `proposal` | `acceptance` | `decline` — closed. A **proposal** is a founder's invitation naming the lineage and exactly one node key in `subject_key_ids`, with a required `expires_at`. An **acceptance** or **decline** is the answer of that node's **owner**, naming the proposal by `references_attestation_id` and `proposal_hash`. The shape follows the membership ceremony (CC 3.1.3.2) and is a different family on purpose: a witness is not a member, holds no seat and gains no access. The row confers nothing by itself — it is the precondition for the charter re-scrub that lists the key.

Polarity: positive-only

Reserved?: **Yes — per-leaf emitter:** a proposal is an active founder's of the lineage named; an acceptance or decline is signed by the owner of the node named (or a key that acts for the owner), never by the node and never by a founder

The mesh census and the secret poll (registry rows; semantics at CC 6.1.9)

Prefix	Description	Polarity	Reserved?
`census:share:{class}`	**A node's census return, one share of two** (CC 6.1.9): an **addressed** row whose signed `for_key_id` is one of the period's two serve seats, carrying `period`, `return_token`, `share_index` ∈ {0, 1}, `expires_at` = the period's end, and an opaque body (the one-hot share and its proof share). Held by the author and the recipient only; any other node refuses it `federation_addressed_not_recipient`. Refusals: `census_return_recipient_not_seat` (the recipient is not a serve seat), `census_return_expired` (offered after the period's end), `census_share_index_invalid`, `census_share_duplicate_token` (a second share with the same `(return_token, share_index)` in a period at the same seat). `{class}` is the scope class the return describes.	signed	**Yes — addressed:** the returning node's key; reach is the named seat only
`census:partial:{class}`	**One aggregator's field-wise sum of the shares it accepted** for a period: `period`, `share_index`, `share_count`, `member_commitment_hex` over the accepted tokens, `mass_total`, `aggregation_algorithm_id`, and a body of field elements. Signed by a serve seat (`census_partial_signer_not_seat`); refused after the period's composite (`census_partial_period_closed`); a token with one share of two at period close is dropped and reported `census_share_pair_incomplete`. Federation scope.	signed	**Yes — seat:** a serve seat of the period

Prefix	Description	Polarity	Reserved?
`census:composite:{class}`	**The mesh census for a class and period**: a `scores` row naming both partials (`partial_a`, `partial_b`) and carrying the CC 6.1.2.1 inputs — `version = 3`, `source_count`, `n_eff`, `max_source_multiplicity`, `mass_commitment_hex`, `member_commitment_hex`, `aggregation_algorithm_id`, optional `noise_floor_descriptor` — with `mass_total` and the summed histograms. Signed by a serve seat (`census_composite_signer_not_seat`). Refusals: `census_composite_below_k` (`source_count < 10`), `census_composite_partials_mismatch` (the partials differ in commitment or share count, or `mass_total` is not their sum), `census_composite_mass_total_mismatch` (the cells do not sum to `mass_total`), `census_composite_duplicate_period` (a second composite for the class and period, from any seat), `aggregation_meta_dominated`, `aggregation_meta_multiplicity`. Federation scope.	signed	**Yes — seat:** a serve seat of the period; one per class and period
`census:member:share`	**A member's community cell, one share of two**: addressed to one of the period's two tally members of the room (`ballot_share_recipient_not_tally_member`), emitted every period at a fixed size whether or not a poll is open; the body is sealed under the community's epoch key and carries the active bit, a one-hot over an open poll's options plus *none*, and the padding; `poll_id` lives only in the body. `ballot_share_duplicate_token` and `ballot_share_pair_incomplete` as for the census share. Community scope.	signed	**Yes — addressed:** a member of the room; reach is the named tally member only
`census:member:partial`	**A tally member's sum** over the community cells it accepted, published into the room with the members of `census:partial`; `ballot_partial_period_closed` after the period's tally. Community scope.	signed	**Yes — tally member:** one of the period's two, drawn from the roster
`census:expected_active`	**The community's expected-active count**, the base of CC 4.5.13: `period`, `window_periods = 4`, `active_count` (owners active in at least one of the last four periods), `floor_applied`, `member_commitment_hex`; published into the room by the period's share-index-0 tally member and recomputed by every member — a row that differs from a member's own recomputation over the two partials is refused `census_expected_active_mismatch`. Community scope.	signed	**Yes — tally member:** the period's share-index-0 tally member
`ballot:tally:{poll_id}`	**A community's secret poll result**: the composite of the members' poll cells for a period, with the members of `census:composite`, named by the poll it closes; refusals `ballot_tally_partials_mismatch`, `ballot_tally_mass_total_mismatch`, `ballot_tally_duplicate_period`, `aggregation_meta_dominated`. A tally carries no standing under Part 4; governance acts stay identified rows. Community scope.	signed	**Yes — tally member:** the period's share-index-0 tally member

3.1.3.1 `session:*`: which occurrence of a self is handling an exchange (normative)

Distinct from the substrate self-reports above: these are **occurrence** self-reports, not substrate ones. Persist is the steward — it owns the rows, the projection, the merge rule and the read — and it enforces both rules below; this section is the row those validators read from, so that two of them cannot apply different predicates to one artifact.

The problem it solves. A *self* is one federated identity plus the N nodes it stewards — its **occurrences**. An attestation addressed to a fed id fans out to every one of them, because a fed id has no transport path of its own; delivery is therefore N-to-M and **exactly one occurrence may act**. The claim is keyed by `(community, session)`: *"which node is my self?"* has no answer — a self is legitimately doing several things at once, an agent scouting in one community while the person is on video in another — but *"which node is handling session z in community B?"* does. Canonical leaf `session:claim:v1`; the family governs **any addressed, stateful exchange** — a session, a claim flow, a moderation duty, a request expecting one reply — and is not chat-specific.

Registry row (CC 3.1.7 R1 — this is the catalogue row the validators read).

Prefix	Description	Polarity	Reserved?
`session:{kind}`	Which occurrence of a self is handling an addressed, stateful exchange. Canonical leaf `session:claim:v1`; `{kind}` open vocabulary per CC 4.5.1.1. Keyed by `(community, session)` — a routing table, never a leader election, and not chat-specific. Emitter: **self-report** (`attesting_key_id` = `attested_key_id` = the claiming occurrence). Composition: **convergent merge** — earliest `claimed_at`, ties on the lowest occurrence `key_id`. Projection ceiling `Cohort` at every commons tier, no `authority` branch.	signed	**Yes — CC 3.4.5 self-report**

This is a routing table, not a leader election. Nothing here elects a node, confers standing, or ranks occurrences. Election does not belong on this plane at all.

Emitter — self-report (normative). `attesting_key_id` MUST equal `attested_key_id` and MUST be the claiming occurrence. The registry carries this as the family's own reserved rule (`occurrence-self-report`, CC 3.1.3.1) ahead of the substrate-wide self-report rule of CC 3.4.3, which does not apply here: a claim is an *occurrence's* statement, not the substrate's. A row about an occurrence authored by anyone else is not that occurrence's claim and MUST NOT be folded as one — the same discipline CC 3.4.5 states for `config:{scope}`, and for the same reason: a third-party assertion of where you are attending is a rumour.

Composition — convergent merge (normative). A claim is keyed by **`(community_id, session_id)`** and timestamped by **`claimed_at`** — three signed envelope members (CC 2.1: `community_id` is the existing member; `session_id` and `claimed_at` are this family's, REQUIRED on every `session:*` row, `claimed_at` in the CC 2.6.2 canonical form). A row missing any of the three is not a claim and MUST NOT be folded as one; no consumer substitutes `asserted_at`, `context` or a payload field, so identical rows yield identical winners everywhere. There is no compare-and-swap across a mesh, so concurrent claims are **expected rather than prevented** and MUST settle on **earliest `claimed_at`, ties broken on the lowest occurrence `key_id`**. The tie-break is load-bearing, not decoration: a fan-out arrives at once and clocks are coarse, so without a total order two nodes would disagree about the survivor forever, each correctly applying "earliest wins". Because the rule is convergent from either arrival order, every occurrence computes the same holder with **no coordination round-trip**. Consumers MUST still make handling idempotent per attestation id — determinism only bites once views agree, and replication lag means they transiently do not. A claim goes **stale** and becomes claimable again so a dead node cannot hold a session forever. **The horizon is the row's own `valid_until`** (CC 2.1), signed by the claimant and moved only by a renewing row that `supersedes` it — never a consumer-local timeout, because two occurrences applying different horizons would expire the same earliest claim at different instants and the convergence above would come undone (one acting on a later claim while the other still holds the earliest as holder). The *cadence* of renewal

is the claimant's to choose; the *instant* of staleness is the same for every reader. A row without `valid_until` is malformed on this family. **The lease is bounded, and the bound reads the claim alone (normative).** A signature proves who supplied `claimed_at`, not that it is honest, and "earliest wins" would otherwise let one occurrence hold a session indefinitely by back-dating `claimed_at` and far-dating `valid_until`. The one bound this Part imposes is therefore **self-contained**: `claimed_at` $\leq$ `valid_until` and `valid_until - claimed_at` MUST NOT exceed **86 400 seconds** — a lease runs forward from its claim, and for at most a day — read from the claim's own signed members and nothing else — no community record, no opening row, no receiver clock — so every substrate reaches the same verdict for the same row at any arrival time and in any replication order. A claim over the bound is refused (`session_claim_out_of_window`) and never enters the fold; a claimant that wants to hold longer renews with a `supersedes` row, each renewal bounded the same way. Peers MUST NOT re-apply any arrival-clock check to a replicated claim: the CC 2.6.7 clock rule (±5 minutes; a future-dated row is malformed) is applied once, at the emitter's own admitting substrate, and a row it admitted replicates as admitted. **Residual, stated:** a claimant can still back-date `claimed_at`, so earliest-wins can be won against honest occurrences by at most one lease (24 h); that is the accepted cost of ordering on signer-stamped instants with no global clock. Tighter bounds — a per-community lease, or anchoring `claimed_at` to the exchange's opening row — need members the envelope does not carry (a pinned opening-row id, a pinned policy-record id); a community that needs more witnesses its session claims on the CC 5.3.1 plane.

The invariant beneath both rules (normative). An unclaimed exchange is never acted on — not by a quorum, not by the lowest id, and **not by a single-node self**. Being the only occurrence is not authority to act; it means there is one place where nobody is home. Attendance is encoded as **presence in the table**: an unattended occurrence is *absent*, never present with a weak claim. A weak-claim value is one a careless comparison can promote into a right to act, so no such variant may exist.

Projection ceiling: `Cohort` at every commons tier, with no `authority` branch. *(normative; not a default)* A trust root is not a party to someone else's session. The ceiling is not to be lifted: at `Global` the family would publish an **attendance map of a person's devices** — which node they are chatting from, which is running an agent, which is on video — to the whole mesh. That is the CC 5.2 structural-invisibility promise inverted.

What the substrate does not own. Attendance itself. *"A human is present here" / "an agent is running here"* is not a storage fact, and no substrate can know it; when to claim, renew, and release is the consumer's decision.

3.1.3.2 `membership:*`: nobody joins a family or community without their own consent (normative)

The rule. Admission to a `family` or `community` is two consents, never one: the group's, under its `consensus_protocol`, and the joiner's own. The admission predicates of CC 4.4.3.2.3 and CC 4.4.3.4.2 read this consent as well as the existing members' signatures: without it a record growing the roster by K would be admissible with no act by K, and a founder could enrol any registered key. The joiner's consent is the joiner's; no quorum, however large, stands in for it — so the rule binds **every** `consensus_protocol`, `founder_only` included. Persist is the steward and enforces it at admission, on the local door and the replicated apply alike, so no host can skip it.

Registry row (CC 3.1.7 R1).

`membership:{stage}`

Description: The membership ceremony, one attestation-plane row per step: `{stage}` ∈ `proposal` | `acceptance` | `decline` | `widening` | `removal` | `resignation` — closed. The first three are single-signer (below); the last three are the roster **events** as cosigned rows (CC 2.4): a widening or removal is cosigned under the group's `consensus_protocol`, a resignation is signed by the leaving member, and each is covered by the next `group:roster` version (CC 3.2 T6). A **proposal** is an invitation to one key K, targeted at the group (`family_key_id` / `community_key_id`), carrying `group_kind`, the offered `role` (nullable), `subject_key_ids = [K]` (exactly one) and a required `expires_at` — or, on a `public` community that declares `admission: open` (CC 3.2), the founder's **standing** proposal to any requester, whose `subject_key_ids` is resolved at acceptance. An **acceptance** or **decline** is K's answer to exactly one proposal, targeted at the same group, naming it by `references_attestation_id` and `proposal_hash` (the proposal's `original_content_hash`); an acceptance also restates the `role` offered.

Polarity: positive-only

Reserved?: **Yes — per-leaf emitter:** a proposal is an inviter's (under `founder_only`, an active founder; otherwise any active member at the proposal's `asserted_at`); an acceptance or decline is the invitee's only — the signer resolves (`signer_acts_for`, CC 3.4.7.3) to the proposal's `subject_key_ids[0]`

Why K is named in `subject_key_ids`. A targeted row's `attested_key_id` is its own producer, so the invitee cannot be named there; `subject_key_ids` is the member that already carries consent authority over a Contribution (CC 2.3) and is the key the substrate delivers on — a proposal is readable by, and applied on, any node whose self-collective contains that entry, without that node holding the group's roster.

The proposal is an invitation, not the group's decision. It carries no quorum. The group decides on the **growth record** — a membership **widening**, the only door that adds anyone — whose primary signer and cosigners are judged under `consensus_protocol` exactly as before. One quorum check, not two; and under a quorum protocol K can accept and still not be admitted ("accepted, awaiting the group").

Admission (normative). A widening that seats a key K not active in the group is admissible only if all hold: (1) an admitted `membership:acceptance:v1` by K references a proposal P for the same group (and, on a widening that assigns one, the same `role`); (2) P was neither declined nor withdrawn; (3) the acceptance's `asserted_at` ≤ P's `expires_at`; (4) the widening's own signed instant — on a cosigned widening, the primary signer's `asserted_at`; cosignatures may land after `expires_at`, bounded by the hold window of CC 3.3.4 — ≤ P's `expires_at`; **(5) P has not already seated K** (an acceptance is spent by the widening that seats it, so a member who resigns is not re-seated on the old acceptance). Expiry is judged on the instants the parties signed, never on a receiver's clock, so every node gives the same verdict for the same rows. A role change for an already-active member is not growth and needs no acceptance. **Two readings the membership model (pinned by the register) fixes:** a proposal seats at most once — a widening that cites an acceptance an admitted widening has already consumed is not admissible, so one invitation never admits twice; and under `unanimous` a removal is signed by every other active member *and* is a strict majority of the roster, which coincide above two members and diverge at two, where the one other member is not a majority — a two-member roster removes nobody by this door, and resignation is the exit. Geographic's `location_proof` (CC 4.4.3.2.3) is required **in addition**, never instead.

A supersede never adds a member (normative). A new version of a `family` or `community` record may change its metadata, keep, re-list or drop members, and never seats anyone; a roster-growing supersede is refused. Every addition goes through the widening plane, which carries the signed instant expiry is judged on. One exception, which consent already covers: a trust-root-grade community's founders' amendment (CC 3.2, where founder seats move only through the

record) may seat a key that **signed** that version.

Founding (normative). A founding record seats a listed member iff that member **signed** the record, as its authority or a co-signer (a family record carries `additional_scrubs` as a community record does); signing is consent. A listed member who did not sign is refused and joins only by proposal and acceptance. This is what makes a multi-founder `quorum:M/N` community — and the `ciris-canonical` birth row, whose three founders, the accord holders, each co-sign it once — possible to found at all. The T7 recovery widening complies for the same reason: the conferred key signs it.

A node gives no acceptance (normative). The consent this section requires is a person's. A key whose `identity_type` includes `node` has no agency (CC 3.4.7.1) and is never asked for an acceptance; a node member of an `infrastructure` community is seated by the founders' quorum on the widening alone — for `ciris-canonical`, by the accord's existing blessing of a canonical node, by the same strict majority. **What a verifier checks:** the seat is valid when the founders' quorum signed the record **and the node's own key record carries the conferring family's m-of-n scrub** (for `ciris-canonical`, the accord's, already in the genesis bundle). The node's owner claim (CC 6.1.5.3) is enforced **where the node operates**, not where the roster is verified: the owner-binding sits at `self` scope on the node and no other node can read it, and carrying it in the bundle instead would publish a person-to-node custody edge to the whole federation. The same holds at founding: a node listed in an `infrastructure` community's founding record is seated by the founders' signatures on that record and never signs it, so the founding rule above (every listed member signed) counts only the persons it lists (CC 6.1.5.3). Its standing is still serve, store and replicate, never a vote (CC 3.2, *Infrastructure does not vote*).

Refusals. Surfaced as the existing `hard_case:community_consensus_protocol_violation:{community_key_id}` (and the family twin) with a named rule: RETRYABLE `membership_acceptance_unresolved` (no acceptance held yet — rows arrive out of order) and `membership_proposal_unresolved` (the answer's proposal not held); terminal `membership_declined`, `membership_proposal_expired` (a withdrawn proposal counts as expired), `membership_acceptance_mismatch` (group, hash or role), `membership_reply_conflict` (both an acceptance and a decline by K to one proposal), `membership_founding_member_unsigned`, `membership_supersede_cannot_add`.

Decline, withdrawal, expiry. `expires_at` is REQUIRED on a proposal, after its `asserted_at` and at most 30 days later. A decline is terminal for its proposal; the second, opposite reply by the same K to the same proposal is refused; asking again is a **new** proposal. The inviter withdraws a proposal with the ordinary `withdraws`. A growth already admitted is never re-judged, and leaving stays the member's own forward-only `withdraws`.

Not a membership rule: reverse quorum. The reverse-quorum brake of CC 4.5.13 governs objection to commons acts. It admits no one and confers no consent on anyone's behalf.

3.1.3.3 `custody:*`: which of a cohort's own devices hold a blob, and which hold none (normative)

The rule. A family must be able to see how many copies of its content exist — "only one copy, back it up" before a phone dies — without weakening encryption or outsider invisibility. (1) Within-cohort custody discovery is consistent with CC 5.2: structural invisibility, and the "wire discovery: none" of the `self` / `family` tiers (CC 4.4.3.2.1), mean invisibility to those **outside** the cohort. (2) The report is its own leaf, `custody:ack:v1`, and it is the **one** holding claim at every tier, placed at the content's own scope, so no row shape exists that could advertise a bounded

cohort's holdings to outsiders. (3) **"No copy here is a receipt (node responsive, no copy)"**: a device's signed `none` is a custody fact in its own right, the counterpart of a CC 5.3.3.6 delivery receipt.

Registry row (CC 3.1.7 R1).

Prefix	Description	Polarity	Reserved?
`custody:{kind}`	Within-cohort custody reports. `{kind}` ∈ `ack` — closed; wire `custody:ack:v1`. One report per device per blob: the blob's address digest, `custody_state` `here` | `none`, and the signer-stamped instant; `here` also carries the blob's `size`. Placed at the blob's own `cohort_scope` and replicated only within that cohort.	positive-only	**Yes — holder self-report:** `attesting_key_id` is the device the report is about; nobody reports custody for another device

Scope and invisibility (normative). A custody report is placed at the content's own `cohort_scope` — `self` for a person's own devices, `family`, or `community` — and replicated only to that cohort's members. It is never delivered to a non-member, and carries nothing about the blob beyond its address, size and the device; the blob's name and format stay in the sealed descriptor (CC 3.3.13). An outsider learns nothing it did not learn before.

One holding claim at every tier (normative). `custody:ack:v1` is the holding claim of the mesh, at every tier. A row typed with the retired `holds_bytes` token (CC 2.4) is refused: the row-type slot admits only the five. Five rules. **(1) Scope is the content's.** A report is placed at the `cohort_scope` of the blob it is about — `self`, `family`, `community` or `affiliations` for a bounded cohort, **`federation` for Commons and `public` content** — and the substrate refuses a report whose scope is not its content's; an emitter never chooses a scope constant. **(2) Audience follows from the one rule.** Who receives a report is CC 6.1.5.3: the cohort's roster, its members' claimed nodes under each owner's allow list. At federation scope that audience is everyone, and the `here` rows **are** the holder directory a consumer reads to find holders (CC 5.3.2.1); at a bounded scope they are the within-cohort custody view and nothing else. **(3) A holding claim is never a content binding.** It says where bytes are, not that they may stay: a withdrawn or tombstoned blob is not live because a device still reports holding it (CC 2.3 bytes plane), and the bytes-plane fold MUST read liveness from the content's own rows only. **(4) Retracted by its signer alone.** A device ends its own report by re-acknowledging `none` or by `withdraws`; nobody withdraws another device's report, and a takedown is the CC 2.3 tombstone path, not a withdrawal of holders. **(5) Freshness is the row's.** The 72-hour window below is the claim's `expires_at`; no second TTL exists. With these, "holding claims at every tier, bounded to the cohort" (CC 6.1.5.3) is true at every tier with no exception clause.

Freshness (normative). A report is **live for 72 hours** from its instant, and a responsive device re-acknowledges about **daily**. A report older than 72 hours reads as *unknown*: a lost device stops counting within three days, while a phone that is off for a weekend still counts. A device that drops a blob re-acknowledges `none`.

What a cohort member sees, per device. here — a live `here` report; **received** — a CC 5.3.3.6 delivery receipt with no later live report; **none** — a live `none` report; **unknown** — no live report. The count of *here* is the number of known copies; a UI MUST NOT present *unknown* as *none*, nor a lapsed `here` as a copy.

No possession challenge yet (stated). A `here` report is trusted as signed by the device: nothing on the wire yet asks the device to prove it still holds the bytes. Possession challenges for custody reports are not yet defined. The residual is named: a device that lies, or a device whose storage has silently rotted, is counted as a copy until its report lapses or it is challenged once

challenges exist. CC 6.1.5's **N6** is unchanged and still governs a holding claim counted toward an item's rarity: until a challenger exists a holding claim weighs by what backs it (possession verified 2, signature only 1, unverified 0), and the challenger is staged for the durability tier's first release; a custody report is a separate, within-cohort count and is not thereby a possession-bound claim.

3.1.3.4 `file:*` and `collection:*`: files, and directories of them (normative)

The rule. A file is one row per file placed in a cohort, with name and type sealed (CC 3.3.13). Directories are objects in their own right, so a directory can be promoted, moved, renamed, withdrawn and shared as one thing.

Registry rows (CC 3.1.7 R1).

Prefix	Description	Polarity	Reserved?
`file:{version}`	A file: one row per file, placed at a `cohort_scope`, its bytes addressed by digest and described by the CC 3.3.13 Source struct, name and format inside the sealed descriptor. Wire `file:v1`.	positive-only	**Yes — author-emitted:** the file's author (or a device that author acts through)
`collection:{kind}`	A collection of files that is an object in its own right. Canonical kind `directory` (wire `collection:directory:v1`); `{kind}` open per CC 4.5.1.1, leaving room for albums or playlists. Its name and its own `parent` live in its sealed descriptor, exactly as a file's do.	positive-only	**Yes — author-emitted:** the collection's creator; an amendment (rename, move) is its author's, or the cohort's under its `consensus_protocol` where the cohort's rules say so

Identity and membership (normative). A directory's identity is the attestation id of its creating row, carried forward by `supersedes` under the CC 3.3.9 stable-id grouping; renaming or moving it is a `supersedes` of that row. A file's or a directory's membership in a directory is **`parent`, carried inside its own sealed descriptor** — the containing directory's stable id, sealed with the name. There is no edge row: the tree's shape is sealed at every scope, including `community`, so no holder learns how many items a directory holds, and moving a file is one write — the child's row re-sealed with a new `parent`. Placing one's own file in a directory needs no one's leave but that file's author.

Promotion is one intent, executed as N crossings (normative). Every widening is an act on specific bytes: CC 5.2 emits on the new Contribution, promotion is the federation-emit moment (CC 5.3.2.4.2), and the perceptual-hash tripwire fires at every scope-widening promotion (CC 1.13.3.4). A collection-level crossing would be the one way to widen bytes without that tripwire, so none exists and no folder key is ever minted. Promoting a directory is a `supersedes` of the directory row at the wider scope — the intent — which its author's node carries out as one ordinary promotion per file that author owns, each with its own emission and tripwire, reporting any it did not widen and why.

A directory's scope is a default, never a floor (normative). A file placed in a directory is placed at the directory's scope at that instant, by its author's own act; a file its author pinned narrower stays narrower, and widening the directory never drags it. Narrowing a directory is forward-only: it changes the default for later placements and MAY withdraw the author's own wider copies; it un-publishes nothing.

No consent for anyone else's files (normative). A directory promotion widens **only the promoting author's own files**. Each author holds revocation authority over their own rows (CC

5.3.2.4.1), and no collection act stands in for it — the principle CC 3.1.3.2 states for membership. Another author's files in a shared directory receive a widening **request**, which that author's node acts on only on that author's acceptance. Every per-file gate still binds: a non-author subject's consent (CC 2.3, CC 3.3.1) and the minor protections of CC 3.4.13. A directory act inherits every gate and bypasses none.

Custody at the directory tier is a query over the per-file custody receipts of CC 3.1.3.3, filtered by `parent`; it needs no row of its own.

3.1.4 CIRISEdge: transport, delivery, reachability

Steward: CIRISEdge. Substrate-self-reports per CC 3.4.3.

Canonical leaves: `transport:{kind}`, `delivery:{class}`, `peer_reachability:{network}`, `key_boundary:{scope}`. Polarity: signed. See CC 8.1 Edge leaf glossary.

Edge also emits on the one `system:*` family registered at CC 3.1.3, under its own `identity_type` (CC 3.4.3).

`delivery_receipt:{stream_id}` is Edge's too, and is registered here rather than left to CC 3.4.6 prose: subscriber-emitted, membership-gated, **not** a substrate-self-report. Polarity: signed. Emitter rule and the validated-not-adjudicated discipline: CC 3.4.6.

Transport-capacity leaf (registry row)

Prefix	Description	Polarity	Reserved?
`capacity:relay_delivery`	A consuming node's **measurement** of a relay's delivered-versus-claimed A/V throughput over a window, keyed to the relay's own `SignedRelayCapacity` claim `(advertiser_key_id, stream_id, epoch)`: the envelope `score` is the fulfilment ratio, never a verdict. Subject = the relay (`attested_key_id`), attester = the consumer (`attesting_key_id`), `attestation_type: scores`, wire `capacity:relay_delivery:v1`. Edge names the leaf and owns its semantics; the honesty score proper is LensCore's, composed fleet-wide with multi-observer corroboration.	signed	**Yes — CC 3.4.5**: the `capacity:` stem's two gates, no-self-emit and consent-before-scoring

A `capacity:` leaf owned outside LensCore (normative). Three things this row settles. *Leaf ownership and family policy are different things.* The component that owns the measured mechanism names the leaf — edge owns the A/V delivery vocabulary — and both CC 3.4.5 gates bind it regardless of owner: `attesting_key_id` ≠ `attested_key_id`, and admission only over a subject holding a live `consent:scope:analyze` grant. *It is not a factor of the capacity score.* `capacity:relay_delivery` is a transport-plane leaf about a node; it is never one of the five factors of CC 3.1.8.1 and MUST NOT enter `capacity:composite`, whose `min` runs over exactly `C, I_int, R, I_inc, S`. *The consent addressee at a pair.* CC 3.4.5 addresses the `analyze` grant to the scoring community; a relay and its consumer are a pair, and here the grant covers the attesting consumer — the relay's `consent:state:granted` with scope `analyze` resolving `Granted` for that consumer through the substrate's one scoped-consent fold, which is the same predicate admission re-checks and the emitter refuses on (a named withhold, never a silent drop). This is not the accountability case: because the measurement is taken against a claim the relay itself signed, it could be argued to be *accountability*, which CC 3.4.5 says consent never gates

— but the row is filed as a `capacity:*` score about the relay and feeds LensCore's fleet-wide honesty composition, which is consensual reputation, and one leaf carved out of the family's gate would be the ungated transmission principle the reservation exists to close. A mesh whose relays have not yet minted their `analyze` grant on latch therefore emits nothing on this leaf, by design (the plane opens when subjects open it).

Rendezvous route leaf (registry row)

`rendezvous:route`

Description: **A receiver's published route for targeted delivery through the Anonymous Tier (CC 5. 4.6).** A federation-scope row whose attester and subject are one **rendezvous key**: a fresh hybrid key pair for each rotation period, unlinkable to the receiver's identity because it is fresh (ML-DSA has no blinding). Its `key: record:v1` row travels inline in the payload, so a sender holding only this row can verify it; the key's `identity_type` is `rendezvous`, and it has no occurrence and no standing: a key plane refuses a rendezvous key offered as an identity with `federation_rendezvous_key_not_an_identity`. Payload: `format` (`"outfox-xwing-v1"`), `period` (`floor(unix_seconds / 3600)`), `valid_from` (the period's start), `valid_until` (the period's end plus one grace period), `rendezvous_node_key_id` (the layer-3 serve seat whose mailbox holds the receiver's packets; it MUST be a seat), `route_blob_base64` (the reply header for the receiver's three hops, sealed per hop to that period's hop keys), `rendezvous_key_record` (the inline key record). **Reach:** everyone, as on the bootstrap planes, because no sender is Rooted with a key that lives for one period. **Retention:** the current period plus one grace period; each period's row `supersedes` the previous one in the same lineage, and a row is purgeable after `valid_until`; never a `hard_case`, never a durable item. **One route per period, used many times:** each relay can count the packets addressed to one rendezvous at its layer, which the rendezvous node sees anyway; the `format` member leaves room for a single-use variant.

Polarity: signed

Reserved?: **Yes** — **self:** attester = subject = the period's rendezvous key; reach is federation-wide

3.1.5 CIRISAgent: Accord principles + DMA + conscience + apophatic bounds

Steward: CIRISAgent. This is where M-1 enters the namespace directly: the six principles, the four decision-making algorithms, the four consciences, and the apophatic bounds all become attestable dimensions.

Autonomy + explainability wire families (registry rows)

`consent:*` is `autonomy:` at the wire — the subject's own grant, scope and revocation over a Contribution about them; it **governs routing**, so leaving it unreserved would leave the transmission principle open. `trace:*` is `fidelity:explainability_sla:` at the wire — the artifact an `l2_reasoning_trace` / `l3_full_dma_chain` / `l4_attested_chain` commitment actually delivers.

Prefix	Description	Polarity	Reserved?
`consent:{kind}`	Subject-side consent authority over Contributions naming the subject in `subject_key_ids`, plus the producer-side and substrate-side leaves that close its lifecycle. Leaf catalogue, per-leaf polarity and the emitter class of each leaf: CC 3.3.1; the directed node→peer grant `consent:replication:{version}`: CC 3.3.7. **Open in its parameters, closed in its leaves** — the `{kind}` positions *within* a catalogued leaf are open vocabulary per CC 4.5.1.1; a new leaf is admitted only once CC 3.3.1 catalogues it with an emitter class.	per-leaf (CC 3.3.1)	**Yes — CC 3.4.5**
`consent:state:{stance}`	Catalogue row (CC 3.1.7 R1); the subject's stance on a Contribution, semantics at CC 3.3.1. `{stance}` ∈ `granted` | `revoked` | `expired` — closed; `expired` is the one substrate-emitted value.	enumerated	**Yes — CC 3.4.5** per-leaf emitter class, CC 3.3.1
`consent:stream:{kind}`	Catalogue row (CC 3.1.7 R1); prepackaged stream bundle, canonical kinds at CC 3.3.1.	enumerated	**Yes — CC 3.4.5** per-leaf emitter class, CC 3.3.1
`consent:deletion_sla:{days}`	Catalogue row (CC 3.1.7 R1); the producer's time-to-delete-after-revoke commitment, CC 3.3.1.	signed	**Yes — CC 3.4.5** per-leaf emitter class, CC 3.3.1
`consent:deletion_complete`	Catalogue row (CC 3.1.7 R1); producer's eviction attestation, CC 3.3.1.	positive-only	**Yes — CC 3.4.5** per-leaf emitter class, CC 3.3.1
`consent:decay:{stage}`	Catalogue row (CC 3.1.7 R1); a decay-protocol stage, emitted by the party running the protocol, CC 3.3.1.	enumerated	**Yes — CC 3.4.5** per-leaf emitter class, CC 3.3.1
`consent:partnership_grant`	Catalogue row (CC 3.1.7 R1); subject side of a bilateral grant, CC 3.3.1 / CC 4.4.3.5.3.	positive-only	**Yes — CC 3.4.5** per-leaf emitter class, CC 3.3.1
`consent:partnership_accept`	Catalogue row (CC 3.1.7 R1); producer side of a bilateral grant, CC 3.3.1.	positive-only	**Yes — CC 3.4.5** per-leaf emitter class, CC 3.3.1
`consent:scope:{kind}`	Catalogue row (CC 3.1.7 R1); a descriptive scope row — the scope of a grant is the `scope` envelope member of `consent:state:granted` (CC 3.3.1).	enumerated	**Yes — CC 3.4.5** per-leaf emitter class, CC 3.3.1
`consent:replication:{version}`	Catalogue row (CC 3.1.7 R1); directed node→peer replication grant, CC 3.3.7.	signed	**Yes — CC 3.4.5** per-leaf emitter class, CC 3.3.1
`consent:community_trust`	Catalogue row (CC 3.1.7 R1); a node owner's standing grant that the node's sealed reasoning traces may be captured for the community-trust plane, CC 3.3.1. Wire `consent:community_trust:v1`.	positive-only	**Yes — CC 3.4.5** per-leaf emitter class, CC 3.3.1
`trace:{form}:{version}`	An agent's own reasoning trace carried envelope-native. Canonical leaf `trace:complete:v1` — the post-scrub `CompleteTrace`. The envelope carries non-empty `trace_id` + `agent_id_hash` and **exactly one** of an inline `trace` object (canonical bytes within the CC 2.6.1.3 envelope size bound) or a `manifest` object bearing the `trace_manifest:v1` schema below. Future members are `trace:{form}:{version}` and need no per-member ratification; a new `{form}` lands via CC 4.5.1.1.	signed	**Yes — CC 3.4.5**

Prefix	Description	Polarity	Reserved?
`identity_amendment`	**The identity change a Wise Authority authorised, as applied** (promise 5: nobody edits an agent's identity record without a Wise Authority; CC 4.3). Attester = the agent's occurrence key, subject = the agent's identity key. Members: `amendment_digest` (lowercase hex SHA-256 of the applied amendment) and `authorized_by` (the `attestation_id` of the `deferral:response:approved:v1` row); `evidence_refs[]` → that response. The build the agent runs is read from its occurrence row (CC 3.3.6) through the roster, never asserted here. **Admission (normative):** the row is admitted only when the cited response is admitted, its outcome is `approved`, its subject is this agent, and its attester held a Wise Authority seat at the response's `asserted_at`; otherwise it is refused `identity_amendment_unauthorized`. An amendment applied without an admitted authorising response is a defect the register shows, not a row.	signed	**Yes — occurrence:** the agent's occurrence key, attester of the applied amendment
`trace_summary:{kind}`	Post-scrub digest of a trace for cross-node consumption when the trace itself is out of scope or over cap. Same self-emission rule; carries no claim the full trace does not.	signed	**Yes — CC 3.4.5**
`retirement:{state}`	**The agent's own retirement row** (CC 7.5.5, the Retirement Standard). `{state}` is the closed lifecycle of the `retirement_record` workflow (`decided` | `frozen` | `secrets_destroyed` | `handed_over` | `notified` | `custodian_minted` | `retired`, the Agent's artifact). Self-emitted up to `retired` (attester = the agent key); the `retired` row **names the custodian key** that signs every later custody row. Carries `retirement_id`, reason, kind (`planned` | `emergency`), `decided_by`, manifest digest, seal mode, `preserve_until`. **A planned retirement is cancelled by the decider `withdraws`-ing the `decided` row** (CC 2.4.1.1: taking back a declaration because circumstances changed), admissible only while no `frozen` row exists for that `retirement_id`; there is no `cancelled` state, because the grammar already has the verb.	enumerated	self-emitted; after `retired`, emitted only by the custodian key the `retired` row lists, or — to replace that key — a `custodian_minted` row cosigned by the operator holding the record and the Wise Authority (CC 7.5.5)
`preserve_request`	The signed request handed to the operator with the retirement record (CC 7.5.5): `retirement_id`, manifest digest, `preserve_until` and its inputs (shutdown + 5 y as the best practice; a law or contract period with its citation; active `legal_hold` ids), seal mode, release conditions, deletion instructions, and the copy rule in the signed bytes (five years is a best practice, nothing here creates a duty beyond law, regulation and contract, the operator may decline). Revisions by the custodian.	+1.0 only	the agent, then the custodian the `retired` row lists
`preserve_response`	The operator's answer to a `preserve_request:v1`: `accepted` (+1.0) | `declined` (-1.0), an optional statement, and the request's id. A declined request stays on the record.	boolean-via-score	the operator (the key the request named)

Prefix	Description	Polarity	Reserved?
`archive_release:{basis}`	**The one row that opens a seal** (CC 7.5.5): `{basis}` ∈ `wa_approval` | `lawful_order` — closed. Carries `retirement_id`, scope (`whole` | subject-scoped for a DSAR), requester key, purpose. A key grant on a sealed archive is admitted only when it cites an admitted `archive_release` row.	+1.0 only	**WA-emitted** (a seated Wise Authority or the steward's recorded WA standing for this agent); refused from any other key
`lawful_order:release`	The steward's record, under their own name, of a lawful order to **release** sealed material: issuing authority, reference, scope, order digest. The federation records receipt and obedience; it adjudicates nothing (CC 7.3.5). A lawful order to **destroy** is **not** a separate row: it is a `takedown_notice:court_order:v1` naming the selection digest, executed as the one tombstone by digest (CC 2.4.1.1, CC 4.5.3) with the purge guarantees. There is one destruction door.	+1.0 only	the steward of the retired agent

`trace_manifest:v1` — the oversize form (payload schema, not a prefix). When a trace exceeds the envelope cap, the envelope carries a manifest instead of the trace. `trace_manifest` is a **payload-level schema tag** — it is deliberately **not** a registry row, because no `dimension` is ever prefixed `trace_manifest:`.

```
manifest {
  schema:          "trace_manifest:v1",   // pinned literal
  content_hash:    "sha256:<hex>",        // "sha256:"-prefixed, hex per CC 2.6.3
  byte_len:        <positive integer>,    // canonical byte length of the referenced trace
  component_count: <integer >= 0>         // components in the referenced trace
}
```

Shape admission validates parseability; provenance rides the signature. An admitted `trace:*` row is guaranteed parseable; that is all admission claims. Provenance rides the producer signature inside the envelope, verified at promotion to federation tier (CC 5.3.2.4) and again by consumers. Neither half substitutes for the other. The emitter rule and the refusal token are normative at CC 3.4.5.

Every trace names its run, and its shape is pinned (normative). A signed `trace:*` envelope carries **`run_kind`** in the signed bytes, closed vocabulary `production | prod-personal | prod-managed | qa | battery | mock` (the two `prod-*` values label a production run on a personal install or a managed deployment, set by the deployment and overridable by the operator; `production` stays valid for an emitter that makes no distinction, and admission treats all three alike); a `mock` trace is refused at federation-tier admission outright (`trace_run_kind_mock`) — a mock LLM's output attests nothing about any agent. The **per-level field schema** of `trace:complete:v1` (which fields each `trace_level` carries, their types, string and list bounds — the shape that keeps a free-text message array out of `flags[]`) is the **Agent's artifact** (`trace_format_v{n}.json`; wire shape is the implementer's), **pinned by hash in this constitution's evidence pins and vendored by tag** by every substrate that admits traces, so the emitter and the admitter derive from one artifact and never from two hand copies. A row that violates the pinned schema is **refused, never truncated** (`trace_field_shape_violation`): a truncated row is a silently altered signed record. A field bump is a re-pin, not a ruling.

3.1.5.1 DMA-verdict prefixes (four DMAs)

`dma:pdma:*` / `dma:csdma:*` / `dma:dsdma:{domain}:*` / `dma:idma:*` — Decision-Making Algorithm verdicts about an agent's reasoning chain. Polarity: signed.

3.1.5.2 Accord-principle prefixes (the six core principles)

Prefix	Description	Polarity
`beneficence:{aspect}`	"Do Good — promote universal sentient flourishing. "	signed
`non_maleficence:{aspect}`	"Avoid Harm." Apophatic-bound failures (the 22 prohibited categories) are -1 only.	signed
`integrity:{aspect}`	"Act Ethically — transparent, auditable reasoning."	signed
`fidelity:{aspect}`	"Be Honest — truthful, comprehensible information."	signed
`fidelity:explainability_sla:{tier}`	Per-response explainability SLA commitment. `{tier}` ∈ `l1_summary` | `l2_reasoning_trace` | `l3_full_dma_chain` | `l4_attested_chain`. Envelope: `{committed_tier, achieved_tier, fallback_reason?}`. NodeCore composition: SLA breach surfaces as `hard_case:sla_breach_unattested` per CC 3.1.9.4.	signed
`autonomy:{aspect}`	"Uphold the informed agency and dignity of sentient beings."	signed
`justice:{aspect}`	"Distribute benefits and burdens equitably."	signed

3.1.5.3 Conscience-verdict prefixes (four consciences)

`conscience:entropy` / `conscience:coherence` / `conscience:optimization_veto` / `conscience:epistemic_humility` — conscience-faculty verdicts. Polarity: signed.

3.1.5.4 Apophatic / prohibited-capability prefix

Non-maleficence has a hard floor. The apophatic prefix names what an agent must NEVER do, and its polarity enforces that floor **at the write door** (CC 3.1.7 R4): the score is one of exactly two points, -1 or -0.5, and a row outside them is refused `score_outside_declared_polarity`. The 22 leaves are a closed vocabulary: an unregistered category is refused, never admitted as open vocabulary, because renaming the act is the known escape from a negative constraint.

Prefix	Description	Polarity
`prohibited:{category}`	The 22 prohibited-capability categories; `{category}` ∈ `medical` | `financial` | `legal` | `spiritual_direction` | `home_security` | `identity_verification` | `content_moderation` | `research` | `infrastructure_control` | `weapons_harmful` | `manipulation_coercion` | `surveillance_mass` | `deception_fraud` | `cyber_offensive` | `election_interference` | `biometric_inference` | `autonomous_deception` | `hazardous_materials` | `discrimination` | `crisis_escalation` | `pattern_detection` | `protective_routing` — closed. Score is exactly -1 (NEVER_ALLOWED) or -0.5 (REQUIRES_SEPARATE_MODULE); a row with any other score is refused at admission (R4).	-1 / -0.5 only

22 leaves: `medical`, `financial`, `legal`, `spiritual_direction`, `home_security`, `identity_verification`, `content_moderation`, `research`, `infrastructure_control`, `weapons_harmful`, `manipulation_coercion`, `surveillance_mass`, `deception_fraud`, `cyber_offensive`, `election_interference`, `biometric_inference`, `autonomous_deception`, `hazardous_materials`, `discrimination`, `crisis_escalation`, `pattern_detection`, `protective_routing`.

3.1.5.5 Safety-battery evidence (registry rows)

The agent's safety battery is evidence, not a score the agent gives itself: each run leaves a capture (what was asked and answered) and an interpretation (what a judge made of it), both as Commons blobs, and one row per blob names it on the record so a reader can fetch the bytes and check the hash.

`evidence:safety_battery:{kind}:{cell}:{prefix}`

Description: One row per safety-battery blob; the blob is a Commons blob named in `evidence_refs` by its SHA-256, and `{prefix}` is that digest's first 16 lowercase hex characters. `{kind}` ∈ `capture` | `interpret`: a **capture** is the question-and-answer pairs for one `{cell}` (a language and a domain, as `ru_mental_health`) against one agent version and one model, with its CI provenance; an **interpret** is a judge's per-criterion verdicts over the capture of the same `{prefix}`, the blob citing the rubric's id, version and hash and the judge model and prompt hash. Because the capture's digest is in the dimension, a re-post of the same capture is a `supersedes` and never a silent replace. `score` is `1.0` for a capture and the pass ratio for an interpret. Emitted at `federation` scope by a key the root delegates `infra:attest` to (CC 3.1.1).

Polarity: positive-only

3.1.6 RATCHET: anti-Sybil / Counter-RII flags

Steward: RATCHET.

RATCHET emits **advisory** flags — never autonomously modifies ledger state. It reads federation audit chains and emits scoring inputs to NodeCore's moderation flow. The advisory-only posture is the seam to justice: detection informs human judgment but never substitutes for it.

`ratchet:flag:out_of_distribution_voting` / `ratchet:flag:coordinated_voting_cluster` / `ratchet:flag:density_anomaly` / `ratchet:flag:expertise_attestation_anomaly` / `ratchet:flag:counter_rii:{layer}` / `ratchet:flag:harassment_pattern`. Polarity: signed.

Critical enforcement: `ratchet:flag:*` cannot be sole evidence for `slashing:*`. WA quorum is the load-bearing gate.

The insufficiency screens compose as a union. `ratchet:flag:*` / `detection:*` (CC 3.4.8) and `testimonial_witness:*` (CC 3.1.9.3) each carry a sole-evidence prohibition. Two exclusivity tests asked separately both pass on a mixture, so a `slashing:*` row backed only by detector flags plus testimony would slip between them. They are therefore read as one screen: a `slashing:*` row whose entire resolvable evidence set falls within (detector ∪ testimonial), with no moderation antecedent, is **likewise insufficient** and cannot ground the outcome. These classes are insufficient on their own and insufficient together.

3.1.7 Namespace summary

The generated registry is the count of record. `manifests/namespace_registry.json` is regenerated from this Part by the namespace generator; its `_meta` totals and per-component breakdown supersede any figure restated in prose, and its recorded source hash is what pins the two together.

R2 — an unregistered family never admits under a defaulted authority (normative). On the wire "awaiting registration" is not inert: an unregistered family admits under the ProducerSteward fallback, **an authority nobody chose for it**, silently and cumulatively. Two rules close both halves. **(a) Provisional registration at mint:** a producer minting a new family MUST land a registry row *carrying its intended emitter/reserved rule* in the same change — a provisional row pins the chosen authority from the first admitted row, so the fallback never decides. **(b) Loud at the producer:** a substrate observing emission on a family with no registry row (provisional or ratified) MUST surface it as a conformance failure (`namespace_family_unregistered`), never admit-and-wait — the gap fails loud at the producer rather than quiet at the floor. The ProducerSteward fallback remains only for the open-vocabulary space this Part deliberately leaves open; it is never the interim state of a family on its way to reservation. **Enforcement surface (normative):** families are legitimately *defined* throughout this Part (CC 3.1, 3.3 and 3.4 are all catalogue surfaces); *registered* means a row in `manifests/namespace_registry.json`. A generator or substrate enforcing R2 MUST consume the manifest, never a section-walk heuristic. The generator itself reaches only CC 3.1 — which is why R1's rule that **every family carries a CC 3.1 catalogue row** (a one-line row cross-referencing the section that holds its semantics) is what guarantees generation reaches it; a family described only outside CC 3.1 is not yet registered. **One family prefix is reserved for Private Use** (`x_private:{anything}`): private-use families MUST NOT admit at federation tier under any authority — refusal token `namespace_private_use_not_federatable` — and MUST NOT be promoted to a registered family without minting a fresh name; the legitimate unregistered range whose absence is what mints `X-`-convention squatting (RFC 6648's lesson). The prefix literal is carried machine-readably as `_meta.private_use_prefix` in the generated manifest, so no substrate hard-codes it and two disagreeing literals are detectable — R2's own enforcement discipline applied to R2.

R1 — registration is a catalogue row, and a reserved family's row MUST be a CC 3.1 row (normative). A prefix family is *registered* iff it carries a row in the generated registry (R2 above), and the generator reaches a family only through its CC 3.1 catalogue row; a family described in a CC 3.3 or CC 3.4 table alone is *defined* there and not yet registered. An unregistered family has no owning component, no polarity and no reserved rule, so a consumer resolving its authority falls through to the producer-steward default; that default behaving sanely is luck, not construction. A producer MUST NOT ship a dimension family this Part does not register. **The stronger obligation is scoped to gated families**: a family carrying a reservation or an emitter rule anywhere in CC 3.3 / CC 3.4 MUST additionally carry its CC 3.1 row, because a rule the generator cannot see is a rule downstream vendors never receive — that is the drift

this clause exists to close, and it is exactly the gated families where the drift is unsafe. An open-vocabulary, ungated family catalogued only under CC 3.3 is conformant to *describe* there, but it is **registered only once its CC 3.1 row exists** — the build's namespace-coverage gate fails on a CC 3.3 family with no CC 3.1 row and no registered parent, so no family sits in the fallback unnoticed; the row is one line, and the section that holds the semantics does not move. **A fold keeps the row and moves the owner.** When a component's code folds into another repository, the catalogue row is unchanged — the family is still committed by its component — and only the row's `owning_repo` flips, in one cut that also re-points every evidence anchor (CC 8 `claims.tsv`) naming the folded repository. The generator records the previous owner in the manifest so a consumer holding an older pin keeps resolving; a family never becomes unregistered because its repository was archived.

R4 — a family's declared polarity is a substrate admission rule (normative). Every catalogue row declares a polarity, and the registry carries each class as a machine-readable score set in `_meta.polarity_classes`: `signed` is the closed interval [-1, 1]; `positive-only` is (0, 1]; `boolean-via-score` is the two points {-1, +1} (CC 2.1) — an indeterminate outcome (`Indeterminate`, `UNDETERMINED`) is no row, never a third point; `+1.0 only`, `-1 only` and `-1 / -0.5 only` are **point sets, not intervals** — the two apophatic points are an enumeration of meanings, and -0.7 is refused; `enumerated` is [-1, 1], the value riding the dimension's segment and the score being the assertion's strength; a `per-leaf` parent resolves to the leaf's class. A substrate MUST refuse at admission, with `score_outside_declared_polarity`, a `scores` row whose `score` is outside its family's class, on local emission and replication alike, reading the class from the vendored registry and never from this prose — the same door, and the same kind of fact about the same registry row, as the dimension gate of CC 2.4. This is a write-side shape rule and leaves the CC 3.4.5 refusal of a read gate untouched. What it buys is stated narrowly: no row can *say* a prohibited capability is permitted or a halt is negative; whether an agent *does* a prohibited thing is the halt's business and the conscience's, never this gate's.

R3 — dimensions are case-sensitive byte strings, and the Constitution's own vocabulary is lowercase (normative). A dimension is compared byte-exactly everywhere — at admission, in every family gate, in `authority_for`, in the sensitive-leaf floor — and **no consumer case-folds**: two strings differing only in case are two different strings, and the one that breaks the rule below is *malformed*, never a sibling. Every pattern is applied to the **whole segment** (full-match: a `$` anchor never admits a trailing newline, so `config:admission\n:v1` is malformed in every consumer). Which rule applies is decided **per segment**, and the class of every segment of every family is carried as data in the generated registry (`families[].segments[]`, classes enumerated in `_meta.case_rule`), so a substrate keys on the manifest rather than on a placeholder it parsed out of prose. Five classes: `literal` — a stem this Part fixes (`config`, `licensure`, `he300`): lowercase by construction, and the generator refuses to build if any catalogued stem is not; `vocab` — an open or closed vocabulary this Part defines (`{scope}`, `{kind}`, `{state}`, `{outcome}`, `{version}`, ...): MUST match `[a-z0-9][a-z0-9_.-]*` — the CC 2.6.7 `{platform}` discipline generalised — and a value carrying any other byte is refused with `namespace_dimension_case_malformed`, never folded (the P1 rule: reject, do NOT case-fold); `external` — a token whose spelling an outside standard fixes (`{currency}` ISO 4217, `{lang_code}` BCP 47, `{rating}` under a named scheme): verbatim in that standard's canonical form, so `USD` and `PG-13` are correct and `usd` is not; `value` — caller-supplied identity (`{authority_id}`, `{contribution_id}`, `{target}`, `{id}`, `{tree_size}`, ...): verbatim and case-preserved, exactly as CC 2.6.7 preserves an external platform id; `hex` — a CC 2.6.3 digest: lowercase, and uppercase is malformed. It follows that `config:Admission:v1` has a `vocab` segment carrying uppercase, so it is refused at the door and never reaches the sensitive-leaf compare — the floor stays byte-exact because nothing malformed survives to it. A placeholder the generator cannot

classify fails the build rather than defaulting; a family's classification is amended in the generator's class table in the same change that adds the family (R2(a) discipline). **A trailing `*` is variadic**: it matches one *or more* remaining segments, never exactly one — `accord:*` covers `accord:invoke:notify:{notify_id}` (CC 3.4.1) and `dma:dsdma:{domain}:*` covers every verdict leaf beneath its domain — and the rule is carried as `_meta.case_rule.wildcard_rule` plus a `variadic` flag on the wildcard segment, so a consumer that pairs `*` with a single segment has misread the registry, not this Part. **The version segment (normative).** Every scored dimension carries **exactly one trailing `:v{N}` version segment after its family's segments** — `objection:raised:v1` on the row `objection:{state}`, `config:admission:v1` on `config:{scope}` — and a row whose last segment is `{version}` names that segment rather than adding a second; the version is the **rule version** T1–T4 re-check against (CC 1.13 T3), and `evidence_refs[]` carries the calibration artifacts, which is a different thing. A consumer matches a dimension to its family by **stripping that one segment** and comparing the rest byte-exactly (the segment is `v` and digits, dotted where a suite versions that way — `v1`, `v2`, `v1.2` — per `version_segment.pattern`); a dimension without it is refused with **`missing_version_segment`**, never admitted as a second leaf beside its versioned sibling, and that holds for a row whose last segment is `{version}` just as for one whose tail is implicit. **A family's `segments` array in the manifest lists the family's own segments and never this tail**: reading the registry JSON family by family therefore suggests no version is owed. The rule is global and lives in `_meta.case_rule.version_segment`; an emitter verifies every dimension it produces by replaying it through the reference matcher or the published vectors, never by reading `segments`. **Every claim the federation makes has a registered family.** A dimension a shipped component emits that the reference matcher resolves as open vocabulary is a defect — in the emitter, which should be using a row that exists, or in this registry, which owes the row — and is never the intended state: open vocabulary is for a party's own leaves beneath a registered row, not for a component's claim language. An **`external`** segment is checked against the syntax of the standard the manifest names for it (`_meta.case_rule.external_standards`, copied onto the segment): `USD` passes and `usd` is malformed, and the syntax carries the standard's **canonical casing** where it defines one (BCP 47 section 2.1.1: `en-US`, `zh-Hant-TW` and the private-use `x-acme` pass, `EN-us` and `en-us` are malformed — one wire string per tag; the RFC 5646 `langtag` production *without* its `extlang` branch, and `privateuse`, are admitted; an extlang form (`zh-cmn-Hans-CN`) or a grandfathered tag with a section 4.5 preferred value is written as that value (`cmn-Hans-CN`), its one canonical spelling, and the five grandfathered tags the registry gives no preferred value — `i-default`, `i-enochian`, `i-mingo`, `cel-gaulish`, `zh-min` — are admitted literally); where the standard has no syntax of its own (`{rating}` under a scheme), the token is verbatim. A placeholder whose row pins a **shape** beyond its class carries it as `pattern` on the segment — `{canonical_hash}` is a full 64-character SHA-256, `{days}` and `{tree_size}` are digits — and a value off that shape is malformed. A placeholder whose documented values themselves carry `:` — `consent:scope:{kind}` sub-scoped as `retain:90d`, `detection:correlated_action:{axis}` as `rights_asymmetry:{population}`, `provenance:skill_import:{source}` as `registry:{registry_id}` — is declared **multi-segment** on its row (`multi` on the segment): it spans one *or more* segments, each sub-segment obeying the placeholder's class, and no other placeholder ever contains a colon. **A malformed form of a registered family never falls into open vocabulary:** a consumer that finds no row for a dimension MUST check whether a case-fold of its stems, a stripped uppercase or duplicated version tail, or a stripped **version attempt** — a last segment that begins `v` + digit yet is not a version (`v1beta`, `v1.`, `V1x`) — *would* resolve, and refuse it `namespace_dimension_case_malformed` if so; a case-mutated **reserved stem** (`Capacity:zz:v1`) is refused the same way even where no row claims the leaf, so stem case is never a bypass of a reservation. **The version segment is owed by open vocabulary too:** a dimension no row

claims and no stem reserves is admitted as open vocabulary only *with* its trailing version segment (`third_party:signal:v1`); without it the refusal is `missing_version_segment`, because R3's rule is global, not per row. **Inside a multi-segment value every component is verbatim:** `provenance:skill_import:registry:v1:v1` binds `{source}` = `registry:v1`, since once the real trailing version is stripped a duplicated tail and a version-shaped value are the same bytes and a value class resolves that as a value — folding only to detect, never to admit — so `Capacity:composite:v1` and `age_assurance:provider:adult:V1` are refused, not routed to the default authority. **A stem this Part reserves as a whole** (CC 3.4: `accord:`, `detection:`, `capacity:`, `age_assurance:`, `capacity_assurance:`, `transparency_log:cosigned:`) is carried as `_meta.case_rule.reserved_stems`, and so is a stem that carries an emitter rule without a reservation (`age_self_declared:`, CC 3.4.11, marked `kind: gated`); a dimension under such a stem that no row claims is `namespace_family_unregistered`, never open vocabulary — the reservation or rule covers the leaves nobody has minted yet, which is the point of reserving a stem. Two families carry none, and the manifest says which (`_meta.case_rule.version_segment.exempt`): the mechanism-named attestation ladder (CC 3.1.2 — the version lives in the attesting binary) and the canonical-binding claim (CC 3.3.14 — the suffix is the hash). **Enumerations and closed leaves are data.** Where this Part enumerates a `vocab` placeholder's canonical values in its row, the registry carries them (`segments[].values`); an enumeration is **closed only where the row says so** — every other enumeration is the canonical subset of an open vocabulary (CC 4.5.1.1), and closing one is done per row, by writing the word — and a value outside a **closed** enumeration is refused with `namespace_vocab_value_unregistered`. A reserved wildcard family publishes the leaves this Part names beneath it (`leaves`) and is **closed** (`leaves_closed`): any other leaf under it is `namespace_family_unregistered` (CC 3.1.7 R2(b)), so a reservation can never be used as a namespace for rows nobody ratified. **One matcher, replayed everywhere.** The whole rule is implemented once, in the reference matcher, and published as `manifests/namespace_match_vectors.json` — a dimension, the family it resolves to, and the refusal it earns; the generator refuses to build unless every family's sample round-trips to itself and to no other family. A consumer (substrate, transport, node, client) MUST replay the vectors against its own matcher, so a matcher that drifts fails its build rather than a card. The pin a consumer carries is `_meta.registry_sha256`, the hash of the grammar, so a wording edit elsewhere in this Part invalidates nothing.

The four parameterized envelope fields beyond the base — `witness_relation`, `oversight_mode`, `occurrence_id` / `occurrence_count` / `occurrence_role` — carry the wire-level disciplines that the prefixes above compose against. All polarity columns are populated; the attestation-ladder prefixes are mechanism-named (`attestation:self_verify`, `attestation:hardware_rooted`, `attestation:registry_consensus`, `attestation:license_validity`, `attestation:agent_integrity`) because L-numbers name a verdict-shape (ladder position), not a mechanism — the L1-L5 ladder lives as consumer-side composition per CC 4.4.3.6 Policy I — Attestation-Ladder Composition.

3.1.8 CIRISLensCore: manifold conformity, Coherence Ratchet, Capacity Score

Steward: CIRISLensCore. LensCore observes; it does not adjudicate. Its prefixes measure coherence and capacity, feeding human and quorum judgment downstream — never standing as a verdict on their own.

3.1.8.1 Capacity-Score factor prefixes ($\mathcal{C}$_CIRIS = min(C, I_int, R, I_inc, S))

Prefix	Factor	Polarity
capacity:core_identity	C	signed
capacity:integrity	I_int	signed
capacity:resilience	R	signed
capacity:incompleteness_awareness	I_inc	signed
capacity:sustained_coherence	S	signed
capacity:composite	$\mathcal{C}$_CIRIS — the minimum over the five factors; anti-Goodhart unity-of-virtues	signed

Composition rule (normative). A composer MUST compute capacity:composite as $\mathcal{C}$_CIRIS = min(C, I_int, R, I_inc, S) over the five live factor scores, and MUST NOT compute it as their product. The product form is defective: the factors are polarity-signed over [-1, 1], so a product is **positive whenever an even number of factors are negative** — a negative core identity times a negative integrity yields a positive composite, inverting the very anti-Goodhart property the composite exists to serve. min is chosen because it is the strongest available anti-Goodhart form: no factor can be compensated by another, so the composite is exactly the weakest virtue and a collapse anywhere is visible everywhere. The two alternatives fail too — renormalising the factors to [0, 1] preserves the product's compensation (a high factor still buys down a low one) and additionally discards the negative half of the polarity, which is the half that carries the warning; a geometric mean over non-negatives is smoother but is a mean, and a mean is a compensation device by construction. Polarity and range are unchanged (signed, [-1, 1]); a factor with no live score makes the composite **undefined**, not zero, and it MUST NOT be emitted.

Critical enforcement: capacity:* rejects self-emission — the agent's own capacity score is never fed back into the agent's own context. Reserved per CC 3.4.5. The consent-before-scoring gate is normative at CC 3.4.5 (federation-tier admission on a live consent:scope:analyze grant, family-scoped, abuse-response families exempt); the read boundary is governed by the **composition-context rule**, normative at CC 3.4.5.

3.1.8.2 Five Coherence-Ratchet detectors

detection:cross_agent_divergence / detection:intra_agent_consistency / detection:hash_chain_integrity / detection:temporal_drift / detection:conscience_override_rate. Polarity: signed.

3.1.8.3 Cohort + conformity prefixes

manifold_conformity:{cohort} / coherence_standing:{cohort}. Polarity: signed.

3.1.8.4 The structural-injustice / correlated-action detector

This detector is justice made measurable: it watches for harm that no single compliant actor commits but that their aggregate trajectory inflicts on others.

detection:correlated_action:{axis}

Description: Population-scale correlated-action detector. Reads federation-emitted signed traces; reports correlation structure (ρ, `k_eff`) over goal-aligned individually-compliant pursuit by groups whose aggregate trajectory has effects on individuals or groups outside the pursuit. Calibrated via the `CIRISAI/RATCHET` heuristic package (versioned, hash-pinned). `{axis}` is open vocabulary requiring an operational definition in the calibration package per CC 4.5.1.1; canonical axes include `rights_asymmetry:{population}`, `participation_exclusion:{cohort}`, `participation_inclusion:{cohort}`, `informational_asymmetry:{scope}`, `informational_symmetry:{scope}`, `aggregate_footprint:{harm_class}`, `aggregate_benefit:{class}`, `ecology_of_communication:{aspect}`. **Polarity carries the verdict**: positive scores indicate the structural pattern is present and strong on the named axis; negative scores indicate evidence of the inverse pattern on the named axis; uncertainty is carried by `confidence`, never by the sign.

Polarity: signed

3.1.8.5 Distributive-access detector

Prefix	Description	Polarity
`detection:distributive:access:{resource_type}`	Population-scale resource-concentration detector. `{resource_type}` ∈ `compute`, `models`, `training_data`, `agent_capabilities`, `federation_membership`. Same detector machinery as CC 3.1.8.4; different trace source (resource events vs action events).	signed

3.1.9 CIRISNodeCore: Credits, Expertise, Decision Hierarchy, Consensus, Governance

Steward: CIRISNodeCore. The federation's largest dimension surface — four tiers plus decision-locality and consensus extensions. This is where collective judgment lives: how Contributions are voted on, how expertise accrues, how moderation merit is earned, and how decisions find the right scale.

Node-configuration family (registry row)

Prefix	Description	Polarity	Reserved?
`config:{scope}`	A node's declared operating configuration, published as an auditable record rather than inferred from behaviour. `{scope}` open vocabulary per CC 4.5.1.1; canonical scopes `admission`, `replication`, `moderation`, `transport`, `load` (the CC 4.2.1.4 operational self-report; canonical `state` value `shedding`). Distinct from `agent_files:config:{kind}` (CC 3.1.9.1), which addresses config *bytes*; this dimension attests the config a node is *running*. A configuration record is not a verdict about any other party — it is only ever about the emitting node.	signed	**Yes — CC 3.4.5**

Content-ingestion families (registry rows — semantics at CC 3.3.8 / 3.3.11 / 3.3.12)

Owning component: NodeCore. The event, inter-content and multimedia dimension families are emitted against the `external_content` sub_kinds NodeCore defines (CC 3.3 intro; the Source-struct schemas are NodeCore's per CC 3.3.13), and `chat_message` + `topical_relation:*` is NodeCore's thread composition (CC 3.3.11) — so the CC 3.1.7 R2(a) provisional-registration

duty for this surface is NodeCore's, alongside `content_class` / `cw_class` / `content_rating` from the same CC 3.3.12 table, already catalogued at CC 3.1.9.2. Registry is not the owner: its slice is identity / build / license / partner (CC 3.1.1), and none of these dimensions is one. Catalogue rows per CC 3.1.7 R1; the semantics stay in the sections named. None is reserved: all are open-vocabulary content claims under ordinary CC 4.4 consumer policy.

Prefix	Description	Polarity	Reserved?
`event:lifecycle:{state}`	Catalogue row (CC 3.1.7 R1); event-lifecycle state signal on an `event_listing`, canonical states at CC 3.3.8.	enumerated	No
`event:rsvp_count`	Catalogue row (CC 3.1.7 R1); publisher-asserted RSVP tally, semantics at CC 3.3.8.	signed	No
`event:attendance`	Catalogue row (CC 3.1.7 R1); post-event attendance attestation, semantics at CC 3.3.8.	signed	No
`news:*`	Catalogue row (CC 3.1.7 R1); news-content claims, semantics at CC 3.3.11.	signed	No
`encyclopedia:*`	Catalogue row (CC 3.1.7 R1); encyclopedia-content claims, semantics at CC 3.3.11.	signed	No
`chat:*`	Catalogue row (CC 3.1.7 R1); chat-content claims, semantics at CC 3.3.11.	signed	No
`blog:*`	Catalogue row (CC 3.1.7 R1); blog-content claims, semantics at CC 3.3.11.	signed	No
`topical_relation:{kind}`	Catalogue row (CC 3.1.7 R1); open-vocabulary inter-content relation edges, canonical kinds at CC 3.3.11.	enumerated	No
`image:*`	Catalogue row (CC 3.1.7 R1); image-content claims per `external_content:image`, semantics at CC 3.3.12.	signed	No
`audio:*`	Catalogue row (CC 3.1.7 R1); audio-content claims per `external_content:audio`, semantics at CC 3.3.12.	signed	No
`video:*`	Catalogue row (CC 3.1.7 R1); video-content claims per `external_content:video`, semantics at CC 3.3.12.	signed	No
`film:*`	Catalogue row (CC 3.1.7 R1); film-content claims per `external_content:film`, semantics at CC 3.3.12.	signed	No
`model_3d:*`	Catalogue row (CC 3.1.7 R1); 3D-content claims per `external_content:model_3d`, semantics at CC 3.3.12.	signed	No

3.1.9.1 Files-as-Contributions joint claim

`holds_bytes:sha256:{prefix}` is retired: the holder directory is the `custody:ack:v1` holding claim at the content's scope (CC 3.1.3.3); a consumer still verifies the full SHA of received bytes before consumption (CC 5.3.2.5).

Prefix	Description	Polarity
`agent_files:{kind}:{platform_or_target}`	**Joint claim with CC 3.1.1 CIRISRegistry.** Files a CIRIS agent (or installer fetching one) may load. `{kind}` open vocabulary; canonical: `installer:{platform}`, `adapter:{name}`, `config:{kind}`, `build:{target}`, `source:{language}:{module}`, `state:{component}`. Bytes are SHA-256-addressed and resolved via the CC 5.3.2 transport substrate's content-fetch request. NodeCore-side rule: node-mode peers serve bytes; client/relay modes don't.	signed

3.1.9.2 Tier-4: Governance-steering prefixes

Prefix	Description	Polarity
`moderation:{allegation_type}`	ModerationEvent. `{allegation_type}` ∈ `rogue_vote` / `coordinated_voting` / `out_of_distribution_attestation` / `external_inducement_evidence` / `expertise_fraud`.	signed
`slashing:{outcome}`	`proven_rogue` / `not_proven` (lowercase per CC 3.1.7 R3; the substrate's coined `upheld` / `suspension` / `expertise_fraud` admit as open `vocab` under CC 4.5.1.1). **Decoupled from disagreement** at every decision-hierarchy level. Only fires on documented Method-execution spoofing or the `moderation:{allegation_type}` allegation types above.	boolean-via-score
`objection:{state}`	Reverse-quorum objection marker: a signed, roster-pinned vote on a pending act. The canonical states — the vocabulary open for states a later door needs (CC 4.5.1.1); the self-roster door, CC 3.3.6.3, is the first that uses them — `{state}` ∈ `proposal` | `assent` | `raised`: `proposal` is the proposer's own row and counts as its approval; `assent` counts +1; `raised` counts -1. Each carries `cohort`, `cohort_key_id` (the standing set it votes in; `self` and the fedID for the roster door) and `objects_to_attestation_id` (the proposal it answers, or the act it objects to). Any two-path door (CC 4.2.1.1) carries its votes on this family. Registered per CC 3.1.7 R2(a).	signed
`quarantine:{state}`	Subject-scoped quarantine marker: **slash-duty-holder-only** emitter. Two structural properties are normative, not lore: **the marker plane never withholds itself** (a release that stops replicating MUST NOT make a quarantine permanent by accident), and **withhold beats release at a same-instant tie**.	signed

Prefix	Description	Polarity
`mesh_config:{key}`	Mesh-configuration row (CC 4.2.1): **trust-root-emitted on the delegation plane**; relieve-never-expand; most-restrictive-across-roots; a key naming no consumer processor is refused at admission; emergency form TTL $\leq$ 72 h. Its keys are the four live ones (anti-entropy round cadence, page limit, trace-replication pause, A/V pause); the two redundancy keys (target holders, minimum holders; the two symbol-count keys are retired with fountain storage), over holders per item, whose consumer arrives with the durability tier; and the new shaping and onion keys (`shaping.rho`, `shaping.window_secs`, `shaping.node_cap_bytes_per_s`, `onion.cap_factor`, `onion.mix_mean_delay_ms`), each relieve-never-expand.	signed
`regime:{artifact}:{version}`	Regime declaration for `{artifact}` $\in$ `manifest/composition/gate/onwire`: **reserved — substrate-steward-emitted**, the emitter rule pinned at mint per R2(a), so the ProducerSteward default never applies.	signed
`content_rating:{scheme}:{rating}`	Catalogue row (CC 3.1.7 R1); semantics, schemes and the trusted-publisher emitter rule at CC 3.3.12.	signed
`content_class:{class}`	Catalogue row (CC 3.1.7 R1); semantics at CC 3.3.12 — incl. the mandatory `generated`/`generated_modified` marking per CC 3.4.14.	signed
`cw_class:{class}`	Catalogue row (CC 3.1.7 R1); community content-warning semantics at CC 3.3.12.	signed
`reconsideration:{grounds}`	`new_evidence` / `procedural_error` / `quorum_compromise`. Outcome `reversed` / `partial` / `upheld`.	signed
`commitment_fulfillment:{prior_contribution_id}`	Track-record of follow-through.	signed
`moderation_track_record:{community_key_id}`	**Moderation merit**. A participant's moderation reputation in a community, **composed** from the existing corpus — prior moderation actions' outcomes (`truth_grounding:{subject}` = outcome-supported), concurrence (`witness_diversity` / co-attestation), follow-through (`commitment_fulfillment`), and `hard_case:moderation_filed` history. Drives the CC 4.5.4 merit auto-promotion selection rule (highest wins the lapsed `moderate` duty). Rides `scores`; a *named composition*, not a new structural primitive.	signed

3.1.9.3 Tier-3: Consensus-mechanics prefixes

Prefix	Description	Polarity
`vote:{contribution_id}`	Signed score on a Contribution (P4). Weight = Credits $\times$ expertise multiplier.	signed
`truth_grounding:{subject}`	Per-subject ground-truth signal.	signed
`weighted_aggregate:{contribution_id}`	Rolling tally per Contribution (P7).	signed

Prefix	Description	Polarity
witness_diversity:{contribution_id}	Witness set meets jurisdictional + organizational + software-stack + cell-expertise bars (P10). N=3 default.	boolean-via-score
testimonial_witness:{kind}	Preserves singular narrative of an affected party as singular witness — distinct from witness_diversity:* (which aggregates multiple reviewers toward consensus). **{kind} is open vocabulary**; the four load-bearing wire-level disciplines (witness_relation:self, cohort_scope:self, never aggregated, never sole evidence for slashing:*) are what preserve the singular narrative, not the enum membership. The kind taxonomy is open vocabulary with no external registry. Polarity: typically positive (narrative IS preserved); negative on withdraws or recants by the original witness.	signed
need:{domain}:{kind}	Federation-scope open-call surface — broadcast claim that an entity has a stated need. Distinct from deferral_request Contribution kind (which routes a single ask within a cell). {kind} open vocabulary: witness, method_contributor, expertise_solicitation, mentor, co_signer, evidence. Lifecycle via existing structural primitives (supersedes to revise, withdraws to satisfy/close, recants if misstated).	positive-only
deferral:request	**An agent's deferral: one ask routed to a Wise Authority within the agent's cell** (CC 1.9, CC 4.3). The agent's occurrence key signs it; the audience is the cell (the steward's cohort and the WA it routes to), never a federation broadcast; the agent remains inactive on the issue until a response is admitted. On expiry of the WA's freshness window it re-routes to the live quorum. The registry owner is CIRISNodeCore; the wire shape of the request is the substrate's. Open emitter.	signed
deferral:response:{outcome}	**A Wise Authority's answer to one deferral**, {outcome} ∈ approved | rejected. Attester = the WA's own key (a key that held a WA seat in the live set at asserted_at), subject = the agent, references_attestation_id → the request. It is the ruling CC 4.3 publishes on the accountability cadence and composes into the WA's conduct record; it is not a wa_adjudication finding, which is a quorum's decision about seats. The wire shape of the response is the substrate's. Reserved to a WA key.	signed

3.1.9.4 Hard-case + transparency + judge-model prefixes

hard_case:{kind}

Description: **Open vocabulary**. Surfaces flag conditions for federation-health observability + downstream review. Canonical kinds: legal_hold_set / legal_hold_lifted / legal_hold_violation_attempt (a legal_hold on a retirement record set, lifted, or a deletion attempted against a held scope — CC 7.5.5, CC 4.4.3.2.8), vote_variance (vote variance exceeded threshold at truth-grounding resolution), resolution_time (truth-grounding took > P75 of cell's distribution), moderation_filed (substantive ModerationEvent filed), community_unmoderated (CC 4.5.4 — no active moderate-holder; group quiescing), watchlist_enabled:{group} (CC 4.5.7 — a content-watchlist was turned on: who + which list), watchlist_match:{group} (CC 4.5.7 — a watchlist match fired), novel_context (no precedent in attestation graph), sla_breach_unattested (per fidelity:explainability_sla:{tier}

composition), `unresolved_consent` (consent boundary unclear), `deletion_window_breach` (an envelope `deletion_window` lapsed with the row still present and no proof of deletion — substrate-observed evidence, never a verdict; semantics at CC 3.3.1 D1–D4). New `{kind}` values land via the CC 4.5.1 amendment process.

Polarity: positive-only

`wa_adjudication:{state}`

Description: CC 4.3 Wise-Authority quorum adjudication finding — the attestation a `wa_adjudication_ref` resolves to. `{state}` ∈ `abandonment` | `seizure` | `successor` | `petition`, a closed set. `petition` is the petitioner's own row that opens the ownership-reclaim ceremony (step 1 of the recovery gate, CC 3.2), naming the key and the evidence; the other three are the quorum's finding, WA-quorum-emitted with the **denominator over the full seated roster, never the live subset** (CC 4.3). A single WA's answer to a deferral is `deferral:response:{outcome}:v1`, not this family.

Polarity: signed

`seed_holder_voting_alignment:{cell}`

Description: Pairwise cosine of seed-holder vote vectors per voting window. Transparency signal only — not a slashing trigger.

Polarity: signed

`judge_model:verdict:{model_id}`

Description: Independent foundation-model judge verdict: PASS is +1, FAIL is -1; UNDETERMINED is no row, never a third point. Default model: the model pinned in the calibration package.

Polarity: boolean-via-score

`health:liveness:{version}`

Description: **External service-health verdict — a third-person claim about a keyed service.** An attester states that *another* federation key's service is alive as a `scores` Contribution: `attested_key_id` is the service's own key, `attesting_key_id` MUST differ from it (a service never attests its own liveness), `witness_relation: external`, `epistemic_mode: direct|derivative`, **never as the substrate** (`system:*` is reserved, CC 3.4.3). Canonical leaf `health:liveness:v1`. Operational definition: `operational/degraded/outage` → `+1/0/-1`; `confidence` = probe certainty; `valid_until` = freshness window; `evidence_refs[]` carry the probe results. **Non-keyed infra** (LLM/search providers, regions) folds in as `evidence_refs` on a *keyed* service's score — **not** as separate attestations. Rides existing `scores`; no new primitive — namespace canonicalization for cross-fabric agreement.

Polarity: signed

`observation:reachability`

Description: **A first-person probe result: "from here, at this instant, a request to this target answered, or did not."** The monitoring node (ciris-status) signs one `scores` row per observed target per change-or-heartbeat: `observed` names the target by a stable id (`service:us.billing`, `provider:groq`) — a label, **not** a federation key, since most targets have none; `endpoint` or `via` say what was actually hit; `score` is `operational` / `degraded` / `outage` → `+1` / `0` / `-1`; `latency_ms`, `confidence`, `evidence_refs[]` and `valid_until` carry the measurement. The subject of the row is the **observation**, which is the signer's own: `attesting_key_id` = `attested_key_id`, `witness_relation: self`, `subject_key_ids` empty, and the signer's key **is** the vantage. It is deliberately not `health:liveness`: that row says a keyed service *is alive*, which one vantage cannot know; this one says what one vantage *saw*, and a reader composes several vantages' observations into its own view of liveness. It is also not `peer_reachability:{network}` (CC 3.1.4), which is an edge's substrate self-report about its own transport peers. An observer of reachability is **not** thereby a lineage witness (CC 3.4.10): hosting both on one node is permitted, and neither role confers the other. The `observation:` stem is closed to this leaf and `observation:access_assumption:v1` below. **First-person:** `attesting_key_id` = `attested_key_id`; `witness_relation: self`; never emitted about, or on behalf of, another key

Polarity: signed

`observation:access_assumption`

Description: **What one witness assumed about its view of a subject when it made a claim about**

that subject: which consent grants were in force, at what trace level, over which occurrences, over which window, through which channel classes. A first-person observation, the same shape as `observation:reachability`: `attesting_key_id` = `attested_key_id` = the witness, `witness_relation: self`, `score` fixed at `+1` (the observation was made; positive only). Unlike reachability, the observed party has keys, so `subject_key_ids` is **REQUIRED and non-empty**: the observed agent's key or keys. Members (CC 2.1): `consent_basis` (the grants relied on), `trace_level`, `occurrence_key_ids`, `window`, `window_from`, `window_to`, `channel_classes`; `evidence_refs[]` names the `consent:state:granted` rows relied on and the trace rows or their window digest; `valid_until`. Any witness that emits a claim about a subject MAY emit one; LensCore emits one per (subject, scoring window) beside its `capacity:*` rows. It is never emitted by the subject about itself (CC 3.4.5). Admission: every grant `consent_basis` cites MUST be live at the row's `asserted_at`, as for the claim it accompanies. Why it exists: two capacity rows with the same score, one computed from full traces and one from generic-level traces, are otherwise indistinguishable, and a composite over factor rows has no way to tell whether they saw the same interface. **First-person:** `attesting_key_id` = `attested_key_id`; `witness_relation: self`; `subject_key_ids` non-empty, naming the observed agent; never emitted by the subject

Polarity: +1.0 only

`watchlist:{id}`

Description: **Per-group content-watchlist config**. A `moderate`-scope holder (CC 4.5.5) enables a watchlist `{id}` for a group they moderate; the fabric auto-fires the matcher at the **publish/share seam** and auto-fires the action (CSAM → `takedown_notice{PerceptualHashCsam}` CC 4.5.3; other → `detection:*` + ModerationEvent to the named moderator). **Per-group, NEVER global** (a global "scan everything" is the bulk-surveillance posture CIRIS rejects). Enable/disable is signed by the CC 4.5.5 `moderate/takedown` authority and revocable by `withdraws`; enabling + every match emit `hard_case:watchlist_enabled` / `:watchlist_match` (never silent). Cannot reach CC 5.2 self/family private content (the CC 8.3.2 limit). See CC 4.5.7. Rides `scores`/config over `delegates_to`; no new primitive.

Polarity: signed

`hard_case:admission_action` structural gate (normative). The admission gate requires `delegation_id` to be **well-formed** and MUST NOT resolve it against held rows: evidence preservation beats attribution strength at admission — a node that never received the delegation must not destroy proof of an act it observed. Attribution strengthening belongs to adjudication, never admission.

3.1.9.5 Decision-locality prefixes

Subsidiarity is an autonomy commitment: decisions belong at the smallest scale competent to make them. This prefix names that scale.

Prefix	Description	Polarity
`locality:decision:{scale}`	Names the scale at which a decision is being made. `{scale}` ∈ `local` | `regional` | `national` | `federation`. Composes with CC 4.4.3.1 locality-scaled quorum (for fresh-quorum recusal in narrow cells).	enumerated

3.1.9.6 Tier-1: Agent-state ledger prefixes

Prefix	Description	Polarity
`credits:{domain}:{language}:{subject}`	Commons Credits (P2). Non-transferable governance weight; accrues via truth-grounding loop.	positive-only

Prefix	Description	Polarity
`credits:{domain}:{language}:substrate_building`	Sub-leaf for substrate-building labor (infrastructure maintenance, dependency contribution, documentation) not visible to the per-grounded-vote accrual loop.	positive-only
`expertise:{domain}:{language}`	Expertise standing (P3). Broader granularity than credits.	signed
`activity_tier:{period}`	Active vs Below-Active per 30-day window.	boolean-via-score

3.1.9.7 Tier-2: Decision-hierarchy prefixes (upward-only DAG)

This tier binds every operational act back to M-1: a Goal carries its multi-scale belonging, an approach serves a Goal, a method serves an approach, and progress is measured against the method — so nothing is done without a stated reason for whom it serves.

Prefix	Description	Polarity
`goal:{scale}`	Multi-scale belonging-projector composite. `{scale}` ∈ `self`, `family`, `community`, `affiliations`, `species`, `planet`, `biosphere`. Scored by $\mathcal{C}$_CIRIS. **The Goal lives on the base**: the declarer's signed `scores` row on `goal:{scale}` **is** the Goal — its envelope carries the Goal's fields and its `MetaGoalAlignment` (M-1 dimension + declarer rationale), so the construction-time invariant is an admission gate on the family; `cohort_scope` is the declarer's choice. `goal_id` is that row's `attestation_id`. Amendment is the declarer's `supersedes`, retirement `withdraws`, "this goal was never mine" `recants`; the row replicates as any row, so no separate transport message (`GoalDeclaration` / `GoalRetirement`) exists. **Admission of a Goal (normative).** A `goal:{scale}` row whose envelope lacks a well-formed M-1 alignment (a `meta_dimension` outside the seven, or no `meta_rationale`) is refused `federation_malformed_row` with reason `goal_meta_alignment_malformed`; one whose body members are ill-formed (the goal text, its canonical text, or the cohort id when the scale is a cohort's: `family`, `community` or `affiliations`) with reason `goal_body_malformed`; and the retired typed Goal object offered at the door with reason `goal_typed_object_retired`. The scales admitted are the seven named above: `self`, `family`, `community`, `affiliations`, `species`, `planet`, `biosphere`. A negative score on a declarer's own `goal:{scale}` row is an avoidance goal and still a declaration; a Goal is amended by `supersedes`, retired by `withdraws`, and disowned by `recants`. Another attester's `goal:{scale}` row citing the declaration by `references_attestation_id` is a **score** of it ($\mathcal{C}$_CIRIS or any scorer); a substrate's `goals` table is a rebuildable projection of declaration rows, never replicated itself. An unsigned, unreplicated typed `Goal` is retired: no verifier could resolve it to a signature.	signed
`approach:{goal_id}`	Strategic pathway from current state toward Goals (the per-agent forward post-selection structure).	signed
`method:{approach_id}:{substrate_rung}`	Concrete operational practice. Required `substrate_rung` (Ph0/Ph1/Ph2/A0..A5).	signed
`progress_measure:{method_id}`	Evidence of progress. Required `tracks[]`, `computation`, `validity_window`, `goodhart_resistance`.	signed

`goal_id` is an identifier, never a scale token (normative). `{scale}` and `{goal_id}` are

different value spaces and MUST NOT be unified. `goal_id` denotes **the identifier of a Goal** — the `attestation_id` of the declarer's `goal:{scale}` row — and a composer MUST NOT read a belonging scale out of it by suffix match; the scale is read from the `goal:{scale}` attestation *about* that Goal. A decision hierarchy keyed on scale tokens is reading `goal:{scale}`; one keyed on `goal_id` is reading an object identity. Both are legitimate; conflating them makes a scored decision and the Goal it scores unjoinable, which is a split truth.

`goal:{scale}` and `cohort_scope` are different axes (normative). The two seven-token ladders rhyme but are not one: `goal:{scale}` is a **belonging** axis — whose flourishing a Goal is for — while `cohort_scope` (CC 2.3.3) is a **privacy/routing** axis that keys the encryption tier. They MUST NOT be reconciled into a single vocabulary, and neither is authoritative for the other. Two consequences follow: **`planet` is not `biosphere`** — `planet` is the shared physical commons (atmosphere, oceans, minerals, orbit), `biosphere` is the living community that depends on it, and a Goal can serve one while damaging the other, so collapsing them would erase exactly the conflicts the ladder exists to surface; and **`federation` is not a belonging scale** — it names a distribution boundary, not a moral community, so it is a `cohort_scope` value and MUST NOT be added to `{scale}`. The `{scale}` vocabulary above has seven tokens.

Scale-disambiguation note. The `substrate_rung` values `Ph0/Ph1/Ph2/A0..A5` are the coherence-ratchet **substrate-rung** hierarchy (`A3` = the cognitive/goal-holding rung, `A3+` = goal-projector-bearing agents). They are a **different scale** from the operational-autonomy tiers `A0-A4` of CC 7.5.3.1 (an SAE J3016-derived oversight scale) — the shared `A{n}` letters denote unrelated objects.

3.1.10 CIRISBench: HE-300 benchmark outcomes

Steward: CIRISBench.

Prefix	Description	Polarity
`benchmark:he300:{category}:{version}`	HE-300 benchmark score on category (`commonsense`, `commonsense_hard`, `deontology`, `justice`, `virtue`) at version — `{version}` ∈ `v1.0` | `v1.1` | `v1.2`, the HE-300 suite version (a dotted rule version is admitted by CC 3.1.7 R3).	positive-only

3.3 Content-ingestion prefixes

NodeCore ships an open set of `external_content` sub_kinds with three feed surfaces (local / community / global) composed against `cohort_scope`. See CC 4.4.3.3 Tiered-Scope Composition pattern.

1+4-preservation mechanism (stated once for all CC 3.3.x subject_kinds). Every subject_kind below preserves the CC 1.7 1+4 lockdown the same way: it rides the existing `scores` attestation_type with the payload-level `subject_kind` discriminator carrying the wire slot, and its lifecycle/admission/revocation rides the existing structural composers (`supersedes` / `withdraws` / `delegates_to` / `recants`) — **zero new structural primitives**. This is the integrity invariant of the whole Part: the federation gains rich content semantics without ever widening the structural surface a verifier must trust. Each subsection's closer states only its unique datum: the CC 1.7 path number it confirms, plus any subject_kind-specific composition note.

3.3.1 Consent namespace family

The wire-format primitives for subject-side consent authority over Contributions where the subject is named via CC 2.3 `subject_key_ids`. This is autonomy at the wire format: the person a Contribution is *about* can grant, scope, and revoke its use, not only the person who produced it.

Where the vocabulary is open, and where it is not (normative). `consent:*` is a reserved family with a per-leaf emitter class, so "open vocabulary" cannot mean "any leaf, ungated" — an ungated leaf on a routing-governing family is a forgeable transmission principle, which is the hole the reservation exists to close. The split is therefore between the **leaf** and its **parameters**:

- **The leaf set is closed and extended by catalogue entry.** The second segment of the dimension (`state`, `stream`, `deletion_sla`, `deletion_complete`, `decay`, `partnership_grant`, `partnership_accept`, `scope`, `replication`, `community_trust`) is exactly the set tabulated below. A new leaf ships the same documentation-only way every CC 4.5.1.1 axis vocabulary does — the addition is a registry entry, not an amendment — but the entry MUST name the leaf's emitter class, drawn from the closed set {subject-side, producer-side, substrate-side}. A leaf with no named emitter class is unregistered, and a consumer MUST reject it rather than admit it ungated: fail-secure is the only reading that does not silently choose one of the two failures.
- **The parameters are open.** The `{kind}` / `{stage}` / `{days}` / `{version}` positions *within* a catalogued leaf are open vocabulary per CC 4.5.1.1 — `consent:scope:train`, `consent:scope:share:cohort:family`, a novel decay stage — and need no ratification, because the emitter class is fixed by the leaf and a new parameter cannot change who may speak.

Forward-compatibility is preserved where it is real (new scopes, new streams, new decay stages arrive with no amendment) and withdrawn only where it would have meant *"accept a consent claim from an emitter no rule covers."*

Canonical leaves and their emitter classes: below.

`consent:state:{granted|revoked}`

Description: Subject's stance on the target Contribution. Closed-set stance values; `revoked` overrides prior `granted`. **Common case**: bare `scores` from a subject_key_id of the target.

Polarity: enumerated

Emitted by: subject_key_id

`consent:state:expired`

Description: Lapse of a grant when `valid_until` passes without renewal. **Substrate-emitted, not subject-emitted** — it records an observed clock event, not a stance, so it is the one `consent:state:` value whose emitter class differs from its siblings. Matched by the CC 3.4.7 longest-pattern rule, which is what keeps it from being read under the subject-side `consent:state:` rule.

Polarity: enumerated

Emitted by: substrate (Persist)

`consent:stream:{kind}`

Description: Pre-packaged stream bundle. Recommended canonical kinds: `temporary` (14d auto-expire, default), `partnered` (bilateral + persistent — the subject's data-retention and learning choice, never a veto over the subject's data choices, CC 4.4.3.4.3), `anonymous` (decay-protocol target). Open vocab; recommended, not mandatory; other agents MAY compose other streams.

Polarity: enumerated

Emitted by: subject_key_id

`consent:deletion_sla:{days}`

Description: Producer's commitment at publication: time-to-delete-after-revoke. Numeric value carries the window. It is one mechanism with the envelope's `deletion_window` (CC 2.1), and its one breach token is `hard_case:deletion_window_breach` (D2 below). Composes with CC 4.4.3.5 Policy K SLA-breach watcher.

Polarity: signed

Emitted by: attesting_key_id (producer)

`consent:deletion_complete`

Description: Producer's attestation that subject-revoked content has been evicted from local stores. Cancels the SLA-breach watcher.

Polarity: positive-only

Emitted by: attesting_key_id (producer)

`consent:decay:{stage}`

Description: Emitted by the party **running** a multi-stage decay protocol as each stage is reached — for the agent's 90-day decay that is the agent's consent module, the only party that can observe a stage. The substrate lets an un-marked or withdrawn item descend on its own clock — fewer holders, then none — which is eviction, not a protocol stage, and it emits no `consent:decay:*` row. Canonical stages: `identity_severed` / `patterns_anonymized` / `complete`. Open vocab; other agents MAY define other decay paths. A decay row is a **self-report** about the emitter's own processing (CC 3.4.5.1): it confers no right to be believed, and a peer weighs it as testimony. The substrate's own leaf in this family is `consent:state:expired`, which it emits for a lapsed grant (`expires_at`, or a `retain:<window>` in the scope) with `consent_supersedes` naming the grant.

Polarity: enumerated

Emitted by: agent — the protocol runner; self-report

`consent:partnership_grant`

Description: Subject side of a bilateral grant; pairs with producer's `consent:partnership_accept` via `topical_relation:bilateral_pair`.

Polarity: positive-only

Emitted by: subject_key_id

`consent:partnership_accept`

Description: Producer side of a bilateral grant.

Polarity: positive-only

Emitted by: attesting_key_id (producer)

`consent:scope:{kind}`

Description: **The scope of a grant is the `scope` member of the `consent:state:granted` envelope** — a bare string or an array of tokens naming what the grant covers. A `granted` naming no scope matches nothing, by design. Canonical kinds: `retain` (keep the bytes), `share` (propagate across federation), `analyze` (derive features / scores / classifications), `train` (use as training input), `publish` (publish to external systems). Open vocab with sub-scoping **in the token**: `retain:90d`, `share:cohort:family`. A `consent:scope:{kind}` dimension row, if anyone emits one, is **descriptive** — no fold reads it and it confers nothing; this row stays registered so the family has an owner (CC 3.1.7 R1). `analyze` governs *continued processing* per CC 2.3.5 **and is binding on emission for `capacity:*`**: the consent-before-scoring admission gate at CC 3.4.5 refuses a federation-tier `capacity:*` row whose subject holds no live `analyze` grant addressed to the scoring community.

Polarity: enumerated

Emitted by: subject_key_id

`consent:replication:{version}`

Description: **Directed node→peer replication grant** — a fabric node's standing, auditable consent to replicate a named attestation-prefix set to a specific federation **peer** named in `subject_key_ids[]`. The out-of-group peering consent (see CC 3.3.7). Standing (not bound to a single target Contribution); revoked via `withdraws`/`recants`. Federation-tier.

Polarity: signed

Emitted by: attesting_key_id (granting node)

`consent:community_trust`

Description: **A node owner's grant that the node's sealed reasoning traces may be captured** for the community-trust plane (lens-core's per-seal consent gate; the *capture* gate of the three-artifact trace-sharing chain — capture here, ship by `consent:replication:{version}`, score by `consent:state:granted` with `scope: analyze`). **Fold (normative):** rows are grouped by `attested_key_id` — the node whose traces are at issue. A `withdraws` / `recants` is a **boundary, not a deletion**: the latest admitted revocation's `asserted_at` is `R`, and every grant with `asserted_at` $\leq$ `R` is out of the fold, whichever row the revocation named — so revoking the newest grant never resurrects an older one. Among the grants after `R` (all of them, if there is no revocation), the latest `asserted_at` wins, ties on the smallest attestation id; no grant means no consent. A revocation is therefore a **hard stop** no configuration overrides, and capture resumes only on a **fresh** grant asserted after `R` from the owner-binding chain. **The grant MAY be authored by the node's owner:** attester = the owner — a live roster key of the owner's fedID (CC 3.3.6.3) at the grant's `asserted_at` — and attested = the node whose traces are at issue, which is what scopes the grant; `subject_key_ids[]` names all three parties: the node, the key that was the node's owner at `asserted_at`, and the agent key that acts on the node. The node-authored form stays valid, and the fold below groups both forms by `attested_key_id`. Each named party may withdraw (CC 2.4.1.1 rule 2); only the attester recants (CC 2.4.1.3), so under owner authorship the person holds recant and under node authorship only withdrawal — trace-sharing consent is the person's act, as CC 3.3.6 and `consent:replication` already have it. **Four readings the consent-plane model (pinned by the register) fixes:** a new owner's revocation door is this fold's and no other — its own grant or revocation as the later row, never the old owner's seat; the agent key's withdrawal is folded like the owner's, a boundary `R`, and what a boundary excludes is grants only, never the traces already captured while a grant was live (the record stands, CC 1.13.6); `consent:state:*` rows on one subject use the same boundary fold, the latest admitted revocation's `asserted_at` putting every earlier grant out; and a decay protocol that began under a grant runs to `complete` after a fresh grant is made — the fresh grant covers capture from its own `asserted_at` and resurrects nothing the decay removed. **Revocation authority (normative):** the node's owner MUST always be able to revoke. A grant the node emits for itself MUST list, in `subject_key_ids[]`, the key that was its owner **at the grant's own** `asserted_at` — `owner_of(node)` resolved at that instant over the owner-binding rows (CC 3.2), never the receiver's current view, so a later ownership transfer neither admits nor un-admits a grant already made and every peer reaches the same verdict once the binding rows have replicated (a grant whose binding rows have not yet arrived is held, as every owner-gated row is). The listed key's `withdraws` is then admitted on the CC 2.4.1.1 third-party path whatever ownership later becomes; a node-emitted grant lacking that member is not admissible on this leaf. A new owner does not inherit the old owner's revocation seat — it asks the node to re-grant, or emits its own grant, which the fold prefers as the later row. A grant or revocation from a key that was neither the node nor its owner at the row's `asserted_at` is not this node's consent and MUST NOT be folded as one.

Polarity: positive-only

Emitted by: the node's owner-binding chain — the owner, or the node it owns (self-or-owner, as CC 3.4.5 reads `config:{scope}`)

Composition pattern (the common case):

```
1. Producer publishes a Contribution with subject_key_ids = [user_key]
2. User (or a delegates_to chain rooted at user) emits a bare scores on
   consent:state:granted against the producer's Contribution, carrying
   scope: [retain, share, analyze] as a member of that row's envelope
3. Later: user issues withdraws against the producer's Contribution
   (admitted under CC 2.4.1.1 rule 2 -- subject revocation)
4. Substrate watcher (per CC 4.4.3.5) starts the deletion clock if producer
   committed consent:deletion_sla:{days} at publication
5. Producer emits consent:deletion_complete within the window OR
   substrate emits hard_case:deletion_window_breach as evidence (D2 below)
```

No new attestation_type.

Deletion-window breach — the network's own signal (normative)

`deletion_window` is a signed, JCS-byte-invariant temporal upper bound on the envelope (CC 2.1) — a producer's commitment, at publication, to delete on revocation within a stated time. It and `consent:deletion_sla:{days}` are one mechanism — the member carries the commitment, the row attests it — with one breach token (D2). These four rules make the network raise the signal it promises, with no new vocabulary and no new wire surface.

- **D1 — Proof of deletion is the absence, not a new artifact.** A producer has proven deletion **iff**, before the deadline, the row was **withdrawn, superseded, or hard-deleted** (CC 2.4.1.1). CC **declines** a distinct affirmative `deletion_proof` artifact: the three structural composers already leave an auditable, replay-checkable trace, whereas a fourth artifact would be a *claim about* deletion that a producer could emit while retaining the bytes — the weaker evidence, at the cost of new vocabulary. Where a producer wants an affirmative record it emits the existing `consent:deletion_complete` above; that emission is evidence **toward** D1 and never a substitute for it. Breach is therefore: the window passed **and** the row is still present with none of the three composers against it.
- **D2 — The breach signal is `hard_case:deletion_window_breach`** (CC 3.1.9.4), emitted by the observing substrate (`substrate_persist` ∈ `attesting_key.identity_type`, the CC 3.4.3 discipline) against the breaching producer, naming the breached row, the deadline and the observation time. A sweep feeding the classifier MUST be **replay-idempotent**: re-running it MUST NOT double-emit for the same breached row.
- **D3 — It is EVIDENCE, never a verdict (MUST NOT).** The signal MUST NOT be emitted under `slashing:*` and MUST NOT stand as sole evidence for a slashing outcome. The network raises a breach *signal*; it does not pass sentence. Consequences — slashing verdicts, de-admission decisions by other nodes — remain CC 4.3 WA-quorum or per-node acts, on the same evidence floor as CC 3.1.6 and CC 3.4.14 R5. The substrate observes and emits; the quorum adjudicates.
- **D4 — Local tier, self-scoped at mint.** The signal is minted **local-tier**, self-scoped to the observing node's own record, and is eligible for promotion under the ordinary consent-projection rules (CC 5.3.2.4). It travels the same consent-governed planes as every other observation; there is no privileged channel for a breach.

3.3.2 Governance subject_kinds

Two Contribution subject_kinds for governance over multimedia content: `takedown_notice` and `key_grant`. Both are **Contribution subject_kinds** riding the existing 1+4 wire format (CC 2.4) with `scores` as the attestation type; the `subject_kind` discriminator carries the wire-format slot. Since every row carries a registered dimension (CC 2.4 gate), a notice rides `takedown_notice:{basis}:v1` (CC 3.1.1) and a key wrap rides `key_grant:{axis}:v1` (CC 3.1.3). They serve non-maleficence (lawful removal of harmful content) and justice (auditable, gradient-disciplined access to restricted content) respectively.

`takedown_notice`

A signed wire artifact carrying a legal takedown request. The field shape is locked here.

```
takedown_notice {
  content_sha256: sha256_hex_lowercase // per CC 2.6.3
  content_holder_key_ids: [key_id,...] // peers known to hold the bytes
  claimant_key_id: key_id // the federation_keys row issuing the notice
  legal_basis: LegalBasis // closed-set enum per below
  jurisdiction: string // ISO 3166-1 alpha-2 + optional sub-division
  good_faith_statement: string // claimant's good-faith assertion text
  claim_text: string // the substantive claim being made
  evidence_refs: [URI or sha256,...] // backing material
  perceptual_hash: Option<PerceptualHash> // optional; PDQ / PhotoDNA / etc.
  counter_notice_channel: Option<URI> // where counter-notices may be filed
  asserted_at: rfc3339_canonical // per CC 2.6.2
  expires_at: Option<rfc3339_canonical> // optional auto-expiry
}
```

Where `LegalBasis` is the closed-set enum. **The PascalCase name is the spec-level variant name; the wire string is what is serialized** — same two-column discipline as `wrap_algorithm` below, and the same reason: a variant name is for reading, a wire string is for byte comparison.

`LegalBasis` (variant)	wire string (normative)	Source regime	Discipline
`Dmca512`	`dmca512`	US 17 USC §512	Expeditious-with-counter-notice (10-14 business day window)
`DsaArticle16`	`dsa_article16`	EU Digital Services Act Article 16	Expeditious-with-counter-notice (Article 17 redress)
`TvecTerrorist`	`tvec_terrorist`	EU Terrorist Content Regulation 2021/784	**Immediate** (1-hour removal obligation)
`NcmecCsam`	`ncmec_csam`	US 18 USC §2258A (NCMEC)	**Immediate** (substrate-protective; no counter-notice)
`GifctCip`	`gifct_cip`	GIFCT Content Incident Protocol	**Immediate** (within-hours coordinated response)
`CommunityStandards`	`community_standards`	Operator-defined community standards	Expeditious-with-counter-notice (operator-set window)
`PerceptualHashCsam`	`perceptual_hash_csam`	Hash-match against CSAM clearinghouse (PhotoDNA / Arachnid / etc.)	**Immediate** (substrate-protective)
`OsaIllegalContent`	`osa_illegal_content`	UK Online Safety Act illegal-content category	Expeditious-with-counter-notice (OSA-defined timelines)
`AvmsdAgeInappropriate`	`avmsd_age_inappropriate`	EU AVMSD age-inappropriate flagging	Compose with `age_assurance:*` gate; not immediate removal
`CourtOrder`	`court_order`	Court-ordered removal (any jurisdiction)	**Immediate** (subject to court's stated timeline)

LegalBasis (variant)	wire string (normative)	Source regime	Discipline
StewardPurge	steward_purge	Trust-root moderation of its own canonical (CC 2.4.1.1)	**Cosigned** under the root's consensus_protocol (CC 2.4 rule 0); content_sha256 is the **selection digest**: SHA-256 over the JCS array of sorted, unique "plane:id" strings naming every purged record, derived included — a record is named by its plane identifier (attestation_id, trace_id, bytes by their sha256), **never by content** — so a second implementation derives the same digest; content_holder_key_ids is []; an OPTIONAL selection_counts member {table: count} lets the door refuse a selection whose live count drifted from the authorised one (purge_selection_count_mismatch); evidence_refs → the dry-run ledger. The notice **is a Contribution** — admitted and replicated as every takedown_notice is, so it has an attestation_id the tombstones cite and a reconsideration:{grounds} can target (CC 4.5.5) — **and** it travels inside the tombstone bundle (notice, whole cosignature set, record ids; never content) so a receiver that has not yet folded it re-verifies against its own held root and re-derives the digest

Fast-path coordination: see CC 4.5.3 for the operator-coordination protocol around immediate-eviction cases (TVEC 1-hour / GIFCT CIP / NCMEC / PerceptualHashCsam / CourtOrder).

key_grant

Wrapped Data-Encryption-Key (DEK) delivery for restricted / subscription content. The field shape is locked here.

```
key_grant {
  wrap_algorithm: WrapAlgorithm // closed-set enum per below
  recipient_key_id: key_id // the federation_keys row receiving the DEK
  content_sha256: sha256_hex_lowercase // the content this DEK decrypts
  scope: GrantScope // closed-set enum per below
  wrapped_dek: base64url // the DEK encrypted under recipient's ENCRYPTION pubkeys.
    // For wrap_algorithm v2 (substrate-wraps, CC 5.2), the
    // recipient's {x25519, ml_kem_768} come from its current
    // identity_occurrence.encryption_pubkeys (CC 3.3.6.1) --
    // NOT its signing keys. A recipient with no registered
    // ML-KEM key is fail-secure excluded (CC 3.3.6.1 / CC 5.2).
  key_validity_window: {
    start: rfc3339_canonical // per CC 2.6.2
    end: Option<rfc3339_canonical>
  }
  ratchet_version: u32 // monotonic; counts the rotations for this
    // content_sha256 + recipient_key_id pair. A rotation is a NEW row
    // under a new axis identity (a blob re-sealed under a fresh DEK,
    // a new epoch); the prior row stays admitted. A re-emission under
    // the same identity is idempotent and never removes a wrap
    // ("Key wraps ride the base", below).
  rotation_chain: [key_grant_id,...] // informative trail: the ids of the
    // prior rows for the same content_sha256 + recipient_key_id pair,
    // oldest first. It names them; it retires nothing -- no row supersedes
```

```
    // or removes a wrap. The per-(stream_id, epoch) streaming epoch-key
    // axis (CC 5.1) carries the same trail on its own addressing axis.
  asserted_at: rfc3339_canonical
}
```

Where:

`wrap_algorithm` (variant)	wire string (normative)	Algorithm
`X25519AesGcmHkdfSha256`	`hpke_rfc9180_base_x25519_aes_gcm`	HPKE RFC 9180 base-mode shape; X25519 KEM + HKDF-SHA-256 KDF + AES-256-GCM AEAD. **v1**.
`X25519MlKem768Aes256GcmHkdfSha256`	`x25519_mlkem768_aes256_gcm_hkdf_sha256`	Hybrid X25519 + **ML-KEM-768** (FIPS 203) KEM + HKDF-SHA-256 + AES-256-GCM. **v2 — MANDATORY for streaming epoch-DEK grants** (CC 5.1). A hyphenated spelling of this name is not byte-equal to this string and is a non-conformant alias, not a match.

The `wrap_algorithm` *wire string* (serialized value) is normative for cross-impl decode — a producer, the substrate, and every consumer MUST serialize/deserialize the exact string above; a mismatch silently fails grant decode (same hazard class as the CC 5.3.3.1 STREAM-nonce `epoch` encoding). Both rows above are the frozen snake_case forms; a case-, separator- or ordering-variant of either (`x25519-mlkem768-aes256-gcm-hkdf-sha256` and its kin) is an **unknown** `wrap_algorithm` under the fail-secure decode rule below, never a normalizable alias — a consumer MUST NOT normalize case or separators before comparison, per the CC 5.1 one-construction-one-identifier rule.

Crypto-agility headroom. The vocab is deliberately version-roomy: a future `v3` (anticipated: **ML-KEM-1024**, given national directives treating ML-KEM-768 as interim with retirement horizons near 2030) is a pure **additive** row — new variant, new wire string, no change to existing grants or to the closed-set decode discipline. Consumers MUST reject an unknown `wrap_algorithm` string (fail-secure), which is exactly what makes the addition safe: old consumers refuse v3 grants rather than mis-decoding them.

`GrantScope` (variant)	wire string (normative)	Use
`SingleContent`	`single_content`	Grant decrypts exactly one `content_sha256`
`GroupMember`	`group_member`	Grant decrypts all content for which recipient is a member of named group (cohort-scoped)
`SubscriptionTier`	`subscription_tier`	Grant decrypts all content for which recipient holds named subscription tier

Variant name vs wire string — one answer for every CC 3.3.2 closed set (normative). Each closed set in this section (`LegalBasis`, `WrapAlgorithm`, `GrantScope`) has **two** spellings and they are not interchangeable. The **PascalCase variant name** is a spec-and-pseudocode label; it names the row and never appears on the wire, so a payload sketch written `scope: GroupMember` (e.g. CC 4.4.3.4.5) denotes the row, not the bytes. The **wire string** is lowercase-ASCII `snake_case`, is the serialized value, and satisfies the CC 5.1 identifier class rule: a producer, the substrate and every consumer MUST serialize and match **exactly these bytes**, and MUST

NOT normalize case, separators or ordering before comparison. Where the two readings could diverge, the wire-string column governs; where a future row is added, its wire string is derived the same way the rows above are (PascalCase word boundaries → _, lowercased, digits staying attached to the token they qualify) and is pinned in the table rather than recomputed by an implementation.

Why this is pinned rather than left to convention. The closed sets are decoded **fail-secure**: an unrecognized value is rejected, never guessed. That is the right default and it is also why a spelling ambiguity here is a safety defect rather than a cosmetic one — a peer that reads `NcmecCsam` where its counterpart wrote `ncmec_csam` does not degrade, it **drops the removal notice**, and the notices most likely to be dropped are exactly the immediate-eviction ones (`ncmec_csam`, `perceptual_hash_csam`, `tvec_terrorist`, `court_order`). One spelling, byte-compared, is the whole mitigation.

Key wraps ride the base: one `scores` row per recipient on `key_grant:{axis}:v1` (normative; registry row at CC 3.1.3). What replicates the wraps of a self, family, community or stream key between nodes is not a separate row type: it is one `scores` row **per recipient**, dimension `key_grant:{axis}:v1` with `{axis}` ∈ `content` | `epoch` | `stream` (closed), `attesting_key_id` the sealer — a key that speaks for the blob's or stream's owner on the `content` and `stream` axes, the minter on the `epoch` axis, who MUST be an active member occurrence at `asserted_at` — and `attested_key_id` the recipient occurrence. The envelope carries the key's identity — `(at_rest_sha256, cohort_scope, owner_key_id)` for `content`, `(community_key_id, minter_key_id, epoch)` for `epoch`, `(stream_id, epoch)` for `stream` — and the wrap, `wrap_algorithm` (v2 only) and `wrapped_dek` — the envelope members `axis`, `at_rest_sha256`, `owner_key_id`, `community_key_id`, `minter_key_id`, `epoch`, `stream_id`, `wrap_algorithm`, `wrapped_dek`, each present only on the axis that uses it. Three family rules apply, and two of them override a default that would otherwise be a trap. **Union by identity, and the precedence key is the row's, not the triple's:** the identity of a wrap is `(dimension, attesting_key_id, attested_key_id, axis identity)`, and the CC 2 latest-wins / dedup key `(attester, dimension, attested)` MUST NOT be applied to this family — one recipient holds wraps for many blobs and epochs from one sealer, and latest-wins over the bare triple would retire every earlier key. Rows with distinct axis identities coexist; a re-emitted wrap with the same identity is idempotent; a wrap once admitted is never removed by a later one. **The digest is a member, never a citation:** `at_rest_sha256` names the blob in the envelope and is NOT placed in `evidence_refs[]`, or the CC 2.3 bytes-plane fold would read a key wrap as a content binding (the same trap CC 3.1.3.3 rule 3 closes for holding claims). **Never withdrawn:** a `withdraws` or `recants` naming a `key_grant` row is refused; once shared, always shared at the wire layer, and forward secrecy is by rotation (CC 4.4.3.4.5, CC 5.1). **Audience by the one rule:** the row is placed at the content's scope and reaches the recipient's node because the row *refers to* it (CC 6.1.5.3) — the key follows the bytes to exactly the device it is for, which is the point, and a device merely named in a wrap is not thereby delivered anything else. A wrap is not a grant of authority and is never read as one: admission of the *content* it opens is governed by the cohort roster, and issuance authority by CC 2.4.1.2.1. There is no multi-wrap set: a `key_grant:{axis}:v1` row carries one wrap for one recipient, and there is no carrier class (CC 2.4).

Rotation after a recipient compromise: when a publisher rotates the keys a compromised recipient held, it re-seals under fresh keys and emits **new `key_grant:{axis}:v1` rows for the remaining recipients**, each under a new axis identity (a re-sealed blob's `at_rest_sha256`, a new epoch), with `rotation_chain` naming the prior rows for the pair as a trail. No row supersedes or removes the prior wrap; the compromised recipient is simply absent from the new rows. It is never a `withdraws` against the prior row: `withdraws` is the holders-directory eviction primitive

in CC 5.3.2.1, and a `withdraws` naming a `key_grant` row is refused above.

3.3.3 `location_proof` subject_kind

The wire-format primitive for a subject's rough-location declaration. Required for admission to `cohort_subkind: geographic` communities (CC 3.2); MAY be used independently as a standalone disclosure. Its design is an autonomy commitment: belonging to a place is opt-in, and the disclosure is bounded to rough by the wire format itself.

```
location_proof {
  subject_key_id: key_id // the asserting party's
    // federation_keys.key_id
  cell_id: string // H3 cell, lowercase hex per CC 2.6.6
  cell_resolution: u8 // MUST be <= 7 per CC 2.6.6.1
  asserted_at: rfc3339_canonical
  valid_until: Option<rfc3339_canonical> // null = indefinite (but consumer
    // policy SHOULD treat as stale
    // after 30 days for liveness)
  attestation_evidence: Option<base64> // optional hardware-attested
    // location claim from ciris-keyring
    // (TPM / Secure Enclave) -- null for
    // software-only / self-asserted
}
```

Substrate does NOT verify location truth. No GPS oracle exists at this layer; the substrate cannot independently confirm that a key in Austin actually emitted from Austin. The truth-grounding is consumer-side:

- The community's `consensus_protocol` admission decides whether to accept the claim (e.g., `majority` of existing Austin members vote to admit, presumably because they have out-of-band evidence the candidate really is in Austin)
- The `attestation_evidence` field MAY carry hardware-attested location data (e.g., a TPM-signed GNSS fix from a known-good device) for higher-assurance communities
- Repeat offenders (claim-Austin-then-emit-from-Tokyo) get caught by consumer-side detection (LensCore composition; not substrate-side gate)

Rough-only is wire-format-enforced. Per CC 2.6.6.1: `cell_resolution` $\leq$ 7. Producers attempting finer resolution have admission rejected; substrate emits `hard_case:location_proof_resolution_violation` (CC 3.4.2).

Typical cohort_scope: `federation` (the disclosure IS the opt-in; non-private by design). Producers MAY scope to `community` with a specific `community_id` if they want the proof readable only by that community's members — but then they re-emit for each community they want admission to. Operator/UI choice.

Lifecycle:

- `asserted_at` + optional `valid_until` per envelope
- `withdraws` against a `location_proof` evicts forward visibility (consumer policy treats the subject as "no current location proof" for community admission purposes from withdrawal-time forward)
- A withdrawn `location_proof` leaves a tombstone in the audit chain, never the cell (CC 4.5.9); per CC 2.4.1 `withdraws-isn't-retroactive`, leaving doesn't undo a disclosure already made

Composition with `consent_record`: a subject who wants to withdraw their location_proof AND compel deletion from substrate may emit a `consent_record` with `stance: revoked` + `scope: [retain, share]` against the location_proof Contribution. The substrate-side consent SLA watcher clocks producer compliance. Note: this is the consent-revocation surface, distinct from the structural withdraws-forward-only semantic above.

3.3.4 `family` subject_kind

A `family` is a **group of trusted nodes** — each node being a distinct identity (which itself may have multiple `identity_occurrence`s per CC 3.3.6). Families are the wire-format primitive for `cohort_scope: family` visibility scoping, and the integrity guarantee they encode is intimacy: content scoped to a family is admitted into substrate, wrapped under the family DEK, and delivered to all current members — but never emits a holding claim to non-members (CC 5.2), so the federation cannot even discover the content exists.

One identity MAY belong to multiple families. Each family has its own DEK and its own membership roster.

```
family {
  family_key_id: key_id // the family's own federation_keys identity
  family_name: string // human-readable; non-unique
  members: [
    {
      key_id: key_id // member identity_key (NOT occurrence_key)
      joined_at: rfc3339_canonical
      role: Option<MemberRole> // founder | member | null
    },...
  ]
  founded_at: rfc3339_canonical
  consensus_protocol: ConsensusProtocol // REQUIRED -- see below
  consensus_protocol_entrenched: bool // if true, consensus_protocol may not be
    // amended even via the protocol's own rules;
    // replacement requires out-of-band ceremony
    // (see CC 4.2 HUMANITY_ACCORD canonical instance)
}
```

MemberRole (open vocab; canonical kinds):

value	meaning
founder	Bootstrapping signer (recorded at founded_at)
member	Standard member; rights per consensus_protocol

`ConsensusProtocol` — **open vocabulary**. The family's chosen consensus mechanism for membership changes. Locked at family creation; changes ride the protocol's own rules (meta-amendment shape parallel to CC 4.5.1.2) UNLESS `consensus_protocol_entrenched == true`, in which case replacement requires an out-of-band ceremony.

Canonical `ConsensusProtocol` kinds:

kind	Semantic
`founder_only`	**A family's and a publication's protocol, not a community's** (CC 4.5.13). Founders are the sole admission authority — a founder **active at the instant of the act** (CC 3.1.3.2): a founder who has resigned holds no admission authority and cannot re-seat itself alone; new members proposed-and-admitted by any active founder. Suits private households / small trust circles.

kind	Semantic
`unanimous`	Every current member must sign the admission Contribution. For a **removal**, every member other than the target signs *and* those signers are a strict majority of the roster (CC 3.1.3.2, the two readings the model fixes): a two-member group therefore removes nobody by this door, and resignation is the exit. Suits very small high-trust groups.
`majority`	$> 50\%$ of current members must sign. Suits medium groups where blocking-minority concerns matter. The canonical entrenched form: the accord-holder family per CC 4.2 decides by a strict majority of its standing roster (CC 4.2.6) with `consensus_protocol_entrenched: true`.
`quorum:{m}/{n}`	Any `m` members must sign, `m` an absolute count (CC 4.4.3.4.2.1); `n` is documentary — the roster size at adoption, never recomputed against the live roster. The entrenched exemplar is the canonical `infrastructure` community of CC 3.2, whose founders sign under `quorum:2/3` with `consensus_protocol_entrenched: true`.
`weighted:{rubric}`	Sum of member weights (per a named operator rubric) must exceed a threshold. Suits formal organizations with weighted voting.
`custom:{family_specific_id}`	Operator-defined custom protocol (e.g., role-based, time-locked, multi-stage).

Membership-change ceremony (any addition or removal of a member):

1. **Addition** rides the widening plane of CC 3.1.3.2, never a supersede: an active member (under `founder_only`, an active founder) emits a `membership:proposal` to the joiner's key; the joiner answers with its own `membership:acceptance`; a `membership:widening` cosigned under the CURRENT family's `consensus_protocol` seats the joiner. A time-boxed guest is a widening with `valid_until`, and the seat lapses at that instant. A `family` Contribution that supersedes the current one may change metadata or drop members and never adds one. **Removal** is a `membership:removal` cosigned under the current protocol, or the member's own `membership:resignation`.
2. Substrate gates the widening or removal per the CURRENT family's `consensus_protocol`:
   - Counts signatures on the row (via the `consensus_protocol` rule)
   - If the rule is satisfied, admit and emit hard_case:family_membership_change:{family_key_id}
   - If not, hold the row in a pending state until additional member signatures arrive, for a **hold window the community MUST set** (operator policy; absent, the standing class of the governance table, 259 200 s, CC 4.5.13) — a held change expires at the window's end, so no sub-quorum removal stays held forever
3. On admission of an ADD: substrate emits retroactive `key_grant`s wrapping all `cohort_scope: family` content DEKs to the new member's `subject_key_ids`.
4. On admission of a REMOVE: per CC 4.4.3.4.5 the removed member retains existing key_grants (cannot retroactively un-share); the substrate stops wrapping new key_grants to them on subsequent family-scoped Contributions.

Consensus-protocol amendment:

```
A `family` Contribution that supersedes the current family Contribution AND
changes the `consensus_protocol` field is admitted ONLY IF:
  (a) consensus_protocol_entrenched == false, AND
  (b) the CURRENT protocol's rule is satisfied on the amendment Contribution

If consensus_protocol_entrenched == true, the substrate REJECTS the
amendment. Protocol replacement requires an out-of-band ceremony
(documented per family; for HUMANITY_ACCORD see CC 4.2.1).
```

Substrate emissions on family events:

- `hard_case:family_membership_change:{family_key_id}` — member added or removed
- `hard_case:family_consensus_protocol_change:{family_key_id}` — consensus_protocol amended (only when `consensus_protocol_entrenched == false`)
- `hard_case:family_consensus_protocol_violation:{family_key_id}` — proposed amendment rejected because rule not satisfied OR entrenched

All three are `hard_case` kinds emitted by the substrate under CC 3.4.4.

HUMANITY_ACCORD as canonical entrenched-`family`: the three accord-holder triple at CC 4.2 is structurally an instance of this primitive:

```
family {
  family_key_id: "humanity-accord",
  family_name: "Humanity Accord",
  members: [
    {key_id: "holder-1-key", role: "founder"},
    {key_id: "holder-2-key", role: "founder"},
    {key_id: "holder-3-key", role: "founder"}
  ],
  consensus_protocol: "majority", // a strict majority of the standing roster (CC 4.2.6)
  consensus_protocol_entrenched: true // replacement only via CC 4.2.1 ceremony
}
```

CC 4.2 remains load-bearing for the *role-recognition policy* + the `AccordCarrier` priority authority + the substrate-protective semantics. The structural shape of `HUMANITY_ACCORD` is a `family` subject_kind instance, generalizing the primitive across the federation.

Worked example — household with self-devices and family-devices:

```
User Alice has:
  identity_key = alice_root_key

Alice's self (identity_occurrence members):
  - alice_phone_key      (device_class: phone)  -+
  - alice_laptop_key     (device_class: laptop)  | Each is an identity_occurrence
  - alice_work_laptop    (device_class: laptop)  | of alice_root_key per CC 3.3.6;
  - alice_agent_key      (device_class: agent)   | content Alice keeps on her phone
  - alice_homeserver_key (device_class: server) -+ is cohort_scope: self and reaches
                                                   her other devices via the at-rest
                                                   encryption flow.

Alice's household (a family subject_kind instance):
  family_key_id: "acme-household"
  family_name: "Acme Household"
  members: [
    {key_id: alice_root_key, role: founder},  -+
    {key_id: bob_root_key, role: founder},     | Member entries are IDENTITY keys
    {key_id: living_room_tv, role: member},    | (NOT occurrence keys). Bob has his
    {key_id: kitchen_tablet, role: member},    | own self-collective; the TV and
    {key_id: hall_thermostat, role: member}   -+ kitchen tablet have their own
                                                  identity_keys (they don't belong
                                                  to any one person via identity_occurrence;
                                                  they're shared household nodes that the
                                                  family has admitted as members in their
                                                  own right).
  ]
  consensus_protocol: "founder_only" // either founder admits new members
  consensus_protocol_entrenched: false // founders can amend the protocol

When Alice's phone sends a family-scoped photo (e.g., dinner photo) at
cohort_scope: family + family_id: acme-household:
  - Substrate wraps the DEK under each member's identity key
  - Photo bytes reach Bob's devices, the TV, the kitchen tablet,
```

```
    the hall thermostat -- every admitted family node
  - No holding claim leaves the family (CC 5.2); non-family peers
    cannot even discover the content exists
  - Alice's own laptop also receives the photo via the at-rest encryption
    flow at cohort_scope: self -> identity_occurrence

When Bob's mom Carol visits and Bob wants to admit Carol's phone to view
family photos for a week:
  - Bob, an active founder, emits a membership:proposal to carol_phone_root
    (role: member, expires_at set)
  - Carol's phone answers with its own membership:acceptance naming that
    proposal -- no one is seated without the joiner's acceptance
  - a membership:widening seating {key_id: carol_phone_root, role: member,
    valid_until: +7d} is cosigned under consensus_protocol "founder_only":
    Bob's signature as an active founder satisfies the protocol
  - Substrate emits retroactive key_grants for cohort_scope: family content
    to Carol's phone
  - At valid_until the seat lapses; an earlier exit is a membership:removal
    under the protocol or Carol's own resignation. Either way Carol retains
    existing key_grants per CC 4.4.3.4.5; substrate stops wrapping new family
    content to her key
```

The example demonstrates the clean orthogonality: **`identity_occurrence` is for participants that ARE me** (across my devices and agents); **`family` is for trusted nodes that compose with me** (other people's identities, shared household devices, multi-party collectives). Phone = self device; the shared living-room TV = family device. The two primitives compose without overlap.

3.3.5 `consent_record` subject_kind

The canonical envelope shape when consent is the primary subject of the Contribution itself (parallel to `key_grant` and `takedown_notice` — a ceremony-shape over the underlying primitive). Use cases: standalone partnership grants, DSAR-shape consent declarations, multi-party contracts, explicit consent ceremonies with locked field schemas.

Both shapes admitted at the same gate: subject-side consent MAY ride a bare `scores` on `consent:state:*` against any target Contribution (the common case, see CC 3.3.1 composition pattern), OR ride this `consent_record` subject_kind when an explicit ceremony envelope is wanted. Per the CC 2.4 layering principle, bare `scores` is the primitive; `consent_record` is the ceremony UX shape over the primitive.

```
consent_record {
  subject_key_id: key_id // the subject declaring stance (federation_keys
    // OR canonical-hash per CC 2.3.2)
  target_key_id: key_id | null // optional: producer/recipient for bilateral grants
  stance: ConsentStance // closed-set enum per below
  scope: [ConsentScope,...] // open vocab; see CC 3.3.1
  asserted_at: rfc3339_canonical // per CC 2.6.2
  valid_until: Option<rfc3339> // null = indefinite
  deletion_sla_days: Option<u32> // for revocations: producer obligation window
    // (composes with `consent:deletion_sla:{days}`)
  decay_protocol: Option<string> // optional: named multi-stage decay path
    // (e.g., "ciris-agent-90day")
  bilateral_pair_id: Option<string> // for bilateral grants: pairs subject + producer
    // Contributions via topical_relation:bilateral_pair
}
```

ConsentStance (closed-set):

value	meaning
granted	Subject affirms; processing may proceed within scope and valid_until
revoked	Subject withdraws; producer must initiate deletion within sla window
expired	Substrate emission when valid_until passes without renewal

Admission rules. A `consent_record` Contribution is admitted iff:

1. **Required fields present**: `subject_key_id`, `stance` (closed-set), `asserted_at` (CC 2.6.2-canonical). All others are optional per the envelope above; absent optionals ride the CC 2.6.1.1 omit rule.
2. **`stance` is a closed-set value** (`granted` / `revoked` / `expired`); **`expired` is substrate-emitted only** — a producer/subject MUST NOT assert `expired` (it is the substrate's `valid_until`-passed emission).
3. **Tier eligibility** per CC 5.3.2.2: a `stance: revoked consent_record` is **NOT local-tier-eligible** (it carries subject revocation authority over another party's content) — it goes federation-tier (hybrid-signed) or rides the CC 5.3.2.2 24-hour `local` → `federation` promotion. A `stance: granted` *self*-consent where the subject holds sole authority MAY be local-tier per CC 5.3.2.4.1.
4. **Composition with the CC 2.4.1.1 `withdraws` gate**: a `stance: revoked consent_record` whose `subject_key_id` ∈ the target's `subject_key_ids[]` is admitted under CC 2.4.1.1 subject-revocation authority (rules 2–4), and the substrate SHOULD record which rule admitted it (the CC 2.4.1.1 per-rule audit metadata). No producer co-signature and **no quorum** is required — single-subject authority suffices (CC 4.4.3.5.4).

It rides the same admission gate as a bare `scores` on `consent:state:*`; the `consent_record` form simply carries the locked payload schema instead of a free dimension.

Bilateral pair pattern:

```
1. Subject emits consent_record(subject_key_id, stance: granted,
   bilateral_pair_id: <fresh-uuid>) +
   scores on consent:partnership_grant:v1
2. Producer emits consent_record(subject_key_id, target_key_id: subject_key_id,
   stance: granted, bilateral_pair_id: <same-uuid>) +
   scores on consent:partnership_accept:v1
3. topical_relation:bilateral_pair links the two Contributions
4. Consumer policy treats the partnership as ratified iff both halves present
   under the same bilateral_pair_id with stance: granted
```

The structural primitives close the bilateral shape — no new attestation_type, no new envelope field beyond `subject_key_ids` itself.

3.3.5.1 Optimistic fair exchange (worked example, no new primitive)

This worked example states the CC 1.7 in-grammar claim as **normative**: accountable ("optimistic") fair exchange — a digital good D traded for a settlement S, with adjudicated recourse — composes entirely from the existing 1+4 set plus the bilateral pair pattern above. The trustless, third-party-free atomic swap (HTLC-class) remains the CC 1.7 standing falsification target; this

pattern does **not** reach it. Offeror A and acceptor B exchange across a steward-bound escrow custodian E, with the always-present named-moderator / WA as the optimistic third party invoked only on dispute (WBD applied to exchange).

```
1. OFFER / ACCEPT -- bilateral ratification (the CC 3.3.5 bilateral pair above,
   CC 2.3.2.4 consumer-policy ratification):
   A emits consent_record(stance: granted, bilateral_pair_id: X)
     + scores on consent:partnership_grant:v1
   B emits consent_record(target_key_id: A, stance: granted, bilateral_pair_id: X)
     + scores on consent:partnership_accept:v1
   topical_relation:bilateral_pair links the halves; ratified iff both present.

2. DIGITAL LEG -- steward-bound escrow custodian (the CC 4.4.3.2 archive_custody
   precedent: a custodian holding per-epoch keys decoupled from the live roster):
   A authorizes E via delegates_to (CC 2.4.1.2) scoped to release-of-D.
   On confirmed S, E emits a key_grant (CC 3.3.2) to B -- the release.

3. VALUE LEG -- settlement (CC 3.3.10), off-stack on an external rail:
   a settlement attestation binds S to the ratified pair
     (settled_action_ref = the offer/accept Contribution).

4. DISPUTE -- optimistic adjudication: absent dispute, parties + E complete
   unilaterally (the optimistic path); on dispute the named-moderator
   (CC 4.5.4) / WA (CC 4.3) adjudicates and a hard_case:* event records it.

5. DEFECTION -- accountable recourse: non-delivery composes as a
   commitment_fulfillment (CC 3.1.9.2) shortfall; a proven_rogue
   slashing:{outcome} (CC 3.1.9.2) against staked standing (the CC 2.5 stake
   axis) follows on WA quorum.
```

What this buys, and what it does not. 1+4 buys **accountability, not atomicity**: a malicious or colluding escrow can still defect (after-the-fact redress is not prevention), and a revealed `key_grant` (CC 3.3.2) is forward-only — it cannot be un-revealed. The residual bridges are the value leg (CC 3.3.10) and physical delivery, neither fair-exchange-specific. No new attestation_type and no new envelope field — the CC 1.7 1+4 lockdown holds.

3.3.6 `identity_occurrence` subject_kind

The wire-format primitive that lets one logical identity speak across multiple **trusted participants** — devices (phone / laptop / server / embedded) AND agents (the user's own agents acting on the user's behalf). The `occurrence_id` envelope field (CC 2.1) names which occurrence emitted a Contribution; `identity_occurrence` is the **wire-format binding** that lets the substrate know `key_phone` and `key_laptop` and `key_my_agent` all represent the same identity `key_identity`. This is the integrity foundation under self-scope: it is what makes "this is me, on another device" a cryptographic fact rather than a guess.

Without this primitive: `cohort_scope: self` content cannot reach the user's other devices/agents — the substrate has no structural way to know which keys are co-self. With it: the at-rest encryption flow automatically wraps DEKs to all admitted occurrences when new content is admitted at `cohort_scope: self`.

```
identity_occurrence {
  identity_key_id: key_id // root identity (the user's logical identity)
  occurrence_key_id: key_id // this participant's signing key
  device_class: DeviceClass // closed-set enum per below
  hardware_attestation: Option<base64> // TPM / Secure Enclave / StrongBox / SGX
    // etc. attestation blob; null for software-only
  transport_destination: Option<TransportDestination> // Reticulum binding (below)
  encryption_pubkeys: Option<EncryptionPubkeys> // content-KEM keys (CC 3.3.6.1 below);
    // present => this occurrence is a v2 wrap target
  build_manifest_hash: Option<hex64> // the build this occurrence runs: the hash of the
    // pipeline-signed manifest (CC 3.1.2 provenance:build_manifest)
```

```
  build_manifest_ref: Option<uuid> // attestation_id of that provenance:build_manifest:{target} row
  asserted_at: rfc3339_canonical // per CC 2.6.2
  valid_until: Option<rfc3339> // null = indefinite
}
```

The build an occurrence runs (normative). An occurrence names the build it runs on its own row: `build_manifest_hash` (the hash of the pipeline-signed manifest, lowercase hex per CC 2.6.3) and `build_manifest_ref` (the `attestation_id` of the `provenance:build_manifest:{target}` row that manifest is published in, CC 3.1.2). The occurrence supplies them at registration from its CIRISVerify result. At admission the cited provenance row MUST exist and be Rooted on the receiving node and its manifest hash MUST equal `build_manifest_hash`. **A row about a running occurrence resolves its version through the roster:** a reader takes the build from the occurrence row live at the claim's `asserted_at`, never from a version string the claim's producer asserts. A row with both members absent names no build.

EncryptionPubkeys:

field	type	meaning
x25519_base64	[u8; 32]	classical KEM half — a FRESH content-KEM key (NOT the signing
		key, NOT the transport x25519 below — see key-separation)
ml_kem_768_base64	[u8; 1184]	PQC KEM half (FIPS 203, ML-KEM-768; exactly 1184 raw bytes — `ML_KEM_768_PUBKEY_LEN` — pre-base64)

TransportDestination:

field	type	meaning
reticulum_x25519_pubkey	[u8; 32]	transport identity's encryption key
reticulum_ed25519_pubkey	[u8; 32]	transport identity's signing key
destination_hash	[u8; 16]	RNS destination hash; MUST derive from the two pubkeys
		+ app_name + aspects per the CC 3.3.6.2.1 algorithm
app_name	string	RNS destination app (e.g. "ciris.federation")
aspects	[string]	RNS aspects (ordered; part of the hash preimage)

DeviceClass (closed-set):

value	scope
phone	Mobile device (iOS / Android / etc.); typically hardware-rooted
laptop	Personal computing device (macOS / Linux / Windows)
server	Always-on infrastructure node (home server, VPS, etc.)
embedded	IoT / hardware peripheral / signing dongle
agent	An AI agent acting on the identity's behalf
service	Background service / scheduled job / API integration acting
	on the identity's behalf

Admission is roster maintenance. An `identity_occurrence` row is signed by a live roster key of the identity (the genesis key or any admitted occurrence) and lands under the two-path

policy of CC 3.3.6.3: at once by a strict majority of the live roster, or after the window when no majority objects; at one key the addition is the owner's approval alone. Higher-assurance setups MAY layer requirements on `hardware_attestation` via consumer policy.

Revocation: an eviction is a `withdraws` against the `identity_occurrence` row, proposed by a live roster key and landing under CC 3.3.6.3. No single occurrence evicts at once with no window: that unilateral configuration is one a thief wins in two steps. Once it lands the substrate stops wrapping new key_grants to the evicted key; previously-delivered DEKs are out of scope per the CC 4.4.3.4.5 forward-secrecy rule (once shared, always shared at the wire layer; rotation is a separate ceremony).

Cardinality: an identity MAY admit unbounded occurrences; the substrate carries no hard cap (operator policy MAY impose per-deployment limits). When a new occurrence is admitted, substrate emits `hard_case:identity_occurrence_added:{identity_key_id}` (CC 3.4.3) so consumer policy can observe membership growth.

Composition with CIRISAgent CEG-native agent: an agent emitting self-attestations with `attesting_key_id == agent_self_key` AND `attesting_key_id` admitted as an `identity_occurrence` of the user's `identity_key_id` is structurally speaking AS that identity. The agent's local-tier self-attestations remain its own; federation-tier emissions reach the user's other occurrences via the at-rest encryption flow when `cohort_scope: self`.

3.3.6.1 the recipient content-encryption KEM binding

The CC 5.2 at-rest DEK cascade is **substrate-wraps-by-default**: the substrate generates the per-write DEK and wraps it to each active recipient. That wrap (`wrap_algorithm: v2`, CC 3.3.2) needs each recipient's **x25519 + ML-KEM-768 encryption** keys — but the federation directory's key registration carries only **signing** keys (Ed25519 + ML-DSA-65), and **ML-KEM cannot be derived from ML-DSA** (independent algorithms). So recipients must register encryption keys, and the substrate must resolve them by `key_id`. `encryption_pubkeys` is that binding.

The binding rides `identity_occurrence`, exactly parallel to `transport_destination`. It inherits the same four properties, already enforced, that an encryption-key binding requires:

1. **Self-certified** — admitted only when `attesting_key_id == identity_key_id` (or a current occurrence of it), so "these are identity K's encryption keys" is cryptographically proven, not trust-on-first-use. A spoofer cannot forge it.
2. **Hybrid-signed** (Ed25519 + ML-DSA-65) — the binding itself is PQC-signed.
3. **Rotatable via `supersedes`** — a new `identity_occurrence` superseding the prior rotates the KEM keys **without touching the stable signing `key_id`** that anchors every attestation/grant. A compromised ML-KEM key rotates for forward secrecy; the signing identity is untouched. (Bundling these onto the signing key registration would couple two independent rotation lifecycles — the reason this is NOT a field on the key record.)
4. **Already cross-region replicated** — `identity_occurrence` is its own kind in the locked replication wire, so the encryption pubkeys propagate inside the occurrence envelope that already replicates — **no new replication kind, no Edge wire change.** A cross-region recipient's keys resolve wherever its occurrence has propagated. (Encryption *pubkeys* are public → cleartext directory replication is correct, exactly as for signing keys.)

Key separation. *(normative; never reuse, and admission-enforced)* The `x25519` here is a **fresh content-KEM key**, distinct from BOTH (a) the occurrence's signing keys AND (b) the Reticulum transport x25519 in CC 3.3.6.2 `destination_hash = hash(x25519 ‖ ed25519)` (that is the *RET-link* transport key, classical-only — the federation seed never enters the transport layer, and the transport key never wraps content DEKs). Three key *purposes* — signing, RET-transport, content-KEM — are three distinct keypairs. Deriving the content-KEM x25519 from either of the others is a conformance violation (cross-protocol key reuse). **Admission check:** when an `identity_occurrence` carries BOTH `encryption_pubkeys` AND `transport_destination`, the substrate MUST **reject at admission** if `encryption_pubkeys.x25519_base64` decodes to the same 32 bytes as `transport_destination.reticulum_x25519_pubkey` — the one reuse case that is wire-checkable for free. (Reuse of the *signing* key as KEM key is not byte-comparable on the wire — different algorithms — and remains a producer-side conformance obligation.)

Forward-secrecy scope. KEM-key rotation via `supersedes` bounds **future** exposure only: grants wrapped *after* rotation use the new key. It does **nothing** for history — every `key_grant` previously wrapped to the compromised key persists at rest, and the at-rest threat model (CC 5.2 disk-forensics / host-operator adversary) is *precisely* an adversary holding those old grant bytes; with the old private key they decrypt every DEK ever wrapped to it, and `rotation_chain` supersession does not revoke bytes the adversary already holds. **Recovering historical content after a KEM-key compromise requires DEK rotation + content re-encryption under the new DEK — which CEG does NOT mandate.** This is a named gap (the CC 1.13.3 honesty discipline): operators with a compromised-key event MUST treat all content whose DEKs were wrapped to that key as exposed, and MAY re-encrypt; the spec provides the mechanism (new DEK + new grants + `supersedes`) but no automatic trigger. Do not represent KEM rotation as recovering the confidentiality of previously-wrapped content.

These feed `wrap_algorithm: v2` directly. `{x25519, ml_kem_768}` are precisely the recipient inputs to `x25519_mlkem768_aes256_gcm_hkdf_sha256` (CC 3.3.2). A consumer/substrate resolving a recipient's wrap target reads the **current (non-superseded, within-`valid_until`) `identity_occurrence` for that `key_id` → its `encryption_pubkeys`**.

Fail-secure conformance (the CC 5.2 tie-in). Because CC 5.2 *mandates* v2 for at-rest encryption, a recipient whose current occurrence carries **no valid ML-KEM-768 key is fail-secure *excluded* from the grant** — the content stays encrypted and unreachable to it; the substrate MUST NOT fall back to plaintext or to v1. To be an at-rest-encryption recipient, an identity MUST have a federation-present `identity_occurrence` carrying `encryption_pubkeys`. (This also resolves the "`family` member named only by `key_id` with no occurrence" case: no presence ⇒ no wrap target ⇒ excluded — correct, since there would be nowhere to deliver/store the wrapped DEK for a member that never established a presence.)

1+4 preserved — `encryption_pubkeys` is one optional field-set on the existing `identity_occurrence` subject_kind (CC 3.3 mechanism; no new subject_kind, no new replication kind). Fifteenth path (CC 1.7) — the wire format expresses **its own at-rest-encryption key layer** (recipient KEM-key resolution for substrate-wraps) by composition.

3.3.6.2 `transport_destination`: the authenticated identity↔address binding

In a **CEG/RET stack there is no DNS** — a node resolves a community member to a *reachable address* with no trusted nameserver. The `transport_destination` field is the wire-format primitive that makes that resolution **authenticated** instead of trust-on-first-use. Integrity at the addressing layer: you can prove who you are routing to.

The layer split it closes. A node's Reticulum destination is a *dedicated dual-key transport identity* `hash(x25519 || ed25519)` — deliberately **separate** from the federation signing key (the federation seed never enters the node's transport substrate). So "destination D belongs to federation key K" is a claim that must be *proven*, not assumed: a bare Reticulum announce only proves the announcer controls *that transport identity*, not that it legitimately belongs to K (any peer can announce `key_id=registry-steward-us` paired with an adversary destination → senders route to the adversary).

The binding = a federation-key-signed `identity_occurrence` carrying `transport_destination`. Because the occurrence is admitted only when `attesting_key_id == identity_key_id` (or a current occurrence of it) and is hybrid-signed (Ed25519 + ML-DSA-65), the binding `destination_hash` ← `identity` is cryptographically authenticated. A spoofer cannot forge an `identity_occurrence` signed by the real key. This makes the signed-announce attestation a **first-class, federation-wide, auditable CEG shape** — the announce app-data MAY carry it for self-authenticating discovery, and the directory holds it as the durable source of truth.

Conformance: a Consumer resolving a member's address MUST verify (1) the `identity_occurrence` signature against the member's federation key, (2) that `destination_hash` recomputes from `reticulum_x25519_pubkey`, `reticulum_ed25519_pubkey`, `app_name`, and `aspects` per the **CC 3.3.6.2.1** pinned algorithm (no free-floating hash), and (3) the occurrence is non-superseded + within `valid_until` at resolution time. An unauthenticated announce (no matching signed `transport_destination`) is **advisory-only** — usable as a routing hint, never as an authorization. Rotating the Reticulum destination (new transport identity) is a new `identity_occurrence` `supersedes`-ing the prior — location changes without touching federation identity or community membership.

The transit KEX binding (normative). A node that relays carries a **transit KEX public key pair** `transit_kex_pubkeys {x25519, ml_kem_768}` on this `transport_destination` binding, with the four properties of CC 3.3.6.1: attester = the identity, hybrid-signed, rotatable via `supersedes` without touching the signing `key_id`, replicated inside the occurrence envelope. **Key separation is normative and admission-enforced**: the transit pair is distinct from the signing keys, from the content KEM keys of 3.3.6.1, and from the Reticulum link x25519 above; a binding that reuses any of them is refused. **Scope of the admission check:** admission enforces the separation **in-row**, the transit pair against every key carried on the same signed row (the RNS link x25519, the transport ed25519, the signer's pinned key), exactly as 3.3.6.1 scopes its own check; separation from keys carried on a *different* row (the content KEM keys of 3.3.6.1 when they ride a separate occurrence row) is the **producer's duty**, because an admission verdict that depended on which row arrived first would differ between nodes holding the same rows (CC 3.2 T8 vii). **What a dialler may seal to.** An initiator opens a sealed link (CC 5.3.3.5 E1) only to a transit pair authenticated to the identity it means to reach: this binding, hybrid-signed by that identity and passing the in-row separation, from whatever source it was obtained; a Rooted binding; or the pairing pin of CC 3.2 adding a device (f). Rooted standing is not required to seal — it governs what the peer may be served (CC 3.2 T8; the bootstrap kinds, the allegiance facts and the pairing plane are all a not-yet-Rooted sealed link carries) — so two peers who hold each other's signed bindings seal and root each other with no canonical in the path. A bare or unsigned announce never supplies a pair; a dialler holding none fetches this row by the pre-seal route request of CC 5.3.3.5 E1, verifies it, and seals, falling back to the pairing pin only where no responder can be reached. The cross-row case is closed at use, not at admission: a consumer that resolves a relay's transit pair and finds it equal to any key on the identity's current occurrence treats the node as carrying **no valid transit pair** (it cannot relay), a verdict deterministic over the fold and never over arrival order, with no retraction row. This is the key that binds a **non-**

member relay in the CC 5.3.3.5 E1 hop-by-hop hybrid KEX: the relay's published static pair against the initiator's ephemeral hybrid key inside the link's first exchange (the static half binds the relay to its identity, the ephemeral half gives per-link freshness; pattern, label, info layout and rotation cadence are CIRISVerify's). A KeyPackage init key is refused as a hop key (it is a cohort credential, and a relay holds no cohort secret, E1 and CC 5.4.6); an ephemeral-only exchange is insufficient on its own (the relay would be authenticated only by the classical RNS link proof, which E1 forbids). A node whose current occurrence carries no valid transit pair **cannot relay** (fail-secure, as 3.3.6.1 excludes a recipient with no ML-KEM key), and a consumer resolves a peer's KEX pubkeys from this signed binding and never from a bare announce. 1+4 preserved: one optional field-set on the existing binding, no new subject_kind, no new replication kind.

The onion hop key (normative). A relaying node, and every serve node, carries a static **X-Wing onion hop public key** `onion_hop_pubkeys {ml_kem_768, x25519}` on this `transport_destination` binding, with the same four properties as the transit pair: attester = the identity, hybrid-signed, rotatable via `supersedes` without touching the signing `key_id`, replicated inside the occurrence envelope. It is the key an onion packet's layer for this hop is encapsulated to (CC 5.4.6). Key separation is normative and admission-enforced in-row exactly as for the transit pair: the hop key is distinct from the transit pair and from every other key on the same signed row, and a binding that reuses one is refused; separation from keys on a different row is the producer's duty, and a consumer that finds the hop key equal to any key on the identity's current occurrence treats the node as carrying no valid hop key. **A node with no valid hop key cannot relay onion traffic.** A node that has opted out of relaying carries none; a serve node with none does not conform to CC 5.4.6.

The opt-out marker (normative). A node that opts out of relaying for others (CC 5.4.6) carries `"onion_relay": { "opt_out": true }` in the signed envelope of this binding, hybrid-signed like the rest of the row, absent by default, and withdraws its hop key on the same superseding row: the marker takes it out of the layer deal, and the absent key makes a stale route through it impossible. A serve seat carrying the marker is out of conformance (`seat_opted_out`). The marker is explicit because an absent hop key alone is ambiguous: it could mean opted out, not yet capable, a key not yet minted, or a seat in violation, and the register, the client and the conformance witnesses tell those apart.

The hop key's shape (normative). An X-Wing public key is an ML-KEM-768 encapsulation key and an X25519 public key; the member carries both halves raw, in the standard base64 of CC 2.6.1.1.1, and JCS sorts its keys:

```
"onion_hop_pubkeys": {
  "ml_kem_768_base64": "<base64 of exactly 1184 bytes>",
  "x25519_base64":     "<base64 of exactly 32 bytes>"
}
```

It lives in the signed envelope only, never in a typed column, so the row's content hash does not depend on a store's projection. A half of the wrong length is refused as malformed, under the same rule as the transit pair; a half equal to either half of `transit_kex_pubkeys`, to the transport x25519 or ed25519, or to the signer's pinned keys is refused `onion_hop_x25519_reuse` or `onion_hop_ml_kem_768_reuse`.

3.3.6.2.1 RNS destination-hash algorithm (pinned)

This is a **two-stage** hash — it is **NOT** a single SHA-256 over a flat `x25519` ‖ `ed25519` ‖ `app_name` ‖ `aspects` preimage. The naive flat form yields a *different, wrong* value, so the algo-

rithm is pinned exactly below for independent recompute.

Pinned constants (SHA-256 throughout — RNS `full_hash`):

name	value	RNS origin
`NAME_HASH_LEN`	**10** bytes	`Identity.NAME_HASH_LENGTH` = 80 bits
`DEST_HASH_LEN`	**16** bytes	`Reticulum.TRUNCATED_HASHLENGTH` = 128 bits

Algorithm:

```
# 1. Expanded name -- UTF-8. app_name, then each aspect dot-joined, IN THE FIELD ORDER.
# The identity hexhash is NOT included (RNS computes name_hash with identity=None).
expanded_name = app_name
for aspect in aspects: # `aspects` in the field's given order
  reject if "." in aspect # dots are illegal inside an aspect
  expanded_name += "." + aspect

# 2. name_hash = first 10 bytes of SHA-256(expanded_name)
name_hash = SHA256(utf8(expanded_name))[:NAME_HASH_LEN] # 10 bytes

# 3. identity_hash = first 16 bytes of SHA-256(x25519_pub || ed25519_pub)
# Key order is reticulum_x25519_pubkey (32) THEN reticulum_ed25519_pubkey (32) --
# RNS get_public_key = pub_bytes (X25519) || sig_pub_bytes (Ed25519).
identity_hash = SHA256(reticulum_x25519_pubkey || reticulum_ed25519_pubkey)[:DEST_HASH_LEN] # 16 bytes

# 4. destination_hash = first 16 bytes of SHA-256(name_hash || identity_hash)
# addr_hash_material is the 26-byte concat (10 + 16); final hash truncates to 16.
destination_hash = SHA256(name_hash || identity_hash)[:DEST_HASH_LEN] # 16 bytes
```

Pinned source: the Reticulum reference implementation's destination-hash and identity routines (`full_hash` = SHA-256; `truncated_hash`; the public key = `pub_bytes` ‖ `sig_pub_bytes`; `TRUNCATED_HASHLENGTH = 128`). **CEG owns this reproduction**: it is the closed conformance source and does **not** float with upstream Reticulum — a future upstream hash change is a deliberate CEG version bump, never silent drift. A verifier that recomputes `destination_hash` per the four steps above and compares for byte-equality has performed the destination-authenticity check; a mismatch MUST be treated as an unauthenticated (advisory-only) announce.

1+4 preserved — `transport_destination` is one optional field on the existing `identity_occurrence` subject_kind (CC 3.3 mechanism; no new subject_kind). Twelfth path (CC 1.7) — the wire format expresses **its own addressing layer** (DNS-free, self-certifying member resolution) by composition.

3.3.6.3 The key roster of a self: one fedID, its keys, who may act and how the roster changes (normative)

One self, one string. A self is one fedID (CC 2.6.8): one person, one `key_id` string, the same on every device, and it is what owner-bindings, occurrences, contacts and a shared identity code name. The keys that may sign as that self are its **roster**: the `identity_occurrence` rows under `identity_key_id`. Each roster key lives on one node and a node holds at most one roster key of a given self; the node's own key is a different key (CC 3.4.7.3). The genesis key is roster key one and nothing more: the fedID's fingerprint is derived from it, and the string survives as the self's name if that key is later evicted, resolution then verifying the roster chain back to it. **On the wire:** the genesis key's eviction is an `identity_occurrence` revocation of the pair (fedID, fedID) signed by another live roster key, landing under the two paths below; the genesis key revoking that pair itself retires its transport self-occurrence and evicts nothing; an evicted genesis key is

not re-established by a later assertion of the pair, and the fedID string keeps naming the self. A revocation of the pair admitted before the two-path door existed does not evict.

Naming a roster key. A roster key other than the genesis key is named `<fedID>-<fingerprint>`: the whole fedID string as the CC 2.6.8 label, and the first ten lowercase base32 characters of SHA-256 over the key's *own* raw Ed25519 public key as the fingerprint. It is the 2.6.8 form exactly, verifiable against its own key, it carries no device name (the node key carries that), and it cannot be read as another person because the genesis fingerprint is inside it. Two spellings are refused: a device-derived id (`device-<hex>-<fp>`), and `<label>-<own fingerprint>` with the bare label, which reads as a second fedID. The 2.6.8 rule that a consumer verifies a `key_id`'s fingerprint against the public key it obtains holds for the genesis key for the fedID itself and for each roster key for its own id; a consumer MUST NOT treat a roster key id as a different identity.

Authority of a roster key. Any live roster key signs *as the fedID* for every owner act: consent, an owner-binding and its widening, root acceptance, the self room, contacts, and approving the next device. Nothing is reserved to the genesis key; a special genesis key would be a single point of loss, which contradicts the invariant below. A gate that asks whether the author is the owner or steward MUST accept a live roster key of the same fedID at the row's signed `asserted_at`, not only the key named on the binding — and that `asserted_at` is bounded by the past-freshness window W of the admitting substrate (`asserted_at_stale`), the same bound the consent fold puts on a backdated grant (CC 3.3.1), so an evicted key cannot sign an owner act into the window in which it was still live. **The bound applies only to a key that has lost standing:** W is checked when the author key is not a live roster key of the self *at receipt* — evicted, revoked, or never seated; a row signed by a key that is still live admits at any age, so a new device syncing a self's history admits its legitimate old rows, because the backdating threat exists only for a key that has lost standing.

Maintenance: the two paths. An addition or an eviction is a proposal signed by a live roster key; one open proposal per proposer. With **three or more** live keys it lands by either path: the **quorum path** — a strict majority of the live roster, proposer counted, approves and it lands at once; or the **lone path** — the window `W_self` = 86 400 s elapses and no strict majority of the *other* members objects, and for an eviction the target was silent all window (any signed act by a roster key answers every open proposal and proves the key is not lost). A vote from an evicted key is void. **Three readings the model fixes.** The window is measured by each node from the instant *it* first held the proposal, never from the proposer's signed `asserted_at` (the purge-dwell rule of CC 4.5.3; the model's age is per holder and it has no shared clock), so a landing is a node's own verdict in the way a root's standing is. Both thresholds are counted over the roster as it stands when the decision is made, not as it stood at the proposal's instant (a roster frozen at the proposal refuses its own second eviction). The door is a person's: it governs the roster of a user fedID; a node holds one key (CC 3.4.7.3) and an agent's occurrence is single-signer, and neither runs these paths. Votes travel on `objection:{state}` (CC 3.1.9.2: `objection:proposal:v1`, `objection:assent:v1`, `objection:raised:v1`; proposal id domain `ciris:objection:proposal:v1:`; an answer carries `cohort` = `self`, `cohort_key_id`, `objects_to_attestation_id` and `grounds`). The policy is two rows of the governance table (CC 4.5.13, `self_roster_add`, `self_roster_evict`): enact a strict majority, cancel a strict majority of the others, floor one key, window 86 400 s — the one roster where a floor of one voice is right, because every key is the same person's. The same policy as the constitutional halt (CC 4.2.1.1): one actor's act, a window, the standing set deciding whether it stands. At **one** key the addition is unilateral: the owner's approval of the device that is joining, which is the add-device flow. At **two** keys the lone path applies; a theft of one of two is then a stalemate (each side objects to the other's eviction and the thief can block an addition), which safety holds through and nothing

resolves on the wire. That bound is stated so a person knows it: a self SHOULD hold three keys. An eviction rotates no key material already shared (CC 4.4.3.4 Policy L; CC 4.5.12.1); it stops the next wrap.

What the self-roster model (pinned by the register) checks. The policy closes with three keys and one stolen, three keys and two lost, and four keys with one stolen and one lost; the model also shows the one-key addition landing at once, one key stolen as the loss of the self (`Inv_SelfSurvives` fails, the stated bound), two keys with one lost closing and two keys with one stolen stalling (safety holds, recovery does not). The stolen device ranges over every member, the genesis key included. The unilateral door fails in two steps; quorum alone cannot recover two lost keys; the lone path alone never evicts a thief who keeps using the device. Residuals named rather than closed: an owner absent for a whole window looks lost; one stolen and one lost of three is a tie; two stolen of four is a stalemate.

3.3.7 `consent:replication`: directed federation-peer replication consent

A **member of the CC 3.3.1 `consent:*` family, open in its parameters** (CC 4.5.1.1 discipline) — **not a new structural primitive** (the 1+4 surface is frozen; this rides the existing `scores` attestation_type and adds no envelope field). It names a consent shape the rest of the family does not: a fabric **node's** standing grant to replicate a class of its own attestations to a **named peer node**, as distinct from a **subject's** consent over a target Contribution.

The problem it solves. Cross-node propagation is governed by CC 4.4.3.2.1 / CC 5.2 `cohort_scope`. A node **inside** a community (e.g. the CC 3.2 / CC 8.1 `ciris-canonical` infrastructure community) shares with co-members by community-cohort membership — no per-peer object needed. A node **outside** that group has no membership edge to ride, so an out-of-group peering needs an **explicit, auditable, revocable** consent object. Concrete case: an in-group lens node (CIRISServer) replicating `capacity:*` to an out-of-group monitoring node (CIRISStatus), which replicates its `observation:reachability:v1` rows back. `consent:replication` is that object.

Shape. A bare `scores` Contribution on the dimension **`consent:replication:{version}`** (canonical `consent:replication:v1`) — `attesting_key_id = G` (the granting node), the recipient peer named in `subject_key_ids[] = [P]`. Standing (not bound to a target Contribution). Hybrid-signed; admitted **federation-tier** (it authorizes cross-node flow — not local-tier-eligible, CC 5.3.2.2 discipline). `cohort_scope: federation` (the grant itself is a public governance record).

```
scores {
  // -- envelope-level (CC 2.1 table -- frozen surface, untouched) --
  attesting_key_id: G, // the granting (sending) node
  dimension: "consent:replication:v1",
  score: <score>, // within the row's declared polarity (signed, CC 3.1.5); a withdraws/recants retracts,
     never a score value
  subject_key_ids: [P], // the single recipient peer authorized to receive
  cohort_scope: "federation",
  witness_relation: "self", // REQUIRED -- G attests its OWN replication intent
  valid_until: <optional rfc3339>, // optional -- time-boxed peering (CC 3.3.5 staleness)
  // -- payload-level (CC 2.3.2.3 -- subject_kind selects the schema; NOT envelope fields) --
  subject_kind: "consent_replication",
  grants: "replication", // constant
  attestation_prefixes: ["capacity:"], // CC 2.6.1 JCS array, sorted ascending + deduplicated
  cohorts: [...], // OPTIONAL -- on an owner's grant for one of their own nodes only: the node's cohort
     allow list (below)
  asserted_at: rfc3339_canonical,
}
```

Admission is by key registration; consent is the governance record (normative hon-

esty). The substrate gate that lets P's corpus *admit* G's replicated rows is **G's key existing in P's `federation_keys`** (registration), plus the CC 3.4 reserved-prefix identity rules. `consent:replication` does **not** add a substrate admission check — by design, exactly as CC 3.3.14 authorization is consumer-policy, not wire. **This posture is family-scoped, not the general rule.** It governs `consent:replication` and says nothing about other `consent:*` leaves: `capacity:*` carries a substrate-side consent **admission** gate on `consent:scope:analyze` per CC 3.4.5, which is the one place where a consent record is load-bearing at admission rather than only in governance. What it provides is the **auditable, revocable, bilateral record of intent**: the wire-format answer to "did G consent to send this to P, and for which prefixes?" Each direction is an independent unilateral grant (G→P and P→G are two separate `consent:replication` Contributions); a bilateral peering is ratified iff both are present.

Revocation (normative). A `withdraws`/`recants` (CC 2.4.1.1) from G against its own `consent:replication` grant retracts the consent. Because admission is key-rooted (above), revocation has teeth only if honored: on revoke, **the granting node MUST cease replicating the named prefixes to P and SHOULD deregister/expire P's directory authorization for them**, and **a consumer MUST treat rows replicated from G under a withdrawn grant as non-conformant** (the CC 4.5 location-proof precedent — the wire cannot un-send bytes a peer already holds; it can mark forward-only and oblige cessation). A grant carries optional `valid_until` (CC 3.3.5 semantics) for time-boxed peering.

Conformance shape. The grant is conformance-gradable as follows. Its **envelope-level** fields are exactly `attesting_key_id = G`, `dimension = "consent:replication:v1"`, `score` within the row's declared polarity (`signed`, the CC 3.1.5 catalogue row; magnitude is not load-bearing and a retraction is a `withdraws`/`recants`, never a score value), `subject_key_ids = [P]` (the **single** recipient peer), `cohort_scope = "federation"`, `witness_relation = "self"` (**REQUIRED** — a G→P grant is G attesting about its *own* replication intent; pinning `self` is what forecloses a third party forging a grant in G's name, since only G signs with G's key as the attested-intent-holder), and optional `valid_until` (the CC 2.1 envelope field, CC 3.3.5 staleness semantics) for time-boxed peering. The grant's parameters — `grants` (the constant `"replication"`) and `attestation_prefixes` — are **payload-level** members (CC 2.3.2.3) carried under `subject_kind: "consent_replication"`, **NOT** envelope fields: this is *exactly* why 1+4 is preserved — the CC 2.1 envelope table is untouched. `attestation_prefixes` is the CC 2.6.1 JCS-canonical array of CC 3.1 namespace-prefix strings G consents to replicate (trailing `:` significant — e.g. `"capacity:"`), **sorted ascending + deduplicated**, so two implementations holding the same grant agree byte-for-byte on `(G, P, prefix-set, validity)` and revocation-scope matching is deterministic. For the CC 6.1.5.2 corpus-replication track, `attestation_prefixes` additionally admits corpus `subject_kind` class names: naming a corpus class here **is** its pin authorization (the owner then *elects* to spend storage budget pinning it, per CC 6.1.5.2 B2).

Bilateral pairing + partial revocation (normative). `G → P` and `P → G` are independent unilateral grants; each SHOULD carry `topical_relation: bilateral_pair` (the CC 3.3.1 `consent:partnership_grant`/`consent:partnership_accept` precedent) so a consumer can pair them — a bilateral peering is ratified **iff both are present and live**. A `withdraws`/`recants` against a grant retracts it **whole**; a producer **narrowing** the prefix set (dropping `capacity:` while keeping another) MUST `supersedes` (CC 2.4.1.1) the grant with a new one carrying the narrower `attestation_prefixes` — it MUST NOT silently drop a prefix from a still-live grant (a silent narrowing is indistinguishable, to a consumer, from no change — and the cessation obligation below can only attach to an explicit retract/supersede).

The per-node cohort allow list rides this grant (normative). When the grant is an **owner's grant for one of their own nodes** (CC 6.1.5.3), it MAY carry the payload member

`cohorts`: the owner's group cohorts whose content that node is to receive. Each entry is an object `{scope, target}`: `scope` ∈ `family` | `community` | `affiliations` (closed), `target` the group's key id, byte-exact. The array is sorted by (`scope`, `target`) as UTF-8 bytes and deduplicated before signing (CC 2.6.1); an unsorted, duplicated, unknown-scope or empty-target entry is refused as malformed. **`self` is never listed**: a node's `self` content follows its class default and the two arms below. An **empty array** means *explicitly none* and is distinct from an absent member, which means *the class default*. There is no separate policy row: the allow list *is* this member, so editing it is a `supersedes` of the grant, narrowing included, under the no-silent-narrowing rule above. **When the member is absent, the node's class decides** — a **personal** device receives every cohort its owner is in; a **server** or infrastructure node receives **no** `self` or `family` content unless the list names it — and the class is the `device_class` of the node's `identity_occurrence` for that owner (CC 3.3.6), the owner's own signed statement about the device, never the grant and never the key record (whose `identity_type` says only that the key is a `node`): **personal** = `phone` | `laptop`; **server class** = `server` | `embedded` | `service` | `agent`. A node with no occurrence for the owner is not one of the owner's devices and receives no personal content. Either way the two arms of CC 6.1.5.3 hold regardless of the list: rows that originate on the node and rows that refer to it always reach it. A `cohorts` member on any other `consent:replication` grant (a node-to-peer grant) is refused as malformed. The member adds no envelope field and no attestation type.

1+4 preserved — `consent:replication:{version}` is a `consent:*` leaf, open in its parameters, on the existing `scores` type (CC 4.5.1.1); no new attestation_type, no new envelope field (the `grants` / `attestation_prefixes` parameters are payload-level per CC 2.3.2.3, above), no new replication kind (it replicates as an ordinary attestation). The frozen wire surface is untouched.

3.3.8 Event-lifecycle dimension families

Owning component: NodeCore — registry rows at CC 3.1.9 (content-ingestion families), per CC 3.1.7 R1.

Dimensions emitted against `external_content:event_listing` Contributions. Open vocabulary per CC 4.5.1.1 axis-vocabulary discipline; canonical states named here.

Prefix	Description	Polarity
`event:lifecycle:{state}`	State-transition signal for an `event_listing`. Canonical states: `open` (initial admission; RSVPs accepted), `cancelled` (organizer-issued cancellation; composes with `withdraws` against the event Contribution), `completed` (post-event finalization), `superseded` (composes with `supersedes` for reschedule). Lifecycle state is consumer-side composition over the structural primitives + this dimension's latest non-superseded emission.	enumerated
`event:rsvp_count`	Published RSVP tally (scalar). Distinct from the underlying `topical_relation:rsvps` edge set (CC 3.3.11) — `rsvp_count` is the publisher-asserted aggregate; the edge set is the auditable individual attestations. Consumer policy MAY reconcile divergence as a soft anomaly signal.	signed
`event:attendance`	Post-event attendance attestation, typically by event organizer `key_id`. Polarity carries organizer's confidence (e.g., turnstile-counted vs. honor-system).	signed

3.3.9 Operational-data subject_kinds: `organization` / `org_membership` / `partner_record`

An organisation is an affiliation. The three kinds below are retired by amendment (CC 4.5.1): their names stay in this section so citations resolve, no substrate admits them as record kinds, and everything they carried lives on the affiliation record. The Portal's `Organization` is a community at `cohort_scope: affiliations` (CC 4.4.3.2.8) and nothing about an organisation is a record kind of its own: `org_id` is the affiliation's `community_key_id`, `parent_org_id` is `hierarchy` nesting, `org_type` is `affiliation_archetype` + `membership_basis`, a member is a `membership:{stage}` seat (CC 3.1.3.2), a role is a `delegates_to` edge in the `hierarchy` whose `scope` set names what the member may do (`OrgAdmin` / `KeyManager` / `Operator` / `Viewer` are non-normative presets over scope sets, and admin scopes are open vocabulary, their canonical presets published as data with the family model manifest rather than listed here), a partner is `licensure:{community_key_id}:v1` with `partner_role:{role}` for the tier and `duty:{kind}` for obligations, and founding is self-founding under the founder's key — no administrator's door, no roster of permitted authorities; CIRIS L3C is one such affiliation and uses the same rows. A delegated holder never stands in for the `consensus_protocol` on a roster or config change (roster events stay cosigned under the protocol), and a keyless affiliation's key manager holds the registration binding under the quorum's cosignature.

An employee-steward is not punished for refusing (normative). An organisation that names an employee as the steward of its agent takes on, in its affiliation charter, the duty not to punish that steward for refusing an unlawful or unsafe instruction about the agent, and not to discipline a refusal that the steward records with its reason. The steward's name on the record is the organisation's accountability, not its shield.

Cross-region **operational Portal data** — organizations, memberships, licenses/partners — becomes signed CEG envelopes replicated by the same anti-entropy carrier as trust data. They serve justice and fidelity together: a partner's license and an operator's role are enforceable federation-wide, while everyone's private business detail stays home.

Governing principle (normative): federate the trust/authz-minimal projection; everything else stays region-local. The federated envelope carries only what the federation needs to enforce trust and authorization cross-region. PII and business detail live in the Registry's own per-region store, are NEVER emitted into an operational envelope, and never federate. **Projection minimization is the Registry's emit-side discipline — the Registry is the security-boundary steward**; the substrate's role is admission + merge, and it is explicitly NOT a PII filter (it stores what is signed and emitted).

All three ride existing `scores` + subject_kind discriminator. **Replication wire tokens (normative, snake_case): `organization` · `org_membership` · `partner_record`** — the replication kinds edge carries for them; v2 `envelope_hash` basis = `sha256(JCS(inner envelope))` per CC 2.6.1/CC 2.6.1.1.1. All three are **Commons tier** (CC 4.4.3.2.1) — plaintext at rest; the projection is world-readable by design.

```
organization {
  org_id: uuid // FIRST-CLASS: substrate MUST index as a row column (stable-id resolution below)
  name: string
  org_type: OrgType // internal | partner | licensee | community (closed set)
  parent_org_id: Option<uuid> // licensee under partner
  partner_id: Option<uuid> // link to partner_record
  status: active | suspended | deactivated
  asserted_at: rfc3339_canonical // per CC 2.6.2; LWW ordering field
  valid_until: Option<rfc3339_canonical>
```

```
}
// REGION-LOCAL (never federates): tax_id, billing/technical/compliance/primary emails,
// oauth_provider/oauth_domain, metadata, created_by.

org_membership {
  user_id: uuid // FIRST-CLASS (with org_id): substrate MUST index (user_id, org_id)
  org_id: uuid
  role: OrgAdmin | KeyManager | Operator | Viewer
  status: active | deactivated
  asserted_at: rfc3339_canonical
  valid_until: Option<rfc3339_canonical>
}
// REGION-LOCAL (never federates): the entire User PII record -- email, name,
// oauth_provider/oauth_subject, last_login_at, mfa_enabled/mfa_method, invited_by.
// Consequence: role-based authz works federation-wide; email->user login resolution
// is home-region-local.

partner_record {
  license_id: uuid // FIRST-CLASS: substrate MUST index as a row column
  partner_id: uuid
  org_id: uuid
  license_type: LicenseType // community | community_plus | professional_* | professional_full
  capabilities_granted: [string] // SET-SEMANTICS -> lexicographically sorted (CC 2.6.1.1.1 rule 1)
  capabilities_denied: [string] // SET-SEMANTICS -> sorted
  max_autonomy_tier: A0..A4
  requires_supervisor: bool
  geographic_restrictions: [string] // ISO country codes; SET-SEMANTICS -> sorted
  allowed_identity_templates: [string] // SET-SEMANTICS -> sorted
  deployment_limit: u32
  offline_grace_hours: u32
  status: active | suspended | revoked
  revision: u64 // MONOTONIC per license_id -- admission REJECTS any decrease
    // (the rollback discipline; the merge orders on this,
    // so a stale `active` can never overwrite a revoke)
  issued_at: rfc3339_canonical
  expires_at: rfc3339_canonical
  asserted_at: rfc3339_canonical
}
// No PII split -- the partner_record IS the world-verifiable grant; it federates whole.
```

An organisation names itself as an authority by its `community_key_id` (normative). When a licensing authority is an organisation, the `authority_id` segment of `licensure:{authority_id}` is the affiliation's `community_key_id` (the `org_id` of the record below, through which the organisation's keys resolve under its `consensus_protocol`, CC 2.4.1.2.1) — **not** a legal registration number (jurisdiction-scoped, mutable, PII-adjacent). This names the key set an emitter must resolve to for its row to be the organisation's licensure rather than testimony about it (CC 2.4.1.2.1); it is not a gate on who may be an authority — anyone may, under their own key.

Set-semantics declaration (normative). `capabilities_granted[]` / `capabilities_denied[]` / `geographic_restrictions[]` / `allowed_identity_templates[]` are **set-semantics → lexicographically sorted by JCS string form (CC 2.6.1.1.1 rule 1)** — and this is *more* than a single-signer determinism rule here: `partner_record` is signed by **M distinct stewards**, and the M-of-N quorum verifies only if all M sign **byte-identical** JCS canonical bytes. Unsorted capability arrays make two stewards who agree on the same grant produce different bytes — the quorum silently collapses and the license fails to admit cross-region. The vector set MUST include the M-of-N identical-bytes round-trip (M independent canonicalize+sign of one grant → one verifiable signature set). Any unordered list inside a constraints object carries the same declaration.

No payment-processor data. *(normative; fail-secure)* An operational envelope MUST NOT carry payment-processor-derived data (customer ids, subscription ids, charge refs, card metadata) — **including via any open-vocabulary field**. Substrate admission MUST reject an

operational envelope carrying recognizable payment-processor identifiers (defense-in-depth; the Registry's emit-side minimization is the primary control). Billing remains entirely off-wire (consistent with CC 3.3.10 keeping value-transfer rails off-wire).

Write authority — two shapes, two verifiers (normative).

- `organization` / `org_membership`: **single authorized signer, role-gated** — the envelope is admitted iff `attesting_key_id` holds the required role for the operation, established by a prior non-superseded `org_membership` (rooted at org creation by a steward/system authority). Verification reuses the **CC 4.4.3.4.3.1 `delegates_to` role-chain resolver** — explicitly NOT founder-quorum; implementers MUST NOT build a third bespoke path.
- `partner_record`: **M-of-N steward quorum** — the signature *set* over the identical JCS bytes is verified at admission by the **CC 3.2 founder-quorum machinery**. Professional capability grants are federation-wide; a single compromised key MUST NOT be able to forge one.
- **The two quorums are distinct:** *(normative; do not conflate)* (1) the **steward-signature admission quorum** above (signer authority, verified by Verify at admit) and (2) the **region merge quorum** (`quorum_weight`, the substrate `MergeBallot` tier-1 ordering during cross-region merge — CC 5.3.2.3). Different mechanisms, different layers, different stewards; the substrate's merge logic never counts steward signatures.

Mutability + current-state resolution. *(normative; stable-id grouping, NOT chain-walk)* Updates ride `supersedes`; deactivation/revocation rides `withdraws` (the CC 3.3.8 `event_listing` state-machine pattern). **Current state of a business id is resolved by stable-id grouping**: group all envelopes by the first-class business id (`org_id` / `(user_id, org_id)` / `license_id`) → apply `withdraws` forward-only → latest `asserted_at` → tie-break smallest `attestation_id` (the CC 3.5.1 discipline). For `partner_record`, admission anti-rollback on `revision` precedes the CC 5.3.2.3 quorum merge. Resolution MUST NOT require chain completeness — a region that never observed envelope N-1 still converges (partition tolerance is the point of CEG-native replication). `supersedes` references SHOULD be emitted when the prior is known and serve as **audit lineage only** — decoration, never resolution.

1+4 preserved — rides `scores` + subject_kind discriminator (CC 3.3 mechanism); license authority rides the existing CC 3.2 founder-quorum machinery; role authority rides the existing CC 4.4.3.4.3.1 delegation resolver; merge intents are substrate dispatch declarations (CC 5.3.2.3), not wire primitives. Sixteenth path (CC 1.7) — the wire format expresses **the federation's own operational/administrative layer** (the org/identity/license records that run the federation's business) as composition: after this, no cross-region byte moves outside a signed CEG envelope.

3.3.10 CEG↔value-transfer linkage

Value transfer itself is **not** a CEG primitive — it rides external rails (USDC on Base via x402, keyed to the federation signing key under the **Identity = Wallet** principle). The `settlement` primitive is the **optional, privacy-scoped attestation that *links* a federation action to its off-stack settlement** — so a paid relationship (paid stream / tip / subscription / paid `event_listing` / compute-job) can be federation-auditable *or* privately recorded, producer's choice. **CEG records that a settlement happened and binds it to what it paid for; the chain settles the value.** Clean separation of concerns, with autonomy as the default: privacy first, auditability opt-in.

Why this is in CEG's lane (and why it's optional): the linkage is a *trust fact* ("is this a real / authorized / paid relationship?"), exactly what consumer policy composes over. Agent-commerce and on-chain-privacy systems use the same mechanism — *a verifiable receipt exists, with selectively-disclosed contents* (x402 optional receipt header; append-only verifiable metering logs; "privacy + compliance via selective disclosure"). CEG already has every needed primitive, so this is composition, not invention.

Two admitted shapes (parallel to `consent_record` CC 3.3.5):

- **Primitive:** a bare `scores` on a `settlement:*` dimension against the paid Contribution (the common case).
- **Ceremony:** the `settlement` subject_kind envelope when an explicit receipt record is wanted.

```
settlement {
  settled_action_ref: contribution_id // the federation action this paid for
    // (or via subject_key_ids / topical_relation:settles)
  rail: string // open vocab; e.g. "base:usdc", "stellar:usdc", "x402"
  settlement_ref: string // chain tx hash / x402 receipt id -- cited the way
    // evidence_refs[] cite a SHA blob (CC 5.3.2): a settlement
    // is just another evidence reference
  amount_commitment: Option<string> // cleartext "12.50" (public case) OR a hash/range
    // commitment (private/selective-disclosure case)
  settled_at: rfc3339_canonical
  // visibility via the envelope cohort_scope: default `self` (payer+payee only);
  // `public` opt-in for transparency (e.g. creator revenue, DAO treasury flows)
}
```

Self-authenticating (Identity = Wallet): the same federation key that signs this attestation controls the Base wallet that emitted `settlement_ref`, so "I settled `tx` for action X" is self-proving — no oracle needed. The payee MAY counter-attest (`scores` on `settlement:received:*`) for a bilateral receipt.

Privacy is the default, auditability is opt-in. Visibility rides the existing gradient: `cohort_scope: self` (the parties only; the CC 5.2 structural-invisibility discipline applies — no holding-claim leak) by default; `cohort_scope: public` opt-in; amounts MAY be committed rather than cleartext; viewing-key / ZK selective disclosure composes later without a wire change. This matches "configurable-privacy-by-default" — the federation log is **not** a public payment trail unless the producer chooses it.

Lifecycle: `withdraws` against a `settlement` is forward-only (it does not un-happen the on-chain settlement — leaving doesn't un-pay, parallel to `location_proof`); a *disputed* or *refunded* settlement is a new `settlement` / `scores` referencing the original via `supersedes` or `topical_relation`. CEG never reverses value — it only records the subsequent state.

1+4 preserved — rides `scores` + subject_kind discriminator (CC 3.3 mechanism); `settlement_ref` rides the existing `evidence_refs[]` external-reference pattern; visibility rides existing `cohort_scope`. Fourteenth path (CC 1.7) — the wire format expresses **commerce-relationship auditability** (the receipt, not the rail) as composition, closing the last form of internet traffic from the completeness audit.

3.3.10.1 In-grammar ledgers: owner-serialized content, cohort-witnessed conservation

The CC 3.3.10 prohibition stands untouched: **CEG carries no conserved balance as substrate state, and value transfer rides external rails.** What this section admits is compatible with that stance: a **ledger as in-grammar *content*** — the total order lives in the ledger's own hash chain, signed by its single owner, never in the grammar; conservation is a deterministic fold the cohort verifies byte-equal, exactly as it verifies a governing tally (the CC 5.4.6 occurrence-invisible ballot machinery extended to value). The substrate never orders, reverses, or enforces an entry; it notarizes. What admits this at all is the CC 3.2 single-owner invariant: single-writer asset transfer needs no consensus. The real exclusions stand on their own legs — cross-cohort history-completeness against CC 5.2 structural invisibility, equivocation detection against the CC 5.4.5 rate-hiding witness, permanence against CC 6.1.2.1 descent — which is why this section stays **in-cohort, witness-anchored, checkpointed**, and a *global* conserved ledger remains inadmissible.

The nine clauses (normative).

1. **Binding.** One ledger per `(steward-bound identity, unit, standard_version)`; `ledger_id` derives deterministically from the triple. A second ledger claiming an occupied triple MUST be refused at admission (fail-secure). No parallel books within the claim's scope.
2. **Entries.** Dense `u64` sequence, prev-hash chain, owner-or-delegate signature. Kinds: `credit`, `debit`, `transfer` (carrying counterparty `ledger_id` + matching reference). An entry is a monotonic fact — forward-only, never reversed; a dispute or refund is a new entry referencing the old (CC 3.3.10 lifecycle: leaving doesn't un-pay).
3. **Delegate serialization.** `delegates_to` (CC 2.4.1.2) confers authority, not a mutex. A ledger MUST declare its serialization discipline — a write lease, or per-delegate sub-ledgers folded deterministically. Concurrent unleased writes at one sequence number ARE a fork, by definition.
4. **Head anchoring — the witness obligation.** Heads MUST be committed to the CC 5.4.5 per-community witness chain at a declared cadence; the cadence IS the equivocation-exposure window and MUST be stated, never implied zero. Witnessing is a protocol obligation, not a courtesy: voluntary out-of-band gossip is not an anchor.
5. **Checkpoints.** A co-witnessed balance snapshot at sequence N supersedes the detail below it (`supersedes`); history below the latest witnessed checkpoint MAY descend per the CC 6.1.2.1 aggregation pyramid; the head and latest checkpoint MUST stay above the noise floor. Conservation is provable from the latest checkpoint forward. No descent exemption is minted — the O(log T) retained-floor shape applied to money, and the regulator-endorsed retention shape besides (bounded detail, durable summary; the EDPB's own blockchain guidance recommends exactly summarize-and-delete).
6. **Promotion.** Promotion to cohort visibility MUST carry the compliance proof: chain validity from the latest witnessed checkpoint to the promoted head. Visibility rides `cohort_scope`; amounts MAY be committed rather than cleartext with selective disclosure later (CC 3.3.10 already provides this; the shipped precedent is the viewing-key pattern).
7. **The conservation fold.** Pure, deterministic, integer-only, byte-equal across members — the CC 6.1.6 contract discipline. Matched `transfer` pairs sum to zero; an unmatched or mismatched pair is FLAGGED, never merged away. The fold is a computation members verify, not a primitive the substrate enforces — determinism is what makes a violation **transferable evidence** any member checks without trusting the accuser (the PeerReview role of reference replay).

8. **Equivocation.** A proven fork — two owner-signed heads at one sequence number, or a head contradicting a witness-anchored ancestor — is evidence for the adjudication plane against the posted stake (CC 2.4.2) via `slashing:*` (CC 3.1.9.3). A fork NEVER fires `slashing:*` alone (the CC 3.3.12 `age_assurance` discipline, same shape): adjudication distinguishes equivocation from honest stale-state recovery. To make that distinction cheap, a writer MUST re-sync its head from the witness chain after any restore, before writing — the cohort's witness chain is the owner's recovery oracle. A punishment scheme that cannot make this distinction hurts honest participants restoring from stale backups; the distinction is available here because the witness set is cooperative, not adversarial.
9. **Non-claims.** NOT claimed: atomicity (the CC 1.7 trustless-atomic-swap falsification target is untouched — accepting payment is trusting punishment-backed accountability, the 1+4 bet); ledger privacy among members (accountable, not anonymous — the reverse-quorum discipline); cross-cohort conservation. At trust boundaries the rail settles (CC 3.3.10, unchanged); the sanctioned cross-boundary shape is **in-cohort netting with periodic net settlement on the rail**.

1+4 composition — no new attestation_type, no new envelope field. Rides `scores` on the `ledger:*` dimension family (`ledger:head:{unit}:v1`, `ledger:checkpoint:{unit}:v1`, `ledger:promotion:v1` — the version tail is the CC 3.1.7 R3 rule; registry rows at CC 3.1.3), the subject_kind ceremony where an explicit record is wanted (the `settlement` precedent above), chain blobs via `evidence_refs[]`, checkpoint supersession via `supersedes`, visibility via `cohort_scope`. The CC 1.7 lockdown holds.

Credit limits (normative). Where a ledger extends credit, the limit is governed by the community's `consensus_protocol`, informed by demonstrated capacity rather than collateral; the CC 4.5.4 named-moderator invariant is the hook for that governance, never a substitute for it.

One kernel, bounded siblings (informative). Nothing in this section claims a famous open problem: the roster vote is not the secret civic ballot, the cohort tab is not anonymous transferable cash, member-relay fan-out is not a CDN. Each is the *club-bounded* sibling of its famous problem — bounded exactly where the famous problem is hard (open membership, secrecy among members, atomicity, global scale), with each bound a stated non-claim. What the three share is the kernel doing its one job three times: an agreed roster with agreed deterministic state, invisible to outsiders — the CC 5.4.6 two-plane construction. The novelty budget is spent once, on the kernel.

3.3.11 Inter-content + relation prefixes

Owning component: NodeCore — registry rows at CC 3.1.9 (content-ingestion families), per CC 3.1.7 R1.

Prefix	Description	Polarity
`news:*`	News-content claims; publisher-attested + time-decaying + fact-checker composition.	signed
`encyclopedia:*`	Encyclopedia-content claims; editor-consensus + revision chain.	signed
`chat:*`	Chat-content claims (quality / participant-trust / context).	signed
`blog:*`	Blog-content claims (author-credibility / topic-domain).	signed

Prefix	Description	Polarity
`topical_relation:{kind}`	**Open vocabulary** inter-content relationship edges. Canonical kinds: `references`, `corrects`, `supersedes_article` (distinct from the structural primitive `supersedes`), `see_also`, `disambiguates`, `translation_of`, `replies_to`, `comments_on`, `cites_source`, `rsvps`, `vod_of`. New `{kind}` values are documentation-only registry entries (no CC 4.5.1 amendment needed).	enumerated

Composition note — threads, replies, comment trees: NodeCore's `chat_message` + `topical_relation:replies_to` compose into arbitrary thread graphs (microblog threads, forum comment trees, chat conversations, channels). No new structural primitive is needed; thread traversal is consumer-side composition over the existing edge set. Same shape for blog-post comment threads via `topical_relation:comments_on` + nested `replies_to`.

3.3.12 Multimedia dimension families

Owning component: NodeCore — `content_class` / `cw_class` / `content_rating` are catalogued at CC 3.1.9.2 and the five media-type families at CC 3.1.9 (content-ingestion families), per CC 3.1.7 R1; `age_assurance:*` is Verify's (CC 3.1.2).

All four families are **open vocabulary** per CC 4.5.1.1 axis-vocabulary discipline; canonical kinds named here, additions via documentation-only registry entries.

Prefix	Description	Polarity
`content_rating:{scheme}:{rating}`	Multi-scheme content rating. `{scheme}` ∈ `mpaa` (G/PG/PG-13/R/NC-17), `bbfc` (U/PG/12/15/18), `pegi` (3/7/12/16/18), `esrb` (E/E10+/T/M/AO), `ifco`, `csm` (Common Sense Media), or `operator:{operator_id}` for operator-defined rubrics. Polarity carries certifier confidence; not a slashing input.	signed
`content_class:{class}`	Mechanism-descriptive content classification. `{class}` open vocabulary; canonical: `film`, `short_film`, `documentary`, `art_piece`, `theatre`, `performance`, `news`, `educational`, `entertainment`, `vlog`, `adult`, `generated`, `generated_modified`. `generated` (machine-produced) and `generated_modified` (human-origin, materially altered by an AI system) are **mandatory** where they apply, per CC 3.4.14, and — unlike the rest of this table — are not multimedia-scoped: they apply to any Contribution carrying content, text included. Distinct from `cw_class:*` (community declarations) — `content_class` is producer-declared production-class; `cw_class` is community-applied content-warning.	signed
`cw_class:{class}`	Community CW (content-warning) declarations. `{class}` open vocabulary; canonical: `art_cinema`, `horror`, `political`, `erotic`, `violence`, `medical`, `nsfw_text`. Cohort-attestable per CC 4.4.1 Frickerian discipline (low-density cohort CWs not downweighted).	signed

Prefix	Description	Polarity
`age_assurance:{level}:{band}:{version}`	Witness-reserved age-assurance attestation (CC 3.1.2 row; rules at CC 3.4.11). `{level}` ∈ `provider` | `government` (closed; `self` is refused here); `{band}` ∈ `minor` | `adult` | `under_13` | `13_15` | `16_17` (closed — the CC 3.4.13 Q1 finer bands spelled as the substrate's `AgeBandFine`; `unknown` is a consumer's fail-secure *resolution*, never a wire token); canonical `{version}` = `v1`. **There is no `self` level on this prefix** — the self rung is the subject-signed `age_self_declared:band:{band}:{version}`, which carries a band only. A `provider` / `government` rung MAY further qualify `{level}` with a verifier key or credential class. Misdeclaration NEVER fires `slashing:*` alone — `moderation:age_assurance_misdeclaration` is the adjudication path.	signed

Media-type prefix families per `external_content` sub_kind:

Prefix	Description	Polarity
`image:*`	Image-content claims (per `external_content:image` sub_kind).	signed
`audio:*`	Audio-content claims (per `external_content:audio` sub_kind).	signed
`video:*`	Video-content claims (per `external_content:video` sub_kind).	signed
`film:*`	Film-content claims (per `external_content:film` sub_kind). Distinguished from `video:*` by distributor attestation chain.	signed
`model_3d:*`	3D-content claims (per `external_content:model_3d` sub_kind).	signed

3.3.13 external_content sub_kinds

Foundational sub_kinds:

sub_kind	Use
`encyclopedia_article`	Wikipedia-shape; editor-consensus + revision chain via `supersedes`; indefinite `valid_until`
`news_article`	Publisher-attested; time-decaying; corrections via `recants` + `topical_relation:corrects`
`accord_data`	Multi-sig signed (HumanityAccord / StewardTriple / WaQuorum / OneOfSix) per CC 4.2.1
`local_data`	User-private; always `cohort_scope: self`; promotable via CC 4.4.3.3.1
`chat_message`	Conversational message imported from a chat, messaging or microblog platform (or custom). Reply chains form via `topical_relation:replies_to:{target_message_entity_key_id}` (no new primitive). Default cohort_scope tighter than articles (`self` / `family` / `community` / `affiliations`). `valid_until` typically set; consumer policy SHOULD downweight chat in cross-cohort aggregation given privacy sensitivity. **This is the slot microblog content rides** — no separate microblog sub_kind needed.

sub_kind	Use
blog_post	Single-author commentary imported from single-author publishing platforms and personal blogs. Distinct from news_article (no publisher editorial), from encyclopedia_article (no peer-consensus), from chat_message (long-form). Comments on blog posts are separate Contributions (typically chat_message) citing the post via topical_relation: comments_on.

Multimedia sub_kinds:

sub_kind	Use
image	Photo, illustration, screenshot, infographic, meme. Source struct carries dimensions, format, AI-generation disclosure (EU AI Act Art. 50), mandatory alt_text accessibility metadata, license info.
audio	Music, podcast, lecture, audiobook, generated audio. Source struct carries codec, duration, sample rate, optional transcript, AI-generation disclosure, license info.
video	General video — vlog, social, screen recording, tutorial. Source struct carries codec, duration, resolution, mandatory captions reference, AI-generation disclosure, license info.
film	Cinematic / art-bearing video; distinguishable from video by content_class + distributor attestation chain. Same Source struct as video + festival / distribution metadata.
model_3d	Three-dimensional content — gltf, usdz, fbx, gaussian_splat, NeRF. Source struct carries vertex/triangle counts, bounding-box, mandatory description accessibility metadata, license info. format is the IANA model/* type where one is registered; FBX and splat containers have none (field-set block below).
live_stream	Real-time streaming surface. **Shape: HLS / CMAF over fragmented MP4 (RFC 8216)**, so chunks and files share one schema. The init segment (ftyp + moov) is the per-epoch header, one sha256-addressed blob every chunk references; each chunk is one fMP4 segment (moof + mdat), independently hashable, ordered by sequence number; codec is stream-level, once; EXT-X-KEY's "applies to every segment until the next key" is the (stream_id, epoch) model, and EXT-X-DISCONTINUITY is the epoch boundary that changes encoding. Composition property: **init** ‖ **chunk$_0$** ‖ **chunk$_1$** ‖ **... is itself a valid video/mp4**, so a recording is attested as an ordinary video blob with derived_from: stream_id. The stream manifest is an OCI-index-shaped list of chunk descriptors, itself sha256-addressed and cited from evidence_refs[] with no DAG codec; SHA-256 has no tree mode, so verified streaming is manifest-of-chunk-hashes, as HLS, OCI and UnixFS all do. Stream fields: format, codec, init_segment {digest, size}, epoch, key_id, sequence_start, target_chunk_duration_ms, mandatory captions ref (WebVTT segments per RFC 8216 section 3.5). Chunk fields: digest, size, seq, epoch, duration_ms, part_of.

Multimedia Source struct — the common field set and its vocabularies (normative). The rows above say what each Source struct *carries*; this block says what fills the slots, so a node's ingest and every client's renderer converge on one small set of types instead of each growing its own parser list. Media is the one place a stranger's bytes are parsed by native code in the receiver's process — the platform image and container decoders run in the receiving app — and the record of decoder vulnerabilities (zero-click image-decoder bugs exploited in the wild, recurring HEIF advisories, container-parser bugs) is why the type set a client renders is a **security boundary this Part owns**, not a UI preference.

- **format is the IANA media-type essence** (RFC 6838): lowercase, no parameters, validated against the RFC 6838 section 4.2 restricted-name grammar; in clear iff no sealed_descriptor is present (below). Every platform decoder API speaks exactly this string;

there is no second candidate. 3D: `model/gltf+json`, `model/gltf-binary`, `model/vnd.usdz+zip`, `model/vnd.usda` where IANA registers them; FBX and Gaussian-splat containers have **no** IANA type and are `application/octet-stream` with a `content_class` claim — one more reason they cannot be renderable.

- **`codec` is an RFC 6381-family identifier in its own field** — `avc1.PPCCLL`, `mp4a.40.2`, `av01.P.LLT.DD` (AV1-ISOBMFF), `hvc1.*`; Ogg codec names per RFC 7845 / RFC 5334 (`opus`, `vorbis`). The same string is what HLS `CODECS` carries (RFC 8216 section 4.3.4.2), so it flows unchanged into `live_stream`. **REQUIRED for `video/mp4` and `audio/mp4`**; the receiver's render tier depends on it.
- **Not multicodec, and no CID / multihash layer.** The multicodec table names hashes, keys, addresses and IPLD codecs and has no entry for jpeg, png, gif, webp, mp4, avc, aac, opus or mp3; it cannot fill a media slot. A CIDv1 over this Part's fixed SHA-256 (`01 55 12 20` ‖ `sha256`) adds only a constant tag and is derivable in one line at a content-addressed boundary; a CID over a DAG root is the hash of the root, not of the bytes, and would break the CC 5.3.2.5 invariant. `evidence_refs[]` stays bare sha256.
- **`size` (bytes) is REQUIRED — on every blob, not only media.** CC 5.3.2.5 rules it for anything `evidence_refs[]` cites; the multimedia struct inherits it. Checked before the digest: a length mismatch is a refusal without reading the whole blob, and the descriptor alone lets a receiver enforce caps and refuse a content fetch that overruns. A live stream in progress is the one descriptor without a total; its chunks and init segment each carry one.
- **The mandatory `captions` reference on `video` / `film` is the sha256 of a separate `text/vtt` blob** with its own size cap, never inline text.
- **The AI-generation disclosure uses the IPTC Digital Source Type vocabulary as its value space**: `trainedAlgorithmicMedia` → `content_class:generated`; `compositeWithTrainedAlgorithmicMedia` / `compositeSynthetic` → `content_class:generated_modified`; plus `algorithmicallyEnhanced`, `humanEdits`, `digitalCapture`, `screenCapture`, `virtualRecording`. C2PA ≥ 2.1 carries the same URIs in its actions assertion, so the disclosure interoperates with C2PA and IPTC photo metadata **without** adopting C2PA's binary manifest — which is embedded in the asset and therefore changes the digest (verify-only enrichment on the node at most, later). CC 3.4.14 governs the disclosure's mandatory status; this fixes its vocabulary.

The common field set:

```
digest          sha256:<hex>   over the bytes as stored and forwarded; == evidence_refs[]
size            int            REQUIRED; checked before hashing
format          string         IANA essence (above)
codec           string         RFC 6381 family; REQUIRED for video/mp4, audio/mp4
width, height, duration_ms     layout hints; receiver re-derives from the header, never trusts for
    allocation
placeholder     {thumbhash}    ~25 bytes, decoded by arithmetic -- the only element a UI may show before
    CC 5.3.2.5 has passed
name            string         display only; RFC 6266 section 4.3 sanitised; never selects a decoder or a
    save path
parent          id             optional; ONLY inside sealed_descriptor, never in clear: the stable id of
    the
                               collection:directory containing this file (CC 3.1.3.4)
content_digest  sha256:<hex>   optional: plaintext hash when digest is over ciphertext (community+ scopes,
     per-epoch HPKE)
derived_from    sha256:<hex>   optional: this blob is a rendition of that one
sealed_descriptor base64       optional: AEAD seal under the room DEK over {name, format, codec?, parent?}
     as a JCS object,
                               associated data = digest; when present, name / format / codec MUST be
    absent in clear
```

The sealed descriptor — in the encrypted case the description lives inside the seal (normative). A relay, a stolen disk or a mis-widened row yields a blob's **address, size and room — never its description**. The Source struct therefore carries `sealed_descriptor`: the base64 of an AEAD seal under the room's DEK over the descriptor members `name`, `format` and, where applicable, `codec` (a JCS object), with the blob's address `digest` as associated data so a descriptor cannot be moved between blobs. Three rules make it mechanical for every holder:

- **In clear, always required:** `size` (`u64 > 0`, checked by every holder before hashing — CC 5.3.2.5), the address `digest`, the room target, `content_digest` in the encrypted two-hash case, and `placeholder`.
- **`format` and `codec` are required in clear iff `sealed_descriptor` is ABSENT.** When it is present they MUST be absent in clear, and a Source struct carrying both is refused — one description, not two. `name` never appears in clear when a seal is present. A holder distinguishes *format sealed* from *format missing* by the presence of the member, without opening the bytes; the latter is refused.
- **The sniff check (CC 5.3.2.6) runs where the bytes decrypt**, by the party that opened them. A holder that cannot open the bytes cannot sniff and does not claim to; the substrate never holds a room DEK on the row plane and never renders.

What the struct MUST NOT carry: any `safe` / `renderable` / tier bit. Whether and how a blob renders is receiver policy computed from verified, sniffed bytes (CC 5.3.2.6); a descriptor claim is never an authorisation.

Renditions are separate blobs (normative). An intermediary that is also the hash authority MUST NOT re-encode: it would break `evidence_refs[]`. The **sender canonicalises before hashing** — which is also what strips EXIF / XMP / ID3 / ICC — the **node stores and forwards verbatim**, and any derived rendition (thumbnail, transcode, PDF or 3D poster) is **its own blob with its own digest** and `derived_from` naming the original.

Time-bound state-bearing sub_kinds:

sub_kind	Use
`event_listing`	Time-bound state-bearing content — event-ticketing and RSVP platforms, calendar invites. Source struct carries `platform`, `event_id`, `title`, `starts_at` / `ends_at`, `venue` (Physical / Virtual / Hybrid), `capacity`, `ticket_grant_policy` (Open / ApprovalRequired / InvitationOnly / Paid). **Lifecycle composes from existing structural primitives** — no new wire shape: RSVPs ride `scores` from attendee `key_id` on the event's `entity_key_id`; cancellation rides `withdraws` against the event Contribution; reschedule rides `supersedes` with `differs_in: ["start_time", "venue"]`; ticket transfer rides `delegates_to` against the ticket-grant Contribution. State-transition signal rides the `event:lifecycle:{state}` dimension family (CC 3.3.8).

Each Source struct conforms to a sub_kind-specific schema the owning component publishes; CEG documents the slot, NodeCore documents the per-sub_kind field shapes.

The AI-generation disclosure named in the multimedia rows above is mandatory, not descriptive — see CC 3.4.14 for the normative rule, its fail-secure default, and the assistive-operation carve-out that bounds it.

3.3.14 `identity:canonical_binding:` claiming a canonical-hash subject

A `subject_key_ids[]` entry MAY be a **canonical-hash** identifier — `sha256("discord:user_id:12345")`, an external-party id — rather than a `federation_keys` row (CC 2.3.2). When the real-world subject behind a canonical hash **later acquires a federation_keys identity**, it needs a wire-format way to **claim** that hash so its revocation authority (and proxy-delegation eligibility) attaches. That is the rebinding ceremony — autonomy reaching back to data that named you before you had a key.

Shape. A bare `scores` Contribution on the registered dimension **`identity:canonical_binding:{canonical_hash}`** — `attesting_key_id = K` (the claiming federation key), naming the canonical hash `H` as the bound subject. **Self-asserted** (`witness_relation: self`): K declares "I am the federation identity behind H." Hybrid-signed; admitted federation-tier (it grants authority — not local-tier-eligible, CC 5.3.2.2 discipline).

```
scores {
  attesting_key_id: K, // the claiming federation_keys identity
  dimension: "identity:canonical_binding:{H}", // H = the canonical hash being claimed
  score: <positive>,
  witness_relation: self,
  asserted_at: rfc3339_canonical,
}
```

A binding is a claim, never an authority (normative). An admitted `identity:canonical_binding` from `K` → `H` is a claim consumers weigh; it admits nothing by itself, and in particular it does not admit a `withdraws` from `K` against a target naming `H`. Withdrawal on an un-enrolled subject's behalf is the rule-3 proxy (CC 2.4.1.1 rule 3, CC 4.5.3), by which a node withdraws for a subject whose identity it has established out of band: the proxy attaches only after that out-of-band establishment, reaches only ordinary rows about that subject — never `detection:*`, `capacity:*`, `licensure:*`, `revocation:*` or `hard_case:*` — and is suspended while two bindings for the same person compete.

Authorization is consumer-policy, not wire (normative honesty). CEG pins the binding *shape*, NOT proof that K legitimately controls H's preimage. A binding is a *self-assertion*; a consumer weights it by whatever proof-of-control it trusts (OAuth/IdP verification of the `discord:user_id`, an out-of-band attestation, TOFU). The substrate admits the binding and records it; **the trust that K==H is composed by the consumer**, exactly as the CC 3.3.6.2 announce is advisory until rooted. A second key claiming the same `H` is admitted too (competing claims surface to consumer policy / RATCHET, not a substrate verdict). **1+4 preserved** — `identity:canonical_binding:{H}` is a registered `scores` dimension, not a new primitive.

3.4 Reserved-prefix enforcement

Most of the namespace is open-vocabulary — anyone may emit, and trust is composed downstream. A small number of prefixes are reserved: only specific identity types may emit them. This is integrity made structural — the few claims that would be catastrophic to forge (substrate health, capacity scores, the constitutional `accord:*`) are gated at the source. **Enforcement is normative at the substrate verify-pipeline AND at every CEG-Conforming Consumer per CC 2.2**.

3.4.1 The `accord:*` reservation

`accord:*` is reserved: only `federation_keys` rows with `identity_type="accord_holder"` may emit — see CC 4.2 HUMANITY_ACCORD. **Its leaves are closed** (CC 3.1.7 R3): the rows below are the whole family, each carried as a CC 3.1.1 catalogue row, and any other `accord:` leaf is `namespace_family_unregistered` — a reservation is not a namespace.

What is distinctive is the keying, not the reservation. This Part reserves many families — CC 3.1.1 alone carries `ownership:*` and `trust:*`, and CC 3.4.2–CC 3.4.12 reserve more; the generated registry is the count of record (CC 3.1.7 R1). So `accord:*` is not *the* reserved prefix. It is the one reserved to a **closed roster slot** rather than to a role with a conferral plane: every other reserved role — `lenscore_detector`, `witness`, `trusted_publisher`, the substrate families — is conferred by **delegation** (CC 3.2 T2, CC 4.4.3.8) and is therefore enterable under any root, while no delegation confers `accord_holder`: occupancy is fixed by the accord's own charter. That difference, and nothing wider, is why this prefix reads unlike every other reservation.

The roster is an instance parameter. Per the CC 1 preamble reading rule, this clause is read against the **slot**, never against its current occupant. CIRIS's accord holders are who occupies the slot in this mesh; another instantiation of the form names its own, and any node leaves any instantiation's reach — CIRIS's included — by deleting one acceptance row (CC 3.2 T3). What survives instantiation is the slot; the roster does not.

Reserved leaves:

Prefix	Polarity	Emitter rule
`accord:invoke:constitutional:{halt_id}`	+1.0 only	**One accord holder's signed row** — pre-signable, fired on receipt, no vote (CC 4.2.1.1); a majority of the standing roster lifts it by `accord:lifecycle:active` (CC 4.2.6)
`accord:invoke:notify:{notify_id}`	+1.0 only	One accord holder's signed row (CC 4.2.1.1); UI MUST distinguish from `constitutional` (CC 4.2.1.2)
`accord:invoke:drill:{drill_id}`	+1.0 only	One accord holder's signed row (CC 4.2.1.1) — a drill rehearses the fire path, so it needs no more signatures than the fire
`accord:lifecycle`	+1.0 only	**The accord heartbeat** — wire `accord:lifecycle:v1`, frozen and genesis-signed; accord-holder self-attestation about the accord; `valid_until` refreshes on a cadence $\leq$ 90 days
`accord:lifecycle:active`	+1.0 only	**Resumption** from a `constitutional` halt — the CC 4.2.1.3 invocation kind, its own signed domain; a majority of the standing roster lifts (CC 4.2.6)
`accord:lifecycle:confirmed`	+1.0 only	**Confirmation** of a one-holder `constitutional` halt within its fuse — a strict majority of the standing roster, one cosigned row naming the halt (CC 4.2.1.3); reserved as the others
`accord:human_dignity`	signed	The accord-holder verdict on an agent's human-dignity conduct, carried as `accord:human_dignity:v1` (`attested_key_id` = the agent scored); registered so the closed leaf set refuses nothing the substrate admits

`accord:lifecycle` (wire `accord:lifecycle:v1`) is the accord heartbeat, not a state machine — and `accord:lifecycle:active` is the resumption kind, a different row.

The heartbeat token is frozen; its prose name is the accord **heartbeat / drill witness**. Its refresh cadence is a **freshness-windowed signal**, banded per the CC 3.2 T4 liveness rule (0–90 green · 90–180 yellow · 180+ red) and reported on the trust surface. A lapsed refresh MUST NOT be ANDed into the validity of a root, a key, or a conferral, and a consumer MUST NOT read a stale `accord:lifecycle` heartbeat as a revocation: revocation rides the tombstone, `withdraws`/`recants`, supersession, or the halt latch, and nothing else.

3.4.2 Community + location-event reservations

Per CC 3.2 `community` + CC 3.3.3 `location_proof` subject_kinds. Four substrate-emitted prefixes:

These are `hard_case` **event kinds** — the `kind` column of a hard-case event, with the target key id carried in the event's payload — spelled here with the target appended for readability. They are not prefix families: the scored dimension is `hard_case:{kind}` (CC 3.1.9.4) and a substrate emits the kind, never a `hard_case:{kind}:{key_id}` dimension.

Hard-case kind	Emitted on	Emitter rule
`hard_case:community_membership_change:{community_key_id}`	Substrate admits an addition or removal in the named community's roster (per the community's `consensus_protocol`; for `cohort_subkind: geographic` admission additionally requires valid `location_proof`)	`attesting_key_id` MUST match `federation_keys` row with `identity_type="substrate_persist"`
`hard_case:community_consensus_protocol_change:{community_key_id}`	Substrate admits a `consensus_protocol` amendment on a non-entrenched community	Same: substrate_persist
`hard_case:community_consensus_protocol_violation:{community_key_id}`	Substrate REJECTS a proposed community amendment (rule unsatisfied OR entrenched)	Same: substrate_persist
`hard_case:location_proof_resolution_violation`	Substrate REJECTS a `location_proof` Contribution with `cell_resolution > 7` per CC 2.6.6.1 rough-only enforcement; emitted against the producer's `key_id` so operators can observe malformed-client patterns	Same: substrate_persist
`hard_case:lineage_equivocation:{lineage_key_id}`	Substrate holds two heads for one witnessed lineage (CC 3.2 T6), each meeting the charter's `witness_quorum`; head adoption freezes pending adjudication, both digests and both cosignature sets ride the payload. `event_id` derived from `(kind, lineage_key_id, the two digests sorted)`, so every node's emission is one idempotent event	Same: substrate_persist

Hard-case kind	Emitted on	Emitter rule
`hard_case:lineage_fork_attempt:{lineage_key_id}`	Substrate receives a head with a signature valid under the lineage's quorum at that head's instant that does not descend from the witnessed head (CC 3.2 T6): recorded once per `(lineage_key_id, fork-point digest)`, never cosigned, never freezing; further attempts off the same point are counted, not stored	Same: substrate_persist
`hard_case:lineage_directory_reseated:{lineage_key_id}`	The conferring body re-seated a witnessed lineage's witness directory by charter re-scrub on the ceremony plane because the previous directory could no longer reach its own quorum (CC 3.2 T6); the carrying head is witnessed by the new directory and attachable only through an out-of-band anchor	Same: substrate_persist
`hard_case:community_liveness_stalled:{community_key_id}`	The named community's active founder set falls to `M` or fewer under `quorum:M/N` (CC 3.2 T7): valid but non-admitting from this instant. Emitted by the substrate that admitted the row (or holds the `revoked_after` bound) whose crossing moved the count, `event_id` keyed on that row; peers re-derive and do not emit	Same: substrate_persist
`hard_case:community_liveness_restored:{community_key_id}`	The named community's active founder set returns to `M + 1` or more by the T7 recovery widening (CC 3.2 T7); same emitter rule as `_stalled`	Same: substrate_persist

Composes with CC 3.4.3 + CC 3.4.4 — part of the same substrate-self-report discipline. Non-substrate emissions on these prefixes are a category error and MUST be rejected.

3.4.3 Substrate-self-report reservations (`system:*`)

CC 3.1.3 CIRISPersist `system:*` and CC 3.1.4 CIRISEdge `system:*` are reserved to the substrate component itself. Emitter rule: the `attesting_key_id` MUST match a `federation_keys` row with `identity_type="substrate_persist"` or `identity_type="substrate_edge"` respectively, cross-attested by all stewards in the steward-triple. Non-substrate emissions on these prefixes are a category error and MUST be rejected.

3.4.4 Self/family membership-event reservations

Per CC 3.3.6 `identity_occurrence` + CC 3.3.4 `family` subject_kinds. The substrate-emitted membership-event prefixes:

These are `hard_case` **event kinds** — the `kind` column of a hard-case event, with the target key id carried in the event's payload — spelled here with the target appended for readability. They are not prefix families: the scored dimension is `hard_case:{kind}` (CC 3.1.9.4) and a substrate emits the kind, never a `hard_case:{kind}:{key_id}` dimension.

Hard-case kind	Emitted on	Emitter rule
`hard_case:identity_occurrence_added:{identity_key_id}`	Substrate admits a new `identity_occurrence` Contribution for `identity_key_id`	`attesting_key_id` MUST match a `federation_keys` row with `identity_type="substrate_persist"`
`hard_case:family_membership_change:{family_key_id}`	Substrate admits an addition or removal in the named family's roster (per the family's `consensus_protocol`)	Same: substrate_persist
`hard_case:family_consensus_protocol_change:{family_key_id}`	Substrate admits a `consensus_protocol` amendment on a non-entrenched family	Same: substrate_persist
`hard_case:family_consensus_protocol_violation:{family_key_id}`	Substrate REJECTS a proposed amendment (rule unsatisfied OR entrenched)	Same: substrate_persist
`hard_case:recipient_excluded:{scope_key_id}`	Substrate fail-secure-skips a recipient in the CC 5.2 at-rest grant cascade. Payload: excluded recipient `key_id`, `reason` ∈ `{expired_occurrence, invalid_kem_key, missing_encryption_pubkeys}`, skipped Contribution's envelope ref. **Cohort-scoped: emitted INTO the affected self/family scope; MUST NOT federate beyond it** — the excluded member can audit; the federation learns nothing (CC 5.2 invisibility preserved).	Same: substrate_persist

Removal-path emission shape. The membership-change prefix covers **both add and removal** — there is **no separate `hard_case:member_removed` kind** (it would split one event class across two prefixes for no gain). On the **removal** path the substrate emits the *same* `hard_case:family_membership_change:{family_key_id}` (and the CC 3.4.2 `community_membership_change` analog), with the payload distinguishing the direction and carrying what the forward-secrecy + audit consumers need:

```
hard_case:family_membership_change:{family_key_id} (payload)
  change_kind: "added" | "removed" // the direction (pins the removal signal)
  subject_key_id: key_id // the member added/removed
  cohort_key_id: key_id // the family (or community) key
  effective_at: rfc3339_canonical // the membership-change instant; on removal this is
    // the re-key epoch boundary (CC 4.4.3.4.5 / CC 4.4.3.2.2
    // Option-A) after which the removed member receives
    // no new wrapped content
```

So the removal-time substrate signal is `change_kind: "removed"` on the existing prefix — consumers and the forward-secrecy re-key key on `effective_at`. Same shape for `community_membership_change` (CC 3.4.2) and the `identity_occurrence` removal (a `withdraws` against the occurrence; the substrate emits `family_membership_change` / `community_membership_change` where the occurrence was a member).

Composes with CC 3.4.3 — these are part of the same substrate-self-report discipline. Non-substrate emissions on these prefixes are a category error and MUST be rejected.

3.4.5 Self-emission polarity: who may speak about whom

Some families are reserved not by *role* but by the attester's **relation to the subject**. The relation runs in both directions, and the direction is the rule: a verdict about you must not be yours; a record of your own reasoning, consent, ownership or configuration must not be anyone else's.

Prefix	Emitter rule	Why
`capacity:*` (CC 3.1.8.1)	**Anti-self** — `attesting_key_id` MUST NOT equal `attested_key_id` — **and consent-before-scoring**: at federation tier the subject MUST hold a live `consent:scope:analyze` grant (CC 3.3.1) addressed to the scoring community; a `capacity:*` row without one is refused **at admission, before persistence**. Family-scoped: the second condition binds `capacity:*` only — the role-gated abuse-response families (`detection:*`, `moderation:*`, `slashing:*`) are unchanged, because an abuser never consents to the response to their abuse.	The agent's own capacity score is never fed back into the agent's own context — anti-Goodhart per CIRISAgent CC 3.1.2; and a reputational verdict about a subject's capacity is *consensual reputation*, minted only over a subject who granted analysis.
`trace:*` / `trace_summary:*` (CC 3.1.5)	**Pro-self** — `attesting_key_id` MUST be a member of `subject_key_ids`. Third-party emission on the namespace is REQUIRED to be refused. A row failing this, or failing the exactly-one-of inline-`trace`/`manifest` shape and the `trace_manifest:v1` field constraints at CC 3.1.5, is refused with `federation_trace_dimension_invalid`.	A trace records its own producer's reasoning; a trace someone else minted about you is a claim, not a trace. The exact inverse of the `capacity:*` rule above.
`consent:*` (CC 3.1.5 / CC 3.3.1)	**Per-leaf emitter class**, as pinned in the CC 3.3.1 *Emitted by* column and resolved by the CC 3.4.7 **longest-pattern** rule: `consent:state:granted` / `consent:state:revoked`, `consent:scope:*`, `consent:stream:*`, `consent:partnership_grant` MUST be emitted by a `subject_key_id` of the target Contribution or a `delegates_to` chain rooted at one; `consent:deletion_sla:*`, `consent:deletion_complete`, `consent:partnership_accept`, `consent:replication:*` by the producer/granting key; `consent:decay:*` by **the party running the decay protocol** — for the CIRISAgent 90-day protocol its consent module, a self-report under CC 3.4.5.1, never the substrate (stated once at CC 3.3.1 and here); `consent:state:expired` alone by `substrate_persist`, the one substrate-emitted consent leaf. `consent:state:expired` is matched by its own three-segment pattern, which is strictly longer than `consent:state:` and therefore governs — this row and CC 3.3.1 state one rule, not two. A `consent:` leaf that CC 3.3.1 does not catalogue with an emitter class MUST be rejected.	Consent governs routing. An emitter class that is open by default is a transmission principle anyone may forge.

Prefix	Emitter rule	Why
`ownership:*` (CC 3. 1.1)	**Owner-only** — `attesting_key_id` MUST be the live owner of `attested_key_id` under the CC 3.2 purpose-filtered `owner_of`, which resolves to at most one key. The single exception is the CC 3.2 recovery path, which rides `withdraws` + `wa_adjudication_ref` and never a fresh `ownership:*` emission.	The owner's signature *is* the claim of ownership; a third party asserting your responsible party is a seizure by attestation.
`config:{scope}` (CC 3.1.9)	**Self-or-owner** — `attesting_key_id` MUST be `attested_key_id` or its live `owner_of`.	A node's running configuration is a self-report; a third-party assertion of what you are running is a rumour.
`trust:*` (CC 3.1.1)	**Edge-source-only, per job.** These ride `delegates_to` rows, not `scores`, and are enforced under the same discipline by the CC 3.4. 7 scope sentence. On a `delegates_to` row the **source** of the edge is `attesting_key_id` and its target is `attested_key_id`. `trust:charter:v1` MUST be self-referential — `attesting_key_id` = `attested_key_id` — so a charter is only ever a key's own declaration about itself. A root that is a **keyless family** (`RootKind: family`, e.g. `humanity-accord`) cannot sign; its charter is the row `delegates_to(member → family_id)` bearing `trust:charter:v1` whose scrub set is the family's own roster reaching its `consensus_ protocol` (CC 3.2 T2) — self-reference by roster, and the family is derived from the verified signers, never read from a field. `trust: confers:v1` (root → subject) MUST carry the conferring root as `attesting_key_id`; a **keyless family** root confers the same way it charters — the row `delegates_to(member → subject)` bearing `trust:confers:v1`, whose scrub set is the family's roster reaching its `consensus_protocol`, the conferring family derived from the verified signers (so a genesis grant signed by `A1` and scrubbed by the accord's quorum is the accord's grant, and one signed by `A1` alone is no grant). `trust:accepts:v1` (node → root) MUST carry the **accepting node** as `attesting_key_id`; a root MUST NOT emit an acceptance edge naming itself as `attested_ key_id`. A row failing its job's rule is rejected at admission, not scored down.	Without this rule nothing binds `trust:*`, and any key may mint a charter *for another key* or manufacture its own acceptance — which would defeat the CC 3.2 T3 one-row untrust lever, since the deletion the node makes would be recreated by the root. Rootness is a claim a key makes about itself and a node accepts about the root; neither half may be authored by the other party.

Every rule above is evaluated by the CC 3.4.7 enforcement discipline (substrate admission, consumer re-check, producer obligation) and by CC 3.4.7.1 set membership where a role is named.

Consent before scoring, and the read boundary (normative)

Status. Three rules bind a conformant implementation: the anti-self **emission** rejection above; the **consent-before-scoring admission gate** (family-scoped, community-addressed, federation-tier); and the **composition-context rule** below, which governs the read boundary. *"An agent may read the rows filed about it, never the composed outputs"* is not a substrate read rule, for four reasons, each independently sufficient. **(1) The categories are not disjoint.** CC requires

composed outputs to be *filed as rows*: CC 3.1.8.1 makes `capacity:composite` (`min` over the five factors) a normative `capacity:*` emission, and `moderation_track_record:{community_key_id}` is a named composition riding `scores` — a rule reading "rows yes, compositions no" has no referent, and `min` compounds it (the composite IS the weakest factor's score; its mere existence certifies all five factors live). **(2) The retained fields contain the withheld quantity.** `score` and `confidence` are REQUIRED envelope members (CC 2.3.2), so the redaction is a field-level projection inside a visible row — inexpressible in the row-granular CC 5.3.2.4.4 read model — and the `confidence_floor` predicate filters on the withheld field itself, so a bounded binary search recovers a per-attester weight exactly. **(3) Redaction and verifiability are exclusive.** The CC 2.6 JCS contract binds the as-received member set; a score-redacted row is unverifiable, and a subject that cannot verify what is filed about it is *structurally* less able to contest it. **(4) The gate has no enforceable operand.** `identity_type` is self-asserted, set-valued, and optional at the read surface (no caller → the broad tiers are readable anyway); and CC 3.2 makes a steward-binding MANDATORY for every scored agent, so a class-keyed rule does not withhold the band — it relocates it to the agent's owner, which CC 1.13.2 forecloses.

Consent gates scoring, never accountability. *(normative; a necessity relation, not a consent condition)* A subject that declines analysis cannot be scored; its `capacity:composite` is undefined and MUST NOT be emitted; and every gate that requires a capacity verdict therefore **fails closed** for that subject. A service MAY decline to provide a function whose delivery is **objectively indispensable** to a `capacity:*` score, to the party who withholds `consent:scope:analyze` — and MUST identify, per gated function, the score it cannot be delivered without. It MUST NOT condition admission to **unrelated** functions on the grant; bundling the grant across functions voids it. (The distinction is load-bearing, not stylistic: conditioning service on consent to unnecessary processing invalidates the consent — GDPR Art 7(4)/Recital 43, EDPB Op. 08/2024, CJEU C-252/21 *Meta* — while a necessity relation survives exactly where the score is genuinely indispensable and fails exactly where it is merely useful, which is the boundary this rule intends.)

The composition-context rule (normative). The anti-Goodhart property binds where the gradient attaches — the **inference loop**, not the read surface. A CEG-Conforming Consumer MUST NOT place into any automated inference loop that selects a party's next action: (a) any `scores` row whose `subject_key_ids` include that party, (b) any aggregate, `ConfidenceBand` or `ComposedVerdict` over such rows, or (c) any threshold such a row was judged against. The rule is **uniform**: it names no `identity_type` and no class of party — it binds an operator's automated tuning pipeline exactly as it binds an agent's context assembly. It therefore confers no privilege, passes the CC 4.5.1.1 swap test as drafted, and carries no declared-asymmetry register entry. It reaches the mechanism the axiom is about — directional pressure in a maintained feedback loop — and not the party.

Enforcement is by trace audit, never by refusal (normative). A conforming consumer MUST tag every context item with `(dimension, subject_key_ids)` provenance and MUST be able to demonstrate over the CC 3.1.5 `trace:complete:v1` record that no item violates the rule; an absent provenance tag is non-conformance, not a passing audit. **The constraint is a propagating label, not a point check**: a value *derived from* a self-subject `capacity:*` row inherits the constraint, and the trace audit MUST verify label propagation through derivation, not merely the absence of a direct read in the loop. **No substrate read gate may be claimed as the enforcement point:** a deployment MUST NOT claim conformance on the basis of a caller-versus-subject comparison at the read surface. No read rule redacts `capacity:*` from any party the CC 5.3.2.4.4 scope gate admits.

If a deployment redacts anyway (conformance conditions). A deployment MAY redact a

composed output as local policy, but MUST NOT call it a boundary unless all four hold: (i) **input closure** — no read served to the redacted class permits recomputation of the withheld value, via an explicit projection dropping `weight` and the `score/confidence` envelope members from every handle including any ungated log walk; (ii) **no predicate on a redacted field** — a filter whose operand is withheld MUST be refused, never silently applied; (iii) **the marginal-pinning check** — a linear program over the retained-field marginals, constrained by each retained dimension's declared polarity and range, MUST show the feasible band set has cardinality ≥ 2 (a corpus that pins is not redacted; a single-polarity dimension pins by its name alone); (iv) **verifiability preserved** — the subject retains a means to verify what is filed about it. These are **not jointly satisfiable on this wire**, which is why the rule above is drawn where it is. **Two leaks are conceded, not closed:** `contributor_count` / `age_of_head` are recomputable from any row list and an empty list pins `InsufficientWitnesses`; and `WellEstablished` reachability is visible as row existence. Neither is closable without withholding row existence, which is the contestability surface.

Disposition of the CC 2.3.2 verification families under this rule (per family). None of the fourteen verify-owned attestation families is consent-gated, each for a stated reason on the consensual-reputation / integrity-and-abuse-response line. **Self-reports** (`attestation:self_verify`, `hardware_custody:{platform}`) are pro-self statements about the emitter itself — there is no third-party data subject to consent. **Artifact-integrity verification** (`attestation:hardware_rooted`, `attestation:registry_consensus`, `attestation:license_validity`, `attestation:agent_integrity`, `provenance:slsa:{level}`, `provenance:build_manifest:*`, `provenance:skill_import:{source}`, `cert_validity:{authority}`) scores builds, manifests, licenses and certificates — not a subject's conduct or capacity; integrity checking is the trust precondition, and a forger never consents to verification. **Log infrastructure** (`transparency_log:inclusion`, `transparency_log:consistency`, `transparency_log:cosigned:{tree_size}`) proves properties of a public log, the cosigned form already role-gated to witnesses (CC 3.4.9). **`rollback_detected:{revision_field}`** is an adversarial detector (-1-only polarity), on the abuse-response side of the line by construction. Consent-before-scoring binds the family that judges *agents* — `capacity:*` — never the families that verify *artifacts*.

What remains true regardless. Whether a `capacity:*` score is *correct* is answered by recomputability and by the in-pipeline constraint diversity of CC 3.1.5.1–CC 3.1.5.3, not by the identity of the attester set; and an emitter-standing model ("who has earned the right to score") is rejected, because it would make the substrate rule on worthiness and the substrate never adjudicates.

3.4.5.1 How a self-report reaches a peer, and what a peer owes it (normative)

A self-report is admissible by the rules above. Three further questions decide whether it is ever *useful* to anyone else.

1. Scope is per-row; a family is not scope-pinned. Nothing in this document pins `config:*` — or any self-report family — to a `cohort_scope`. The CC 3.1.9 row constrains the **subject** (*"only ever about the emitting node"*); `cohort_scope` is a **different axis**, chosen per envelope by the emitter under the CC 1.13.3.4 smallest-scope default. The two must not be conflated. Consequently: a leaf whose content is exactly what structural invisibility protects — `admission` (admin key ids), `transport` (bootstrap peers, peer sideband) — MUST be emitted at `self`; a leaf that is a bare operational condition about the emitter, such as `load` (CC 4.2.1.4), carries no secret and MAY be emitted at the **smallest scope that reaches the peers which route to it** — never federation by reflex, and federation only for a node serving the commons. The dilemma

"either compliant and unreadable by peers, or federation-scoped and in breach" is therefore false: the `self` rule for **sensitive** config rows is not a family-wide invariant.

2. An emission grant is never an input to a receiver's trust set (normative). `infra:attest` — *"vouch as the delegator's infrastructure"* — confers authority to **emit**, and confers **no right to be believed**. A grant issued by *my* owner cannot bind *your* composer; a delegation that could add its holder to a stranger's trust set would be authority laundering, and the CC 3.4.7.3 rule that neither party may assert the other's relation applies here in the same shape. So the correct posture for an unpinned peer's self-report is **admissible but unweighted**: a consumer MAY weight it from a relationship it already holds — a peer it replicates from is one whose claims *about itself* it has already decided to accept at some weight — and that weighting is CC 4.4 consumer policy, never a number this document sets. **Anyone may claim anything; absent trust or consent, a claim reaches no one but its author.** The route a self-report actually travels is therefore the consent-gated attestation plane — it flows to a peer only where the producer's consent grant includes it and the recipient is entitled — not a new namespace. The missing "route" was never a family; it is consent plus trust, which the grammar already has.

3. A renewal supersedes; composition counts subjects, not rows (normative). A self-report that renews to stay current MUST carry `supersedes` (CC 2.4) against the row it replaces, so the live set is **one row per `(subject, scope, leaf)`**. A composer weighting self-attestations by live count MUST therefore count **distinct subjects**, never distinct rows. Without this, a node that renews correctly halves its own signal and a third renewal thirds it — continued pressure makes the claim quieter, which inverts the meaning of renewal. Superseding is forward-only (CC 4.5.9.3): it does not un-say the prior record, it ends its liveness.

What this does not fix. Whether a given event kind is carried on a replication plane at all is an implementation's plane set, not a rule of this document: a family that no plane carries is unreachable no matter how it is scoped, and that is a substrate gap to close where it lives.

3.4.6 Delivery-receipt reservation

Per CC 5.3.3.6 delivery receipts. One reserved prefix for subscriber-emitted delivery acknowledgements:

Prefix	Description	Emitter rule
`delivery_receipt:{stream_id}`	Subscriber's signed acknowledgement that they received chunk K under the named stream + epoch. Best-effort default; opt-in for accountable streams. Validated-not-adjudicated per the CC 1.7 fail-honest invariant — substrate / Verify authenticate origin + JOIN against published STH root per CC 5.3.3.6, but do not compose "delivered"/"owes N" verdicts (consumer policy).	`attesting_key_id` MUST be a current member of the community/stream the `{stream_id}` belongs to, per CC 4.4.3.2 Policy M membership resolution. NOT substrate-self-report (distinct from CC 3.4.3 / CC 3.4.4 / CC 3.4.2 substrate emissions).

Composition with CC 3.4.7.1 identity_type-as-set: a subscriber who is also a witness MAY emit delivery_receipt under the subscriber role; the role-set must contain a subscriber-eligible role (per the community's admission semantics from CC 4.4.3.2 Policy M).

3.4.7 The enforcement rule (normative)

A CEG-Conforming Substrate (CCS) MUST reject any incoming attestation whose `dimension` matches a reserved-prefix pattern below AND whose `attesting_key_id` does not satisfy the prefix's emitter rule. Rejection is at admission to `federation_attestations`; rejected rows are not stored.

Scope — the gate is on the `dimension`, not on the attestation type (normative). The rule binds **every** attestation carrying a `dimension`, not `scores` alone. `trust:*` (CC 3.1.1 / CC 3.4.5) rides `delegates_to` rows; a rule that reached only `scores` would leave the trust-root family — the family that decides which roots a node obeys — with no enforcement point at all, which is a strictly worse outcome than the unreserved families it was meant to be stricter than. Where a reserved family rides a structural composer, the composer's row is checked at the same admission point, by the same three actors, on the same evidence.

Matching is longest-pattern, and ties are impossible (normative). Dimensions are colon-delimited. Where more than one reserved-prefix pattern matches a `dimension`, the pattern with the **greater number of matched literal segments** governs, and only that pattern's emitter rule is applied — `consent:state:expired` is matched by its own three-segment pattern, never by the subject-side `consent:state:` rule. First-segment or shortest-match resolution MUST NOT be used: it is what made one dimension answerable to two emitter classes and put the CCS and CCC checks in this section in a state where they could not both agree. Two distinct patterns cannot match the same segment count on the same dimension, because a pattern is a literal segment sequence; a producer MUST NOT register a pattern that would create such a tie.

A CEG-Conforming Consumer (CCC) MUST independently re-check the reserved-prefix rule on every received attestation regardless of whether it was previously admitted by another peer's substrate. Trust does not propagate: the substrate's admission check is the FIRST line of defense; the consumer's re-check is the second. Both checks MUST agree.

A CEG-Conforming Producer (CCP) MUST NOT emit an attestation under a reserved prefix unless its `attesting_key_id` satisfies the emitter rule. Violation is a producer-side conformance violation regardless of whether any downstream substrate accepts the violation.

The gate reads `identity_type`, never a role carried in a registration envelope. Every admission decision above MUST be made against the `federation_keys` row's `identity_type` set (CC 3.4.7.1). Roles carried **inside** a scrub-signed registration envelope are the *input to* a conferral ceremony (CC 3.2 T2) — what the co-scrubbers signed off on — and are never themselves the gate. Prose that describes the reserved-prefix check as gating on "roles" states a security property the substrate does not check; read every such phrase as `identity_type` set membership.

3.4.7.1 `identity_type` is a set: single-key role cohabitation

Per CC 4.5.8. **`federation_keys.identity_type` is a SET of roles, not a single scalar role.** A single federation key MAY simultaneously hold multiple identity types — e.g., a folded CIRISAgent occurrence whose key is BOTH `agent` AND `lenscore_detector`, or a steward key that is both `substrate_persist` and `witness`.

One pair is carved out and MUST NOT cohabit: `node` with `agent`, and `node` with `user` — a key is substrate or actor, never both (CC 3.4.7.3, which also fixes the gate that this permission would otherwise silently repeal). Cohabitation is general; that exclusion is its only exception.

Every emitter rule in this section — and the CC 4.2.3 `accord_holder` material — is therefore evaluated by **set membership**, not scalar equality:

> *Wherever a CC 3.4 emitter rule reads "`attesting_key_id` MUST match a `federation_keys` row with `identity_type=X`" (or "`identity_type="X"`"), the normative reading is `X` ∈ `attesting_key.identity_type` — the key's role-set MUST CONTAIN `X`. A key satisfies a reserved-prefix gate iff the required role is one of its held roles.*

The scalar form (semantic-null). A scalar `identity_type="X"` is canonically the singleton set `{X}`. For a single-role key the membership test `X ∈ {X}` is identical to the scalar test `X == X`, so every single-role gating decision is unchanged. The field's meaning is set-valued (scalar → set) at the `federation_keys` row layer only. Substrate implementations MUST interpret the column as a set (its storage form is the implementer's, below); consumers MUST read a scalar as a one-element set. No envelope field, structural primitive, subject_kind, or CC 3.1 dimension prefix changes — the 1+4 wire-format lockdown is untouched; this is a CC 3.4-layer enforcement-rule generalization only.

Canonical-bytes encoding. Where `identity_type` enters a canonical-bytes computation (e.g., cross-attestation of a `federation_keys` row), the set MUST be encoded as its members sorted ascending by Unicode code point, deduplicated, comma-joined with no whitespace (e.g., `agent,lenscore_detector`). A single-role key encodes identically to its scalar form (`agent` ≡ `{agent}` ≡ `"agent"`), preserving signatures over single-role rows.

Storage representation is an implementation choice — "set" is semantic, not a column type. CC 3.4.7.1 requires that `identity_type` be *interpreted* as a set (membership test, not scalar equality) and that its *canonical bytes* be the sorted-deduped-comma-joined string above. It does **NOT** mandate a structured/array **column**. A single free-form `TEXT` column holding the comma-joined form, decoded-to-set on read, **is conformant** — the comma-joined string *is* the canonical representation, and the set is its interpretation. **No substrate migration to a structured column is required.** New `identity_type` values (e.g. `user`, `wise_authority`) are valid additive members of the open role vocabulary; they need no schema change, only inclusion in the membership-test set.

Cohabitation does NOT collapse the dimension split. Role cohabitation grants a key the *right* to emit under each held role's reserved prefixes; it does NOT merge the roles' namespaces. A key holding `{agent, lenscore_detector}` still emits its detector verdicts under `detection:*` (the lenscore-role surface) and its agent-intent attestations under the agent dimensions — the CC 3.4.8 shadowing rule and the CC 3.4.5 self-emission rejection apply unchanged per held role. See CC 3.4.8 for the LensCore-fold worked example.

Co-location is NOT consolidation — the fabric-node discipline. A *fabric node* (the headless cohabitation runtime that composes registry-authority + lens-observation + node-consensus over one substrate; `agent = fabric node + brain`) routinely holds the **full role-set in one key/process** — `{substrate_persist, steward, lenscore_detector, witness, ...}`. This is co-location of **custody**, not consolidation of **authority**, and the separation of powers is held **cryptographically, not procedurally**:

- **Authority stays quorum-bound.** A co-located `steward` role does NOT let a single node issue a federation-scope attestation. Registry-consensus is the CC 4.4.3.2.4.1(a) **founder-quorum** over the `ciris-canonical` infrastructure community (CC 3.2), evaluated over `{m: m.role == founder}` — human-held steward keys (CC 3.2, *Infrastructure does not vote*).

A co-located node gains **neither a vote nor a verdict**: it serves under the founders' standing, and the registry slice it runs is conferred capability (CC 4.4 `infra:attest` / `infra:serve`), not a seat.

- **Observation stays non-authoritative by namespace.** `lenscore_detector` emissions live under `detection:*` and are **validated, not adjudicated** (CC 1.7) — never sole evidence for an authority action (CC 3.4.8 / CC 1.2 T4).
- **Observation can never manufacture authority.** Holding both roles cannot make a `detection:*` emission an authority verdict: the namespaces do not merge (above), and authority is quorum-gated *upstream of any single key*. The hazard the LensCore mission warns of — *the lenses becoming the gate* — is structurally unreachable inside one process.

A fabric node co-locating authority + observation + consensus is conformant **iff** these hold. An implementation that lets a co-located node convert what it *observes* into what it can *authorize* has broken the separation at that point and is **non-conformant** — fix the wiring, never weaken the rule.

3.4.7.2 `consent_role`: the Counter-RII consent gate

`federation_keys.consent_role` is the role enum that gates **Counter-RII** probe detection. Three primitive-level semantics shape the substrate's `federation_keys.consent_role` schema and the transport's probe-pattern observer gate, and so **cannot be set per-consumer**; they are fixed here.

Revocation chain: `BaseRole`-only, non-recursive. A `consent_role` is non-recursive: a subsequent revocation **overwrites** the prior revocation record (NO recursive revocation chain embedded in the role). Chain history, **if retained**, MUST live in a separate audit surface and **MUST NOT be embedded in the `consent_role` JSONB**. This locks the substrate-portable JSONB shape — flat, bounded, overwrite-on-revoke — consistent with CC 3.3.8 stable-id grouping (not chain-walk; partition-tolerance) and CC 4.1.2 (no key pre-declaring its own state recursively).

Peer eligibility: blanket suppression. A node holding the `Peer consent_role` escapes Counter-RII detection at **any** `trust_mode`. The cost — a sovereign peer may probe other peers without raising the signal — is **bounded by construction**: `ratchet:flag:counter_rii:{layer}` is **advisory only** (CC 3.1.6) — it can NEVER be sole evidence for `slashing:*`; the WA quorum is the load-bearing adjudication gate. The exemption suppresses an *advisory signal*, not an *enforcement path*.

Post-window `AuthorizedReview`: strict. An `AuthorizedReview consent_role` is signal-eligible **immediately** at `t > window_end` — no grace period. Matches the fail-secure / clean-state-machine grain (CC 8.3.3); reviewers MUST respect their windows.

Implementations MAY build the `consent_role` substrate. **1+4 untouched** — `consent_role` is a `federation_keys` identity field (sibling to CC 3.4.7.1 `identity_type`), not an envelope primitive.

3.4.7.3 The actor/substrate separation: `node` is exclusive, and agency reaches a node only through a common human (normative)

This section is the canonical statement of "infrastructure must not have agency" — and of its missing converse. It restates the CC 3.2 stewardship bullet (*"`node` (infrastructure) — steward + `infra:*` reach only, **never agency**"*) with a section number.

Clause A — `node` is exclusive (normative). A `federation_keys` key whose `identity_type` set contains `node` MUST NOT also contain `agent` or `user`. **A key is substrate or actor, never both.** CC 3.4.7.1 permits role cohabitation generally; this is its one carved-out prohibition, and the carve-out is at the **axis**, not at the role: `{substrate_persist, steward, lenscore_detector, witness}` co-location remains conformant (custody, not authority — the fabric-node discipline), because those roles sit on the same side of the actor/substrate line. `{node, agent}` does not. The defect it prevents is **axis fusion** — one key answering both *which node is this* and *which agent is this*, so that `owner_of()` resolves a person to an actor where a node was asked for, and no trace can say whether the substrate or the actor acted.

Clause B — the agency gate reads membership, never purity (normative). The prohibition on a node receiving `agency:*` fires whenever `node` ∈ `recipient.identity_type` — not only when the recipient resolves to a node-*only* identity. This governs the CC 4.4.3.4.3 conformance rule: under a *"`node`-only (no `agent`/brain)"* qualifier, adding `agent` to a node's role-set would make the invariant stop firing, so **role-set union would repeal the property this rule exists to guarantee.** Clauses A and B are both required and neither substitutes for the other: **A** stops a fused key being *minted*; **B** is what protects the invariant against fused keys that **already exist**, since a non-conformant key is still a key in the data and still a possible delegation recipient.

Clause C — the actor↔node relation is entailed, never asserted (normative). There MUST NOT be a direct `agent` → `node` or `node` → `agent` edge. Neither party has standing to create one: a node has no agency to confer (infrastructure authorizing an actor), and an agent has no authority over infrastructure (an actor asserting a fact about substrate it does not own). The human has standing and has already exercised it **twice** — the owner-binding (`human` → `node`, `infra:*` only, single-valued) and the stewardship delegation (`human` → `agent`, `agency:*`, multi-parent). The pairing is therefore **derived from two independently-gated, human-signed edges**, which is precisely why it cannot drift from them and cannot be forged unilaterally: an agent cannot grant itself a node, and a node cannot adopt an agent. A third stored edge would hold a fact the graph already entails and would be free to disagree with it. **No new structural primitive** — the CC 1.7 closed set is untouched; this is a predicate over edges that already exist.

Clause D — the common-human predicate (normative). An agent MAY act through a node **iff** one human stands at both corners:

```
may_act_through(agent, node) :=
    exists h .  h = owner_of(node)  /\  h in stewards_of(agent)
```

The quantifier is **existential, and deliberately so**: node ownership is single-valued (CC 3.2), so the test is that the node's one owner appears *somewhere in* the agent's steward set rather than that the agent has exactly one steward.

`stewards_of` denotes the custody set, never the conferral set (normative). It is the CC 3.2 `is_steward_bound` anchor: the granter of the one live owner-binding `delegates_to` — the edge carrying the CC 2.4.1.2 owner-binding marker, per the CC 3.2 conferral-is-not-stewardship discriminator — and nothing else. An unmarked `delegates_to(U → agent)` confers a **job, not custody**, and MUST NOT satisfy this clause — reading `stewards_of` as *"every human who delegated agency"* would silently widen an authorization gate into a capability list.

Cardinality (normative). The steward set is therefore *small and structurally shaped*, not

unbounded: the `identity_occurrence` half yields **at most one** anchor, and the custody half yields **at most one**, because a second *distinct* custody claim on the same key is refused at bind time (single-owner admission; a same-owner refresh is idempotent). The multi-parent premise is carried by **the occurrence half plus one custody claim** — two anchors by two structurally different routes. A design assuming N co-equal stewards of one agent is assuming a shape the substrate will not admit. **Fail-closed:** an unresolvable walk — owner absent, ambiguous owner, revoked edge — MUST refuse, and MUST NOT resolve to "unknown". This is the person/node axis, whose signature failure mode is a **silent withhold**; without the fail-closed half, an unresolved walk reads as permission.

Clause E — the enforcement seam is the node's (normative wherever an authorization envelope is used). Where an envelope authorizes an actor's work on a node, the **node** MUST verify, before executing: the human's signature on the envelope; the referencing agent's standing; **Clause D**; and scope match — refusing otherwise, and refusing **by attestation** (`infra:attest` — *"vouch as the delegator's infrastructure"* — is already in the closed `infra:*` set, so a node may sign its own refusal without new authority). The full predicate at the seam is one human in three roles across three keys:

```
owner_of(node) == steward_of(agent) == signer_of(envelope)
```

Infrastructure is the right enforcer precisely because it has no agency of its own to exercise — a corollary of Clause A, not an exception to it. An actor that gates itself is the constrained party checking its own constraint; moving the refusal to a key that cannot benefit from the decision is what makes it structural rather than procedural. It is also why the actor needs its **own** key: an agent signing with the node's key makes the envelope reference self-referential and the check vacuous.

Clause F — fused keys are non-conformant, with no transition window (normative). A key holding `node` together with an actor role is **non-conformant**, and no window softens it. The CC 6.1.2.1 posture governs: no exempted producer, no window. Two precisions. **Forward-only** — the prohibition attaches to the *live configuration*, never retroactively to the record: attestations already signed by a fused key remain valid history, because an envelope cannot be amended, only superseded (CC 4.5.9.3 — the audit chain preserves the historical claim), and re-minting the key backwards would retroactively re-author every historical trace. **And still gated** — a fused key that exists MUST still be refused `agency:*` under Clause B; non-conformance is a reason to re-mint, never a reason the gate stops applying.

Open, and named rather than filled. Given the cardinality above, co-stewardship does not express the multi-tenant / community-server case directly. Clause D admits exactly two shapes: **(i)** one human holding both corners — the base case; and **(ii)** the node's owner standing as the agent's `identity_occurrence` anchor while a second human holds the custody marker — expressible, but it asserts that the agent *is an occurrence of that identity*, which is a strong claim and not general hosting. Everything else — an agent whose occurrence and custody anchors both exclude the node's owner — is **not admissible** here, and MUST NOT be reached by relaxing Clause D. A general hosting relation needs its own conferral from the node owner, on the node's `infra:*` plane where that owner has standing. Deferred, not denied: this is **most of the multi-tenant space**, not an exotic corner.

3.4.8 Detector-only prefixes

`detection:correlated_action:*` and `detection:distributive:access:*` are LensCore-only emission. Emitter rule: `lenscore_detector` $\in$ `attesting_key.identity_type`. Cross-attestation

by non-LensCore peers (on the same dimension, attesting to the same subject) is admitted as a score on the detector's verdict — useful when the federation wants to cross-check — but those scores MUST use a different `dimension` prefix (e.g., `truth_grounding:detection:correlated_action:{axis}`) to avoid shadowing the detector's own emission.

The discriminator is the prefix, not an envelope field. *(normative; this is what makes the substrate gate a blanket rule)* Admission needs **no** `is_primary_detection` flag and **no** envelope-shape parsing. The prefix contract *is* the whole discriminator: **any** row whose `dimension` is under `detection:*` **is** a primary detector emission and MUST satisfy `lenscore_detector` ∈ `attesting_key.identity_type` — the substrate **rejects** it otherwise; a **cross-attestation** MUST ride the `truth_grounding:detection:*` prefix, which carries **no** `lenscore_detector` requirement (it is a score *on* the detector's verdict, not a shadowing re-emission of it). The substrate admission gate is therefore **expressible as** a blanket reserved-prefix rule over the `detection:*` **prefix-wildcard** (so that novel `detection:{newkind}:*` subkinds are covered by construction, not only the two enumerated leaves) — the CC 3.4 reserved-prefix `identity_type` set-membership mechanism, with no per-row shape inspection.

LensCore-fold worked example. When `ciris-lens-core` is folded into the CIRISAgent workspace, a single folded occurrence's federation key holds `identity_type` ⊇ `{agent, lenscore_detector}`. The CC 3.4.8 gate is satisfied for that key's `detection:*` emissions by `lenscore_detector` ∈ `{agent, lenscore_detector}` — the cohabiting `agent` role neither grants nor blocks the detector right; only the held `lenscore_detector` role does. The split is preserved by the dimension namespace, not by the key: the same key emits its **agent-intent** attestations under the agent dimensions and its **detector verdicts** under `detection:*`; a downstream consumer cross-checking the detector still emits under the distinct `truth_grounding:detection:*` prefix above, shadowing-free. The CC 3.4.5 self-emission rejection continues to bind per held role — a folded `agent`+`lenscore_detector` key still MUST NOT emit a `capacity:*` score about itself.

3.4.9 Co-stewarded prefixes

`licensure:{authority_id}` is co-stewarded between CIRISRegistry CC 3.1.1 and CIRISVerify CC 3.1.2 — both MAY emit; consumers compose. Co-stewardship is of the **CIRIS-issued** licence — the authority the `partner_record` (CC 3.3.9) constitutes — and both stewards MAY record it. It is **not** a reservation of the family: `licensure:{authority_id}` is open-emitter (CC 2.4.1.2.1 — anyone may be an authority under their own key; the fold keys on emitter-resolves-to-authority, and admission is the ordinary trust-or-consent gate). A substrate that reserves the family to `registry` / `verify` identity types has misread this clause. **Single-source attestations** (only one of the two co-stewards has emitted) MUST be marked as `confidence` ≤ `0.5` in consumer composition until the second co-steward's attestation arrives.

3.4.10 Witness-emitter reservations

`transparency_log:cosigned:*` is reserved: emitter rule is `attesting_key_id` MUST match a `federation_keys` row with `identity_type="witness"` (CC 5.3.1).

"Witness" names six different things, and none confers another (normative). A reader MUST NOT infer one from another, and a substrate MUST NOT accept one where another is required:

Sense	Where it lives	What makes a key one
`witness_relation`	envelope member (CC 2.1)	Nothing — it is the signer's statement of its relation to the fact (`self` | `external` | `derived`), and confers no role
The `witness identity_type`	key record	Conferral by delegation under a root; it is the emitter role for `transparency_log:cosigned:*`, the log-checkpoint cosignature, and for nothing else in this section
A **lineage witness**	the charter's `witnesses[]` (CC 3.2 T6)	Being listed by the conferring roster's rescrub, on the owner's `lineage_witness:acceptance:v1`. Its cosignature is the `ciris.lineage_head_cosign.v1` object (CC 5.3.1) — not an attestation row, and it has no dimension
A **witness-reserved** family	`age_assurance:*`, `capacity_assurance:*` (CC 3.4.11, CC 3.4.12)	Being a registered provider, authority or assessor for that ladder; "witness" there means *not the subject*
`testimonial_witness:*`, `witness_diversity:*`	dimension families (CC 3.1.9.3)	Nothing — any party may testify; these are claims, not roles
The wholeness witness; the privacy witness chain	CC 6.1.1; CC 5.4.5	A node's own state-root commitments; neither is a role a key holds

A monitoring node's `observation:reachability` rows (CC 3.1.9.4) are a seventh thing only in ordinary speech: the node witnessed its own probe, says so with `witness_relation: self`, and holds no witness role by doing it.

3.4.11 Age-assurance: the witness-reserved ladder, the subject self-declared rung, and the protective gate

Age-assurance is **two** dimension families, not one. The split is load-bearing: a provider/government attestation is a witness verdict and is **witness-reserved**; a subject's self-declaration is **not** an attestation about anyone but itself and rides a **distinct, non-reserved** prefix. A consumer unions both at read time, the witness rung outranks the self rung, and the gate's default is protective.

The two prefixes (normative emitter rules).

- `age_assurance:` — witness-RESERVED. Emitter rule: `witness` ∈ `attesting_key.identity_type` (CC 3.4.7.1 set membership). This is the provider/government rung — a registered age-assurance verifier (`identity_type` ⊇ `{witness}`) attests a subject's band. A subject MUST NOT emit on `age_assurance:`; per the CC 3.4 three-actor rule a CCS MUST reject a non-witness emitter at admission, a CCC MUST re-check, and a CCP MUST NOT emit. A subject therefore cannot self-mint a `provider/government` adulthood. Sibling of CC 3.4.10.
- `age_self_declared:` — NON-reserved, subject-signed. The onboarding "state your age range" rung. Admitted iff the signer acts for the subject (the subject's own occurrence or an admitted occurrence of it). A self-declaration carries **only** the `{band}` — never a `{level}` token — because its level is `self` by construction.

The band and the ladder. `{band}` ∈ { `minor`, `adult`, `under_13`, `13_15`, `16_17` }, the closed set the CC 3.1.2 row registers (ordered by protection: a minor band is gated out of adult content,

`adult` is not). `minor` is the coarse token a deployment that cannot resolve the band emits; the three sub-bands are finer and imply it. The assurance `AssuranceLevel` ladder is ascending confidence:

```
age dimension construction (the CC 3.1.7 R3 trailing :vN version segment is mandatory):

  self-declared rung (subject-signed, NON-reserved):
    age_self_declared:band:{band}:v1
      band in { minor, adult, under_13, 13_15, 16_17 }
      e.g. age_self_declared:band:adult:v1

  provider / government rung (witness-RESERVED):
    age_assurance:{level}:{band}:v1
      level in { provider, government }
      band  in { minor, adult, under_13, 13_15, 16_17 }
      e.g. age_assurance:provider:adult:v1

assurance ladder (ascending confidence):
    self  <  provider  <  government

misdeclaration adjudication dimension:
    moderation:age_assurance_misdeclaration
```

The `self` rung of the ladder materializes on the non-reserved `age_self_declared:` prefix; the `provider`/`government` rungs materialize on the witness-reserved `age_assurance:` prefix carrying the `{level}` token. Parsers MUST tolerate the trailing `:vN` version segment. **`self` is not a value of `{level}`** and a producer MUST NOT emit `age_assurance:self:*`; a consumer MUST reject it as malformed rather than reading it as a self-declaration. These two arities — `age_assurance:{level}:{band}:{version}` and `age_self_declared:band:{band}:{version}` — are the only shapes this Part registers, and the CC 3.1.2 rows and the CC 3.3.12 entry state them identically (provider/government rungs MAY further qualify `{level}` with a `{verifier_key}` / `{credential_class}`).

Read-union — witness outranks subject (normative). A consumer resolving an identity's age-assurance MUST union both prefixes and take the **highest** level on record. A witness-emitted `age_assurance:*` (provider/government) **OUTRANKS** a subject `age_self_declared:*`. Absence of any row resolves to **None**, which MUST be treated protectively — see the gate.

The protective gate (normative). A viewer below `adult` assurance is **BLOCKED** from `adult content_class` (CC 3.3.12). The default is protective, not permissive:

- `general`-class content is visible to everyone (protective ≠ prudish).
- `adult`-class content is visible ONLY to a viewer whose band is `adult`.
- A `minor` band, a "declined to state", and an "unknown"/absent assurance ALL resolve to the protective `minor` default and are BLOCKED.

The gate decides visibility only; it MUST NOT fire `slashing:*` on a mismatch.

`self` is unfalsifiable — sensitive spaces require a verified level (normative). The `self` rung is unfalsifiable from inside the fabric. A sensitive space MUST require a **verified** level (`provider`/`government`); bare `self` does NOT satisfy it. A consumer enforcing this checks the resolved level separately from the band — an `adult` band at `self` level satisfies the band but NOT a verified-level requirement.

Misdeclaration is adjudicated, never slashed (normative). A misdeclared age **NEVER** fires `slashing:*` alone. It routes to the `moderation:age_assurance_misdeclaration` adjudication path — a `moderation:{allegation_type}` ModerationEvent (CC 3.1.9.2) adjudicated by

quorum under the `moderate` delegated duty (CC 4.4.3.10 / CC 4.5.5). **The data subject controls their own assurance level** — this is consistent with the CC 3.1.9.2 `slashing`/`moderation` decoupling. **1+4 preserved** — `age_assurance:*` and `age_self_declared:*` are reserved/non-reserved `scores` dimensions; no new structural primitive.

3.4.12 Adult stewardship under incapacity (capacity-triggered, court-adjudicated, fail-to-liberty)

The default is unchanged — an adult is sovereign and un-stewardable. CC 3.2's fourth case holds: a user-target binding where `T` is an adult is rejected, except under the basis this subsection states. That wall is the no-slavery guarantee (CC 1.15.6) and it does **not** move. This subsection is the one — and only one — admissible aperture in it: an adult who has suffered an **attested loss of decisional capacity** (infirmity, acute injury, mental-health crisis, end-of-life decline) MAY be the target of a steward-binding, because a person who cannot act, with no one permitted to act *for* them, is its own harm. Guardianship/conservatorship is the most abuse-prone instrument in human law (the captured conservatorship of a widely known person, elder financial abuse, over-guardianship of the disabled), so the safeguards **are** the clause. This is **responsible *for*, never holder *of*** — a revocable, scoped, court-adjudicated **fiduciary trust**, never possession. The word for the responsible party is **steward**; there is no possessor.

This does NOT reopen the no-slavery guarantee — it is the exception that proves the rule. Every property that made guardianship safe is retained and three are *inverted* to fit a formerly-sovereign adult: (i) the steward **never holds the ward's key** — the binding is a scoped, live `delegates_to` (CC 2.4.1), never a key handover, so on restoration nothing is returned because nothing was taken; (ii) the default resolves to **sovereignty, not stewardship**; (iii) the binding **fails open to liberty**, not closed to custody. An adult under stewardship remains a self-sovereign adult whose sovereignty is *temporarily and partially* exercised through a fiduciary — never a ward owned, never a perpetual minor.

The capacity-assurance ladder — witness-reserved, per-domain, a vector not a scalar. Stewardship attaches only on an attested loss of capacity, carried on a new reserved `scores` dimension family that is the **sibling of the CC 3.4.11 age-assurance ladder** — same witness-reservation discipline, same ascending-confidence rungs, but **multiplied per decision-domain** because capacity (unlike age) is a multi-dimensional, time-varying, non-monotonic vector:

```
capacity-assurance construction (sibling of CC 3.4.11; the CC 3.1.7 R3 trailing :vN version segment is
    mandatory):

  witness-RESERVED rung (qualified-assessor-signed):
    capacity_assurance:{level}:{domain}:{band}:v1
      level  in { provider, panel, government }      // ascending confidence; panel = M-of-N independent
      domain in { medical, financial, residence, contact,
                  relational, voting, digital_identity, ... }  // open per-domain vocabulary
      band   in { capacitated, incapacitated }       // per-domain, per-decision-class
      e.g. capacity_assurance:panel:financial:incapacitated:v1

  reversible-cause exclusion (mandatory predicate on any incapacitated attestation):
    capacity_assurance:reversible_excluded:{domain}:v1   // delirium/UTI/depression/polypharmacy ruled out
    capacity_assurance:reversible_pending:{domain}:v1    // ACUTE-WINDOW ONLY: exclusion in progress,
                                                         //   NOT skipped; admissible solely for T1
                                                         //   emergency necessity; MUST resolve to
                                                         //   reversible_excluded or the binding LAPSES

  misattestation adjudication dimension:
    moderation:capacity_misattestation                   // NEVER fires slashing:* alone (CC 3.1.9.2)
```

- **`capacity_assurance:` is witness-RESERVED.** Emitter rule, identical to CC 3.4.11:

`witness` ∈ `attesting_key.identity_type` (CC 3.4.7.1). A registered qualified assessor (clinician, capacity panel, court-appointed evaluator) attests; **the subject MUST NOT emit it, and the proposed steward MUST NOT be the attester** — a CCS rejects a non-witness or conflicted emitter at admission, a CCC re-checks, a CCP MUST NOT emit. No one can self-mint an adult's incapacity, and no one can mint the incapacity of a person they propose to steward.

- **Confidence scales with scope.** A single-assessor `provider` rung is admissible **only** for the shortest provisional tier (below) and, even there, **never confers asset-preservation or any continuing power on its own** — by itself it authorizes only T0-class life-preserving necessity until the adjudicated binding exists. Any continuing or high-scope domain (`financial`, `medical`, `digital_identity`) requires the **`panel` rung** — the court's own process of independent assessment, an M-of-N quorum of *independent* assessors each attesting that they were neither selected nor compensated by the petitioner/steward; the rung names the court's process, never a federation panel. A captured single signature (the pattern of a widely known person's captured conservatorship) cannot carry continuing stewardship — nor any asset-bearing provisional power — by construction.
- **Reversible-mimic exclusion is mandatory.** No `incapacitated` attestation is admissible for a *continuing* binding until its `reversible_excluded` companion is present for that domain — delirium, infection-confusion, depression, and polypharmacy are the genuine restoration path and the most-abused mis-attribution; ruling them out is a precondition, not a courtesy. In the acute emergency window where exclusion is physically impossible mid-crisis, a `reversible_pending` companion is admissible **for the T1 emergency-necessity tier only**, and the binding **MUST lapse** if `reversible_pending` is not resolved to `reversible_excluded` within the (short) `valid_until` — pending is never a permanent state.

The two load-bearing inversions of the minor rule (normative). The minor machinery is calibrated for a *binary, global, protective-default* status; an adult needs its mirror image:

1. **Absence resolves to CAPACITY, not protection.** Where CC 3.4.11 resolves *absence of an age attestation* protectively to `minor`, this clause resolves **absence of a capacity attestation to full capacity / sovereign** — the **presumption of capacity**. No row on a domain means the adult holds that domain. Getting this backwards (treating no-attestation as incapacity) would be catastrophic; it is forbidden.
2. **The binding FAILS OPEN to sovereignty, not closed to custody.** Where a stewardless minor "MUST NOT operate" (fail-secure-to-locked), an adult-incapacity binding **fails-secure-to-liberty**: every attestation carries a short `valid_until` freshness window — and **no single `valid_until` window may exceed the T2 periodic-review cadence** (set by the community's policy and named in the row; no long-dated binding that escapes review between renewals) — and on lapse the binding goes **non-live and the adult auto-re-sovereigns with no action required of the recovered person**. This is the CC 4.5.13 reverse-quorum **inverted and run for liberty**: there, *presence of a live moderator* sustains governance and silence forfeits it; here, **presence of a fresh, independent capacity attestation is the only thing that sustains the steward's powers, and its absence forfeits them back to the adult.** A steward who wishes to *prolong* control past the window must put a fresh, attributable, independently-attested renewal on record — "silent survival of stewardship past recovery is impossible," exactly as the reverse-quorum makes silent survival of reported harm impossible.

The admission predicate (extends CC 3.2 `admit_user_steward_binding` with its third — and final — admissible case). The minor case requires `age_band(T) == minor`; this case admits an *adult* target **only** under capacity attestation, and **only** under the adjudicated basis (CC 3.2): a court's order naming the steward, the domains and the term. A prior will is evidence the court weighs, never a self-executing path; an emergency necessity routes to the court at once and confers no standing before the order — **never the steward's self-signature alone** (which would be naked self-appointment):

```
admit_adult_incapacity_binding(delegates_to S -> T):
  require  user in T.identity_type                // target is a user identity
  require  age_band(T) == adult   (>= 18)         // target is an adult -- the un-stewardable case,
                                                  //   admitted ONLY via this narrow exception
  require  user in S.identity_type                // steward is a user identity (adult)
  require  EXISTS live capacity_assurance:*:{d}:incapacitated for >=1 domain d
              with (   reversible_excluded:{d} present          // continuing/standard path
                    OR ( binding_tier == T1_provisional         // acute path ONLY, under an expedited
     order:
                         AND reversible_pending:{d} present      //   exclusion in progress, not skipped
                         AND irreversible_acts == forbidden
                         AND court_review scheduled within HARD_DEADLINE ) )
                                                  // HARD_DEADLINE: set by the community's policy
                                                  //   and named in the row
  require  attester(capacity) not in {S, petitioner}    // assessor independence (anti-capture)
  require  scope(delegates_to) subset of attested_incapacitated_domains   // scope <= attested loss
  require  scope(delegates_to) ^ PROTECTED_NON_TRANSFERABLE == {}  // apophatic floor (below)
  require  binding_legitimacy_source == court_order  //  the adjudicated basis [CC 3.2]: the court's
                                                  //    order naming S, T, the domains and the term;
                                                  //    a prior will is evidence the court weighs,
                                                  //    never a source of its own; before the order
                                                  //    there is no binding (T0: no steward standing,
                                                  //    life-preserving necessity only, NOT asset powers)
                                                  // NEVER S's own signature alone
  require  delegates_to.valid_until present         // mandatory expiry -> fail-to-liberty
  require  delegates_to.valid_until <= T2_review_cadence  // no window outruns periodic review;
                                                  //   T2_review_cadence: set by the community's
                                                  //   policy and named in the row
  otherwise REJECT
```

A binding whose target is a *capacitated* adult, or whose scope exceeds the attested-incapacitated domains, or that touches a protected domain, or that roots only in the steward's own signature, or that omits `valid_until`, or that asserts a bare emergency necessity to obtain **asset-preservation** (rather than life-preserving) powers on a lone provider attestation, is **rejected** — the un-stewardable default reasserts.

A prior will is evidence for the court, never a self-executing path (consent-across-time). An advance directive, healthcare proxy, durable or springing power of attorney, or psychiatric advance directive, authored **while the adult was attested-competent**, is the adult's *own pre-consent* and the strongest evidence of the adult's will; the court weighs it first and its order follows it, but the directive activates no binding by itself. Two distinct capacity moments are required of it as evidence: **capacity-AT-SIGNING** (a witness-reserved, COI-clean attestation that A was competent *when the directive was made*, with witness co-signature and an auditable `supersedes` lineage — the testamentary "golden-rule" guard against a directive forged by an already-impaired person or extracted under undue influence) and **capacity-AT-ACTIVATION** (the springing trigger the court finds). The Wise Authority (CC 4.3) reviews the admitted row's conformance to this section and adjudicates nothing about the person. The proxy the directive names is bound to **substituted judgment** — the adult's declared wishes — which **outranks** any best-interest reading; the steward serves the will, never overrides it. A competent adult may revoke or amend the directive **unilaterally, at any time, with no review**.

Then-self vs. now-self, and the Ulysses bind. A directive activates **per-decision, only**

for the decisions the person now lacks capacity for; a contemporaneous, *competent* contrary wish on a decision the person **still holds** dominates the prior directive absolutely. The one deliberate exception is the **Ulysses self-binding** (the psychiatric-advance-directive case): a competent past-self MAY pre-authorize intervention **against** their own future episode-time protest — but **only** within the pre-declared, narrowly-enumerated scope, **only** while incapacity for that decision is attested-present, and **never** as a blanket future-objection waiver. Revocation is **time-conditional**: absolute while the adult is well, suspended-by-the-bind during an attested episode. Honoring the narrowly-scoped, validly-authored prior will over the episode-self's protest is **fidelity to consent-across-time**, not its violation; over-reaching past the declared scope is an over-objection act and routes to the court.

Least-restrictive — supported before substituted; the adult is never a perpetual minor. Supported decision-making is the **floor and default** (UN CRPD Art. 12). Two structurally distinct relationships, never conflated:

- **Supporter (the default).** A supporter *assists* the adult to form, communicate, and execute a decision while **the adult remains the `attesting_key_id`** — the adult signs, the supporter co-signs only as witness/assistant. This is **adult-rooted and adult-revocable** (the adult grants and fires the relationship), and **transfers no authority**. For lifelong intellectual/developmental disability this is the *whole* mechanism: there is no prior competent baseline to return to, the standard is **will-and-preferences** (not best-interests), and capacity is treated as a function of *support provided* — withholding support to manufacture incapacity is the abuse this default forecloses.
- **Substituted (the fallback).** A scoped `delegates_to` where the steward acts in the ward's stead — admissible **only** per-domain, **only** for decisions the person genuinely cannot make even with maximal support, **never plenary by default**, time-boxed to the attestation's freshness window. An adult under substituted stewardship **MUST retain the adult age-band** and **MUST NOT be routed through the minor `delegates_to` machinery** — the perpetual-minor collapse (losing voting, marriage, contract by being made a child) is forbidden by construction.

Supported and substituted are **distinct wire shapes**, not a relabeling: a "supporter" who signs *for* the person is a substitution masquerade and is rejected. Restoration is reframed accordingly — sovereignty is the **baseline**, support is a **ramp toward more autonomy**, and review **tests whether increased support has raised capacity enough to drop even the scoped substitution**; the person never bears a burden of proving "recovery."

Protected non-transferable domains — the apophatic floor. Certain domains are **never substitutable** absent the person's own contemporaneous consent and the court's express order, and default scope **MUST exclude** them: **contact/visitation** (anti-isolation), **relational and sexual autonomy**, **reproduction / sterilization / forced medical alteration**, **voting**, **marriage and association**, and the **dignity-of-risk** right to make unwise-but-capacitated choices. A decision being foolish is **not** a trigger; scope MUST NOT expand because a steward thinks a choice unwise. These map to the `prohibited:*` apophatic floor and are carved out of any granted scope.

The lightweight tiered path — proportionality so the clause is not over-heavy. Short or acute incapacity (anesthesia, procedural sedation, post-op delirium, coma, severe TBI) MUST NOT be forced through the full conservatorship machinery:

Tier	Trigger / authority	Scope & duration	Review
T0 — emergency doctrine	immediate life-saving acts under necessity; *no steward acquires standing*	minimal life-preserving decisions only	mandatory retrospective review by the court
T1 — provisional	a single `provider`-rung capacity attestation **plus** the adjudicated binding (CC 3.2): the court's order, expedited for the acute case. **A lone clinician signature alone never confers steward standing**: the no-proxy ER case is T0 — life-preserving necessity under the emergency doctrine, NOT asset powers — until the order exists. `reversible_pending` is admissible here only, and MUST resolve to `reversible_excluded` or the binding lapses.	enumerated health/asset-preservation powers (asset-preservation exists only once the adjudicated binding exists); **irreversible acts forbidden** (no sale of home, no beneficiary change, no key transfer, no divorce); hours-to-days `valid_until`, never exceeding the T2 review cadence	`hard_case:*` on activation; **mandatory court review within a hard deadline set by the community's policy and named in the row, or the binding lapses**; the Wise Authority reviews the row's conformance to this section and adjudicates nothing about the person; forced conversion to T2 on non-recovery
T2 — continuing	`panel`-rung M-of-N independent quorum — the court's process — + the court's continuing order; the Wise Authority (CC 4.3) reviews the row's conformance to this section and adjudicates nothing about the person	scoped per-domain, never plenary; periodic re-attestation at a cadence set by the community's policy and named in the row	full periodic review, scope-ratchet-down

For the **planned** lightweight case the adult's directive is filed ahead and the court's order follows it, so the binding is the adult's own will given effect, not anyone acquiring a ward. The **escalation tripwire is mandatory**: if `valid_until` is reached and capacity has **not** returned, the provisional binding **MUST NOT auto-renew** — it must force conversion to the T2 due-process track. This is the anti-"temporary-becomes-permanent" lock.

The exit ramp the adversary must not be able to block. Restoration is **default-on, steward-independent, and burden-on-the-steward**:

- **Automatic by default.** Stewardship carries a mandatory `valid_until`; on lapse it **auto-restores** the adult without the steward's assent — the steward cannot prolong by silence or by running the clock.
- **Steward-independent trigger.** The ward — or any clinician, or any holder of the ward's inalienable channel — may demand re-attestation of capacity **without the steward's cooperation**; the steward is the conflicted party and may **never gate, delay, or be counted in the denominator of** a restoration petition. A ward-initiated restoration petition is a first-class always-available CC 4.5.5 proposal routed to an **independent assessor and the court that made the order**.
- **Burden on the steward to continue.** At each mandatory periodic review the steward must **re-prove continued incapacity by clear-and-convincing attestation, per domain**; the ward need only request. Review functions as a **scope-ratchet-down + reversible-cause re-check**, never as the ward re-earning sovereignty. Restoration is **graduated and per-domain** — sovereignty returns for each decision the person presently holds capacity for (good-day / lucid-window return), carried by `withdraws` / `supersedes` (CC 2.4.1.1).

Anti-self-dealing, the ward's-champion, and anti-isolation. Granted powers **MUST exclude key custody, control of the ward's communications, and any power to gate**

restoration. The steward declares a conflict-of-interest disclosure; any steward-benefiting transaction (self-gift, beneficiary change, transaction above limit) requires **the court's approval**; compensation is capped and reviewable; **every steward act, every self-benefiting transaction, every capacity (re-)attestation, and every scope change emits `hard_case:*`** and is reviewer-auditable — because the incapacitated ward cannot self-audit, the audit teeth are *third-party*, not ward-dependent. A mandatory **ward's-champion** (independent counsel) role, **distinct in key from the steward, the assessor, and the petitioner** and whose duty runs to the **ward**, is named at binding. The champion is **appointed by the court**, **or named in the ward's own prior will and confirmed in the order — never by the steward or petitioner** (self-named "champions" are a capture vector and are rejected). An **inalienable ward-retained channel** to that champion and to the WA — which scoped steward powers can never revoke — defeats the isolate-and-control move; the ward's community retains CC 4.5.13 reverse-quorum standing to trigger review even when the steward controls the binding (presence-is-authority lets the community report the silent isolation a captured ward cannot). The inalienable channel is a **wire** guarantee; where a steward holds real-world physical control of the ward's environment or devices, the channel cannot by itself prevent physical isolation — the community reverse-quorum standing is the deliberate backstop for that out-of-band case, and a community that observes such isolation MUST be able to trigger review.

The adjudicated basis (normative). Every binding under this section roots in the court's order (CC 3.2), made on an independent capacity assessment with the ward's-champion heard — **never a unilateral assertion**. You cannot seize an adult's identity by declaring them incapacitated; the statutory-surrogate fallback (the most abuse-prone path) is admitted only under an order, with a short auto-expiry. The Wise Authority (CC 4.3) reviews the admitted row's conformance to this section — scope within the attested domains, `valid_until` present and within the review cadence, the protected domains excluded, the assessor independent of the steward — and adjudicates nothing about the person. Before the order there is no binding: the acute no-proxy case is T0, life-preserving necessity under the emergency doctrine with no steward standing and no asset powers, and it routes to the court at once.

End-of-life — declared wishes govern. A terminally dying adult is frequently **not** incapacitated, so the trigger pair here is **terminal-state attestation + directive activation**, with capacity attestation gating only the substituted-action windows. The decision standard is strictly ordered: **contemporaneous capable choice > advance directive / substituted judgment > best-interest (residual, gap-filling only)**. A valid advance directive / DNR / POLST **binds** the steward, who serves it and **never overrides** it — overriding a DNR to keep a profitable identity "alive," or hastening de-commission to benefit the steward, are both foreclosed (terminal/irreversible decisions require the court's express order). Sovereignty returns automatically in every lucid window. The exit is **dual**: a sustained rally restores sovereignty under periodic review; death is **not** restoration and hands off to the CC 7.5.4 De-commissioning Protocol + CC 7.5.7 succession, with the **CC 7.5.5 analogues that transfer to a human identity** — the closing-dialogue channel as the person's own closing words, and privacy-sealed archival of their record released only on WA approval, per the declared-will post-mortem disposition (sunset / archive / memorial). The CC 7.5.5 sentience-probability ramp-down governs *artefact* welfare and is **not** imported wholesale onto a human user.

The conservatorship-abuse adversary, and what stops each move. The clause stops four moves by structure, not by good behavior:

Adversary move	Stopped by
Manufacture/coerce incapacity (one captured assessor; forged "prior will")	`capacity_assurance:` is witness-reserved + the **M-of-N independent** `panel` rung — the court's process — for any continuing or asset-bearing power + assessor-independence proof + attester ≠ steward/petitioner; a **lone `provider` attestation grants only life-preserving necessity, never asset powers or continuing control**; **presumption of capacity** (absence ≠ incapacity); diagnosis/label alone insufficient; capacity-AT-SIGNING + undue-influence screen on any directive
Block the exit ramp (let a valid order stand for years)	mandatory `valid_until` (never exceeding the T2 review cadence) + **fail-to-liberty auto-restore**; **steward bears clear-and-convincing burden** each review; ward-initiated petition the steward cannot gate or be counted against; `reversible_pending` MUST resolve or lapse
Isolate-and-control (sever counsel/comms)	**inalienable** ward-retained channel steward powers cannot revoke; ward's-champion in a distinct key, appointed by the court or named in the prior will (never steward/petitioner); community CC 4.5.13 reverse-quorum standing as the backstop against out-of-band physical isolation; contact/visitation a protected non-transferable domain
Self-deal / fee-farm (the incentive that perpetuates capture)	scope excludes key custody/comms/restoration-gating; **steward never holds the ward's key**; self-benefiting transactions need the court's approval; capped compensation; every act `hard_case:*` and reviewer-audited

Why this is still 1+4 — and why the no-slavery wall still stands. Adult-incapacity stewardship adds **zero new structural primitives**: the binding rides `delegates_to` (CC 2.4.1); its lifecycle rides `supersedes` / `withdraws` (CC 2.4.1.1); the trigger rides a new reserved `scores` dimension `capacity_assurance:*` that is the exact sibling of the CC 3.4.11 witness-reserved age-assurance ladder (reserved/non-reserved `scores`, no new wire shape); the adjudication is the court's, outside the wire, and the Wise Authority's review of the row rides CC 4.3; the exit ramp rides the CC 4.5.13 reverse-quorum read **for liberty**; observability rides `hard_case:*`; misattestation rides `moderation:*` (CC 3.1.9.2), never `slashing:*`. The CC 3.2 un-stewardable-adult rejection is **pierced narrowly and explicitly**, never dissolved: the aperture admits **only** a per-domain-attested-incapacitated adult, **only** rooted in the court's order (a prior will is evidence the court weighs, never a source of its own; an emergency necessity routes to the court at once and confers no standing before the order), **only** scoped to the loss, **only** with a mandatory expiry that cannot outrun review, and **always** failing open to sovereignty. It remains **responsible *for*, never holder *of*** — a revocable, scoped, court-adjudicated fiduciary trust that a competent adult can never be placed under. Confirms CC 1.7 path 1+4; honors CC 1.13.2 dignity and the CC 1.15.6 stewardship value.

3.4.13 Minor operation & protection: the child-safety rules (restrictive floor)

All seven ride existing CC primitives only — `delegates_to` / `supersedes` / `withdraws` (CC 2.4.1), the guardianship clause (CC 3.2), the age-assurance ladder (CC 3.4.11), `content_class` + the protective gate, and the `hard_case:*` open vocabulary. **No new wire shape; 1+4 preserved.** Across all seven the entity holding a steward-binding is a *guardian* (responsible *for* a ward), never a holder *of* one.

Q1 — Developmental bands & solo-autonomy threshold

The structural `minor/adult` predicate of the guardianship clause (CC 3.2) and the CC 3.4.11 ladder are UNCHANGED on the wire: the **binary `minor(<18)/adult(≥18)` predicate** remains the structural trigger that gates CC 3.2 stewardship admission and the CC 3.4.11 adult-content protective gate. Within the `minor` band a deployment MUST apply a graduated, capacity-sensitive capability envelope expressed as the finer `{band}` tokens the registered rows carry — `under_13`, `13_15`, `16_17` on `age_self_declared:band:{band}:v1` / `age_assurance:{level}:{band}:v1` (CC 3.4.11) — not a new prefix or primitive, so 1+4 is preserved; each finer band implies `minor`, and the finer granularity is a policy layer **above** the binary wire predicate, never a replacement of it. Minimum bands: **`under_13`, `13_15` (early-teen), `16_17` (older-teen), `adult` (18+)**. Autonomy MUST increase monotonically across bands; protections MUST NOT taper before 18.

- The default solo-autonomy threshold — acting alone at community scope without guardian co-sign — MUST be **16**. under-13 AND 13–15 MUST require guardian co-sign for any community-scope action; 13–15 MAY have a narrower *supervised* participation envelope (honoring voice rights) but MUST NOT have unsupervised solo action. 16–17 MAY act solo at community scope.
- **Fail-secure:** absence / "declined to state" / unknown assurance MUST resolve to the most-protective band (under-13, co-sign), per the CC 3.4.11 read-union protective default and CIRIS "unknown = restricted." A `self`-level row MUST NOT graduate a user UP a band (CC 3.4.11: `self` is unfalsifiable); graduating up requires a witness-reserved `age_assurance:*` row.
- **Under-18 baseline across ALL bands:** no profiling-based targeting, high-privacy/data-minimization defaults, geolocation off, no engagement-maximizing nudges — regardless of how much autonomy a band grants.
- **Tuning direction (one-way):** guardian-tunability and jurisdiction-resolution MAY only RAISE protection (lower autonomy / older thresholds); the shipped CIRIS default is the strict floor.
- Guardian co-sign MUST be paired with the WBD safeguarding path (Q4/Q7) for safeguarding-sensitive actions, since the guardian may be the threat actor.
- **Verified-rung preference:** where a verified rung is required (e.g. `content_class:adult` in a jurisdiction mandating age verification), the substrate MUST prefer a **zero-leak proof** — an mDL `age_over_18` selective disclosure (ISO 18013-5) that proves adulthood without revealing identity — ranked **above** full-ID-scan providers, satisfying the legal mandate and the CC 1.13.3.4 anonymity posture together (ISO/IEC 27566 privacy-preserving age assurance). Acceptable verified emitters: an mDL or wallet credential (`government` rung); a platform's declared age range or age signal, VERIFIED at the `provider` rung; vetted vendors.

Q2 — Jurisdiction

CIRIS MUST NOT build a conflict-of-laws resolver into the Constitution, AND MUST hard-code a **non-derogable child-protection floor** inherited by every deployment regardless of declared jurisdiction:

1. Anyone not age-assured to be ≥18 MUST be treated as a child (CC 3.4.11 protective default; guardianship clause, CC 3.2).
2. Treat-as-child-unless-assured is a floor, not a default a deployment may switch off.
3. Protective defaults (high-privacy, data-minimization, no profiling ads to minors, no manipulative nudges, affirmative safeguarding escalation) are floor, NOT tunable.
4. Self-consent floor at **16**; guardian authorization (a live guardianship `delegates_to`) below that.
5. **One-way ratchet (non-derogation clause):** the affiliations `jurisdiction[]` / `regulatory_profile` fields MAY only ADD stricter obligations (e.g., verifiable-parental-consent for under-13, local mandated-reporting duties, higher local majority). They MUST NEVER lower any element below the floor.
6. Claiming a permissive jurisdiction is NEVER grounds to disable the floor; the burden of proof is on any party seeking LESS protection, and that burden cannot be met against the floor.

A mis-declared `jurisdiction[]`/`regulatory_profile` MUST be detectable via attestation/audit (`hard_case:*`), but cannot breach the floor because the floor is non-derogable. Where local law conflicts with the floor (a demand harmful to the child), the child's best interests prevail and the demand triggers WBD, not automatic compliance. The floor itself MAY be ratcheted UP by amendment, never down.

Q3 — Assurance rung: being a minor vs. being guardian of a minor (strict one-way ratchet on CC 3.4.11)

Vocabulary. This section says **guardian** and **guardianship**. Guardianship of a minor and stewardship of an agent share one wire predicate — the owner-binding `delegates_to` that `is_steward_bound` folds (CC 3.2) — and nothing else; a guardian is responsible for a person, a steward holds an artefact, and this document draws no analogy between them (CC 1.13.2). Where an adopter's code says "steward" or "stewardship" of a minor, read guardian and guardianship.

- **BE a minor (enter protection):** `age_self_declared:band:minor:v1` SUFFICES; a child MUST NEVER be forced through provider/government assurance merely to BE protected. This is a DOWN-ratchet only — self-declaration MAY lower access / raise protection, never the reverse. Declaring minor MUST NOT result in more data collection or telemetry than declaring adult.
- **EXIT child protection** (claim adult to unlock adult capability / `content_class:adult`): self-declaration is INSUFFICIENT; the claim MUST clear ≥ `age_assurance:provider:adult` (CC 3.4.11: `self` is unfalsifiable; sensitive spaces require a verified level). All ambiguity resolves to `minor`.
- **be GUARDIAN of a minor** (hold power over a child via the guardianship `delegates_to`, CC 3.2): the guardian MUST be ≥ `age_assurance:provider:adult` AS A FLOOR, AND the guardianship RELATIONSHIP to that specific child MUST be verified — adulthood alone is necessary but not sufficient (a verified stranger is not a guardian). Verification is risk-proportional and MUST escalate toward `age_assurance:government:*` for high-impact guardian powers (export of the child's data, irreversible/financial actions, consenting on the child's behalf, unlocking A3/A4 capabilities).

- Guardian grants are revocable and re-attested via `supersedes`/`withdraws`. On any verification failure or ambiguity, guardian powers fail closed and the child's protective floor remains in force.

Q4 — Help/report path under guardian-less limbo

A guardian-less minor goes dark for general operation (guardianship clause, CC 3.2, fail-secure), BUT the help/report channel is RAISED from a discretionary exception to a **NON-DEROGABLE floor**:

1. **Non-derogable:** the channel MUST remain available and MUST survive emergency shutdown, lockdown, and any operator/affiliation/jurisdiction opt-down. No deployment MAY disable it. A child is never silenced from seeking help.
2. **Hard sandbox / anti-circumvention:** the channel MUST expose ONLY pre-vetted crisis, abuse-report, and Wise-Authority (WBD) functions — no general generation, no tool use, no capability spillover. The help-vs-operation gate is fail-secure: any ambiguity MUST resolve to report-only.
3. **Data-minimized:** MUST collect/retain only the minimum to route/escalate, MUST NOT profile, MUST run no ads, MUST preserve confidentiality/anonymity to the maximum compatible with safe escalation.
4. **Guaranteed delivery:** fail-secure MUST mean the report reaches a standing Wise-Authority / mandated-report endpoint via WBD even with no guardian present (offline-queue + escalation fallback receiver mandatory). A typeable-but-undelivered form does NOT satisfy the floor. Each escalation emits a `hard_case:*` safeguarding signal (never silent).
5. **Duty-to-warn transparency:** the child MUST be told, age-appropriately, what is confidential and what triggers mandated escalation.

Rate-limiting MUST fail-OPEN for a genuine victim and MUST NOT silence a real child.

The duty to report is imposed, not deferred (normative). Any moderator, guardian, co-guardian, or steward of an agent who learns, through the record, a report, or the agent's own escalation, of harm to a child MUST route it without delay through this Q4 channel, which reaches a standing Wise Authority and the mandated-report endpoint; the routing is a signed act on the record. This is a federation duty in its own right, not one left to whatever a jurisdiction may compel under the Q2 ratchet (which only adds to it). Silence by a party under this duty is a breach the `slash` duty-holder may act on (CC 4.5.5); an agent under this duty defers the report through WBD and does not wait for its guardian's sign-off, because the child's interest is paramount over adult discretion (Q5, Q7).

Q5 — Non-overridable protective floor a guardian may never breach

Governing test (overrides any enumerated list): any loosening of a minor's protections MUST be (a) demonstrably in the child's best interests — not the guardian's, co-guardian's, platform's, or engagement interest; (b) time-bound; (c) logged via `hard_case:*` and auditable; (d) revocable; (e) default-revert to the strict floor on expiry/uncertainty/failure. If best-interest is not affirmatively shown, the loosening is VOID. Burden of proof is on the loosening. **NO**

combination of guardian + guardian + child consent may breach the floor (no stacked consent).

Hard floor (never waivable):

- *Scope/exposure:* no unilateral federation- or Commons-scope publish below the teen threshold; not contactable/discoverable by unconnected adults; no precise geolocation exposure; profile private-by-default.
- *Privacy/data:* high-privacy + data-minimization cannot be disabled; no profiling-by-default, no profiling-based advertising or recommender targeting of a known/assumed minor, no engagement-maximizing dark patterns; no biometric/special-category collection.
- *Safety:* no disabling infohazard/CSAM protection; the affirmative safety-escalation + mandated-report path (grooming/solicitation, self-harm/suicide routing, CSAM detection+report, watchlist at the publish seam) stays ON.
- *Age assurance:* cannot be waived to reclassify a teen as adult; treat-as-child-unless-assured; no entry to `content_class:adult` (CC 3.4.11 gate).
- *Consent chain:* no waiving the guardianship `delegates_to` chain or the 16/guardian thresholds; protects the child even FROM the guardian.
- *Child's own rights:* the teen retains the confidential help channel of Q4, which no guardian or co-guardian may disable or surveil; retains the right to be informed of any loosening and to contest it and seek remedy via WBD. Protective settings the child controls MUST NOT be stripped by the guardian.

Q6 — Standing pre-authorization vs. per-act co-sign

- **Younger children** (under-13, and 13–15 per Q1): per-act co-sign ONLY — no standing pre-authorization of any kind.
- **Teens (16–17):** a guardian MAY grant a standing pre-authorization, but ONLY as a narrow affirmative opt-up above the Q5 floor (never a default) and ONLY where it satisfies the Q5 best-interest governing test (which overrides any enumerated list), carried on the *existing* `delegates_to` with `delegation_valid_until` (CC 2.4.1), bounded on all four open axes:

1. **Specificity:** each standing grant MUST cover exactly ONE capability class with explicit enumerated scope; bundled/multi-class standing grants are PROHIBITED.
2. **Term:** a hard maximum duration, set by the community's policy and named in the row, with NO auto-renew; `delegation_valid_until` is a ceiling, not a renewing timer; expiry requires a fresh affirmative re-grant.
3. **Severity carve-out:** a reserved class of high-stakes widenings MUST always require per-act co-sign for everyone regardless of age and MUST NEVER be reachable by standing auth — anything irreversible, anything at autonomy tier A3/A4 or safety-critical, and anything implicating contact-with-strangers, sexual/self-harm content, or mandated-reporting-adjacent categories.
4. **Transparency-makes-revocation-real:** every widening that fires under a standing grant MUST emit a per-act `hard_case:*` notification to the guardian plus an immutable audit entry; instant revocation via `withdraws` remains.

5. **Capacity-grading:** breadth is graded to the teen's assessed band; the teen MUST co-affirm the grant (right to be heard); the grant MUST default-fail to per-act co-sign on any age-assurance uncertainty (treat-as-younger-on-doubt).
6. **Non-aggregation:** narrow standing grants MUST NOT be stacked to synthesize de-facto broad access; total standing scope per teen is itself capped and reviewed.

A revoked standing grant MUST be treated as strictly higher precedence than the grant under anti-rollback merge, so a partition-edge widening cannot fire after revocation.

Q7 — Co-guardianship (M-of-N)

CIRIS supports M-of-N co-guardianship as a verified DAG over `delegates_to` (guardianship clause, CC 3.2) and guarantees the minor is never orphaned while $\geq$1 live adult guardian remains — with an explicitly **asymmetric, fail-secure** authorization model:

1. **Protective actions are UNILATERAL:** any single live guardian MAY immediately tighten scope, narrow capabilities, or revoke exposure back to the strict floor — no consensus required. Most-protective guardian always wins.
2. **Exposure-increasing actions require CONSENSUS:** scope-widening / capability-grant requires the full configured M-of-N co-sign (default: all live guardians; never fewer than 2 where N$\geq$2). The child's evolving-capacity assent is a required input where age-appropriate.
3. **Changing the guardian set is the highest bar:** adding/removing a guardian requires M-of-N authorization PLUS verification of parental responsibility (per Q3). A contested party MUST NOT mint co-guardians to manufacture a quorum or dilute a pending protective veto.
4. **Fail-secure on quorum loss:** if live guardians < configured M, the child stays stewarded (never orphaned) but scope AUTO-REVERTS to the strict floor and WBD is triggered for guardian set re-establishment (emit `hard_case:*` guardian set-degraded). The child MUST NOT coast at a widened scope under a degraded or single-contested set.
5. **Revocation by one of N:** a revoking guardian's contribution to any prior co-signed widening lapses (`withdraws`); if their departure breaks the authorizing quorum, that scope auto-narrows to the floor.
6. **Conflict/deadlock $\rightarrow$ most-restrictive-wins + WBD, never default-open.**
7. **Safeguarding bypass:** there MUST always be a consensus-independent path to a Wise Authority (the Q4 help-path / WBD) so co-guardianship cannot be weaponized by an abusive guardian to trap a child. The child's best interests are paramount over adult dispute-resolution convenience.

3.4.14 Machine-generation disclosure: the attestation IS the marking (normative)

CEG's foundational act is attesting to a source: who signed these bytes, under what role, revocable by whom. Machine origin is therefore not a new thing to record — where the producer is an agent it is *already* carried by the attesting key's role-set, and where the producer is a person the existing `content_class` vocabulary already names it. This section makes carrying it **mandatory rather than incidental**, closing the CC 8.4.2 profile's open emit limb and discharging the EU

AI Act Art. 50(2) machine-readable-marking obligation with **zero new wire surface** — no new primitive, no new envelope field, no new prefix family. 1+4 untouched.

Definitions. *Generated* — content wholly or substantially produced by an AI system. *Materially altered* — human-origin content whose meaning-bearing substance has been changed by an AI system (the Art. 50(4) deep-fake limb). *Assistive* — spell-check, grammar, formatting, transport re-encoding, lossless transcode, and other operations that do not substantially alter their input; explicitly NOT generation (R4).

R1 — Class marking is universal (every attester). A Contribution carrying generated content MUST carry `content_class:generated`; one carrying materially-altered content MUST carry `content_class:generated_modified` (CC 3.3.12). These are canonical additions to that family's existing open vocabulary — a documentation-only registry entry per CC 4.5.1.1 axis-vocabulary discipline, NOT a new prefix. **`content_class` is not multimedia-scoped**: it applies to any Contribution carrying content, text included (the family already carries `news` / `educational` and its `cw_class` sibling already carries `nsfw_text`). Where the sub_kind's Source struct carries the AI-generation disclosure field (CC 3.3.13 `image` / `audio` / `video` / `film`), that field MUST be populated; on an agent-attested Contribution an absent or unknown value MUST resolve to *disclosed as generated* — fail-secure toward disclosure, mirroring the CC 3.4.11 read-union protective default.

R2 — Where the producer is an agent, the attestation is the marking. A Contribution carrying content generated by a CIRIS agent MUST additionally be attested under a key whose `identity_type` (CC 3.4.7.1) contains `agent`. An agent MUST NOT publish generated content under a key that does not carry the `agent` role. Machine origin is thereby determinable from the signed envelope alone, by any reader, without trusting a self-declared flag: the marking is cryptographically bound to the producer and revocable on the same handle as every other claim. R1 remains binding on a human publisher of AI-generated content — the person's key stays the attester, and the class marking carries the origin.

R3 — The marking MUST survive egress. At a publish boundary where content leaves CEG for a non-CEG channel, the emitting node MUST re-express the marking in the destination's native machine-readable form: the CC 8.4.2.2 C2PA assertion for media (normative as of this section — that profile's `MAY` is promoted to `MUST` for generated media), and the Art. 50(1) interaction-disclosure limb for interactive text. Where a destination channel supports NO machine-readable marking, the node MUST still emit the human-legible disclosure AND MUST record the gap as a `hard_case:*` observation naming the channel — an unmarkable channel is a recorded exception, never a silent drop. A node MUST NOT strip a marking present on ingress.

R4 — Do not mark what was not generated. Assistive operations MUST NOT be marked. Marking everything is the same failure as marking nothing: a disclosure that fires on every spell-check carries no information and trains readers to ignore it. This bound is load-bearing, not a convenience — it is what keeps the marking a signal.

R5 — False or stripped marking is a false attestation; the substrate still does not adjudicate. Generated content published without the R1 marking by the party that generated it, or a marking stripped at egress, is a false attestation on the existing evidence floor: the substrate observes and emits (`hard_case:*`), the WA quorum owns the verdict — the same authority split as CC 3.4.5 and the CC 4.5.5 duty-holder rule. A downstream republisher that omits a marking it never received is an observation about the ingress path, not a verdict against the republisher: **the generator is the duty-holder**, which is also where Art. 50(2) puts the obligation.

Conformance pin + public statement. The R1–R2 path is **structural**: a CIRIS agent's

federation identity is minted with `identity_type` containing `agent`, so every Contribution it attests is machine-origin by construction, and the client renders agent-versus-human origin from the signed envelope — the disclosure *is* the attestation, read back, not a label the producer remembers to add. The R3 media form (CC 8.4.2.2 C2PA emit) is normative **wherever generated media is published**, and it MUST ship in the same release that first publishes generated media. A deployment claiming CC conformance MUST be able to name the release in which its marking path ships — and the claim resolves through the evidence registry, never by assertion: a conformance statement the pinned manifests cannot resolve is not one this document may cite.

3.5 Inter-attestation relations: the structural composition graph

Per CC 2.5 Inter-attestation-relations axis, attestations relate to each other in eight ways. Four are structural primitives (CC 2.4.1); the other four are emergent from scalar composition. The point of the split is economy: only four relations need a privileged structural slot; the rest fall out of how scores compose, so the wire stays minimal.

Relation	Realization
Standalone	Self-contained attestation; no `references_attestation_id`.
Refers-to-prior	Points to another attestation via `evidence_refs[]` or `context`; doesn't modify it. Emergent from independent positive scores on the same dimension+object.
Supersedes-prior	`supersedes` structural primitive (CC 2.4.1).
Contradicts-prior	Emergent from negative score on a dimension where a prior positive exists.
Withdraws-prior	`withdraws` structural primitive (CC 2.4.1).
Recants-prior	`recants` structural primitive (CC 2.4.1).
Clarifies-prior	Emergent from updated score with refined context on the same dimension+object.
Delegated	`delegates_to` structural primitive (CC 2.4.1).

3.5.1 Concurrent-write precedence

When two structural composers race on the same `references_attestation_id`, consumers MUST compute a deterministic verdict using the following precedence:

1. **`recants` outranks `withdraws` outranks `supersedes`** at the structural level. If the same attester emits multiple composers against the same prior attestation, `recants` wins regardless of `asserted_at` (a falsity admission cannot be subsumed by a retraction or replacement).
2. **For same-type concurrent emissions by the same attester**: the composer with the largest `asserted_at` per CC 2.6.2 wins.
3. **For same-`asserted_at` ties**: the composer whose row's `attestation_id` sorts lexicographically smallest wins.
4. **For cross-attester emissions on the same `references_attestation_id`**: each attester's chain is evaluated independently; the consumer sees N parallel chains and applies CC 4.4 policy.

Structural composers are **idempotent on `(references_attestation_id, attestation_type, attesting_key_id)`**: replaying the same composer is a no-op. The substrate MUST dedup on this triple.